

PROGRAMMIERUNG 2

Kapitel 2 : **R e k u r s i o n**



Prof. Dr. Markus Esch
htw saar

To understand *recursion*, one
must first understand *recursion*.

— *Stephen Hawking* —



Re|kur|si|on, die <lat.>:
siehe Rekursion

Rekursion: Definition durch Selbstreferenz

- ▶ *Oxford English Dictionary*:
“Relating to or involving the repeated application of a rule, definition, or procedure to successive results”

Rekursion



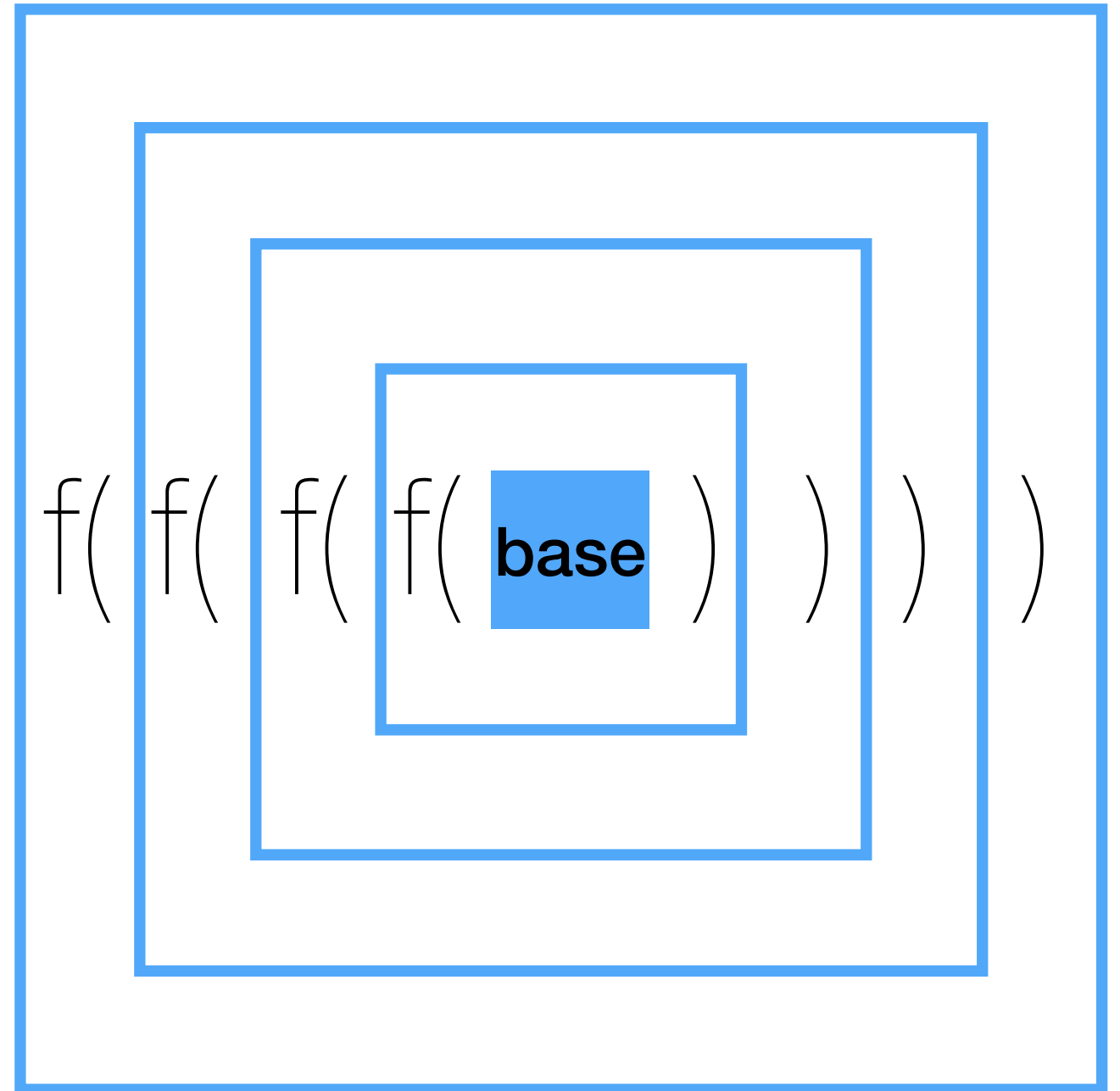
Rekursion: **Ingredients**

1. Basisfall (*Rekursionsanfang*)
2. Regelwerk welches jeden Fall auf den Basisfall reduziert
(*Rekursionsschritt*)



Rekursion: **Ingredients**

1. Basisfall (*Rekursionsanfang*)
2. Regelwerk welches jeden Fall auf den Basisfall reduziert
(*Rekursionsschritt*)



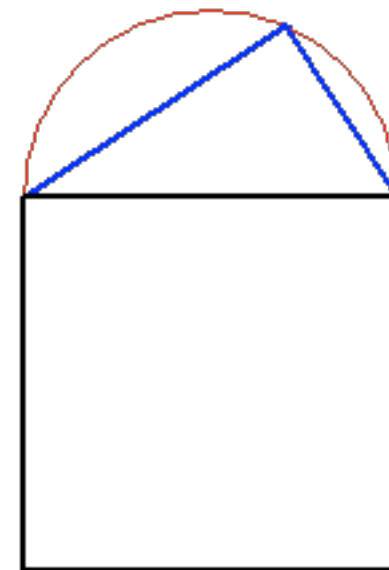
Beispiel **Pythagoras-Baum**

1. Basisfall: Gegebenes Quadrat
2. Regelwerk:
 - ▶ Zeichne Dreieck mit
gegebener Höhe und
Winkeln auf der Oberseite.
 - ▶ Errichte ein Quadrat über
den Schenkeln des
Dreiecks



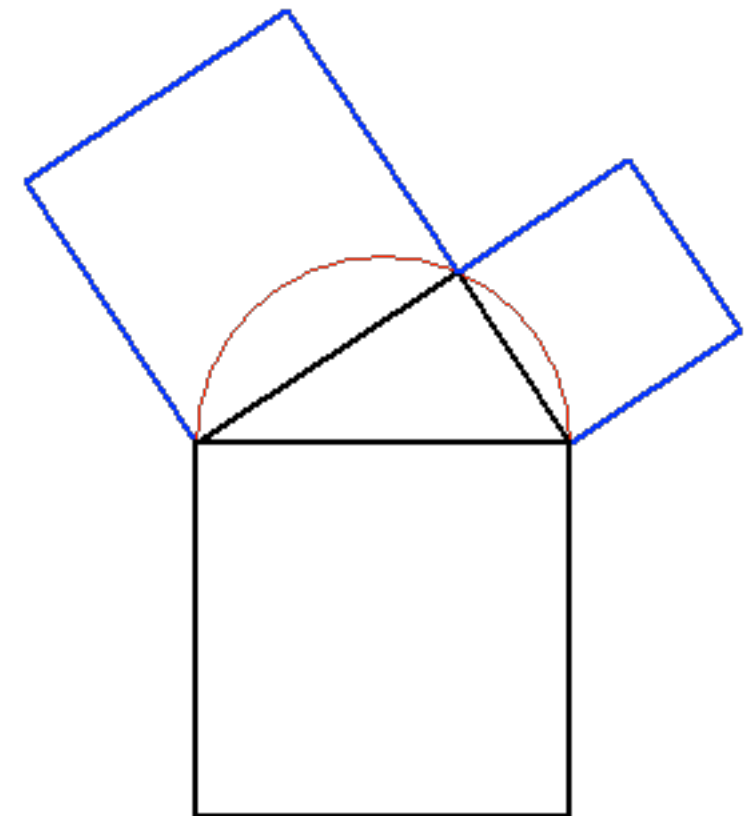
Beispiel **Pythagoras-Baum**

1. Basisfall: Gegebenes Quadrat
2. Regelwerk:
 - ▶ Zeichne Dreieck mit gegebener Höhe und Winkeln auf der Oberseite.
 - ▶ Errichte ein Quadrat über den Schenkeln des Dreiecks



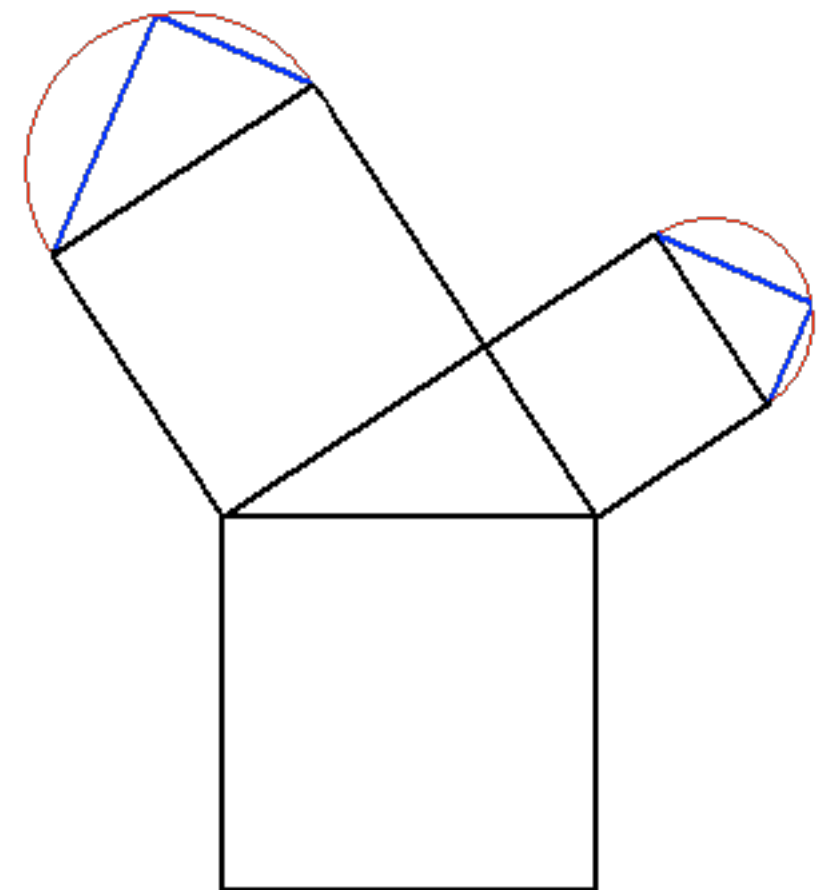
Beispiel **Pythagoras-Baum**

1. Basisfall: Gegebenes Quadrat
2. Regelwerk:
 - ▶ Zeichne Dreieck mit gegebener Höhe und Winkeln auf der Oberseite.
 - ▶ Errichte ein Quadrat über den Schenkeln des Dreiecks



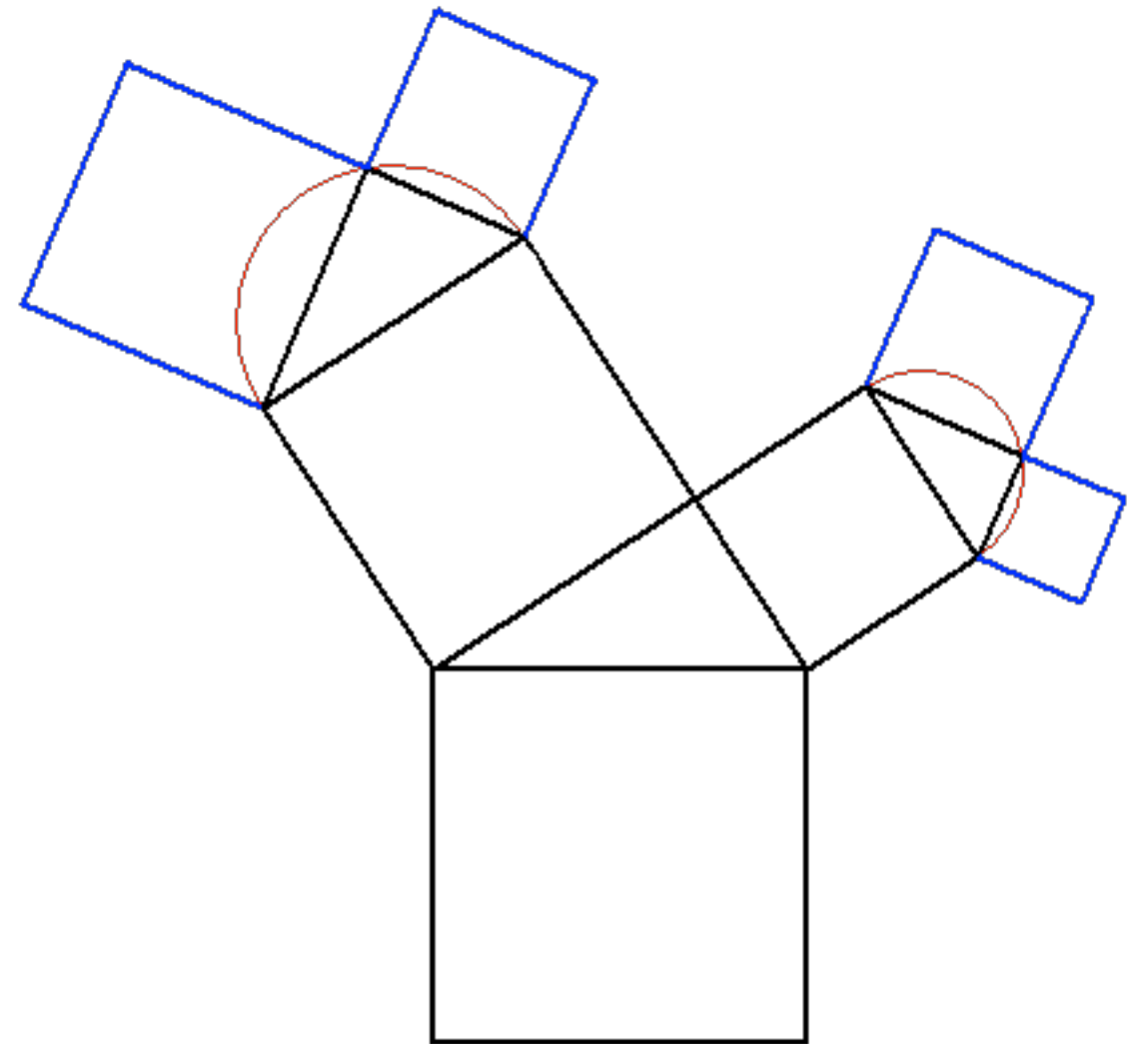
Beispiel **Pythagoras-Baum**

1. Basisfall: Gegebenes Quadrat
2. Regelwerk:
 - ▶ Zeichne Dreieck mit gegebener Höhe und Winkeln auf der Oberseite.
 - ▶ Errichte ein Quadrat über den Schenkeln des Dreiecks

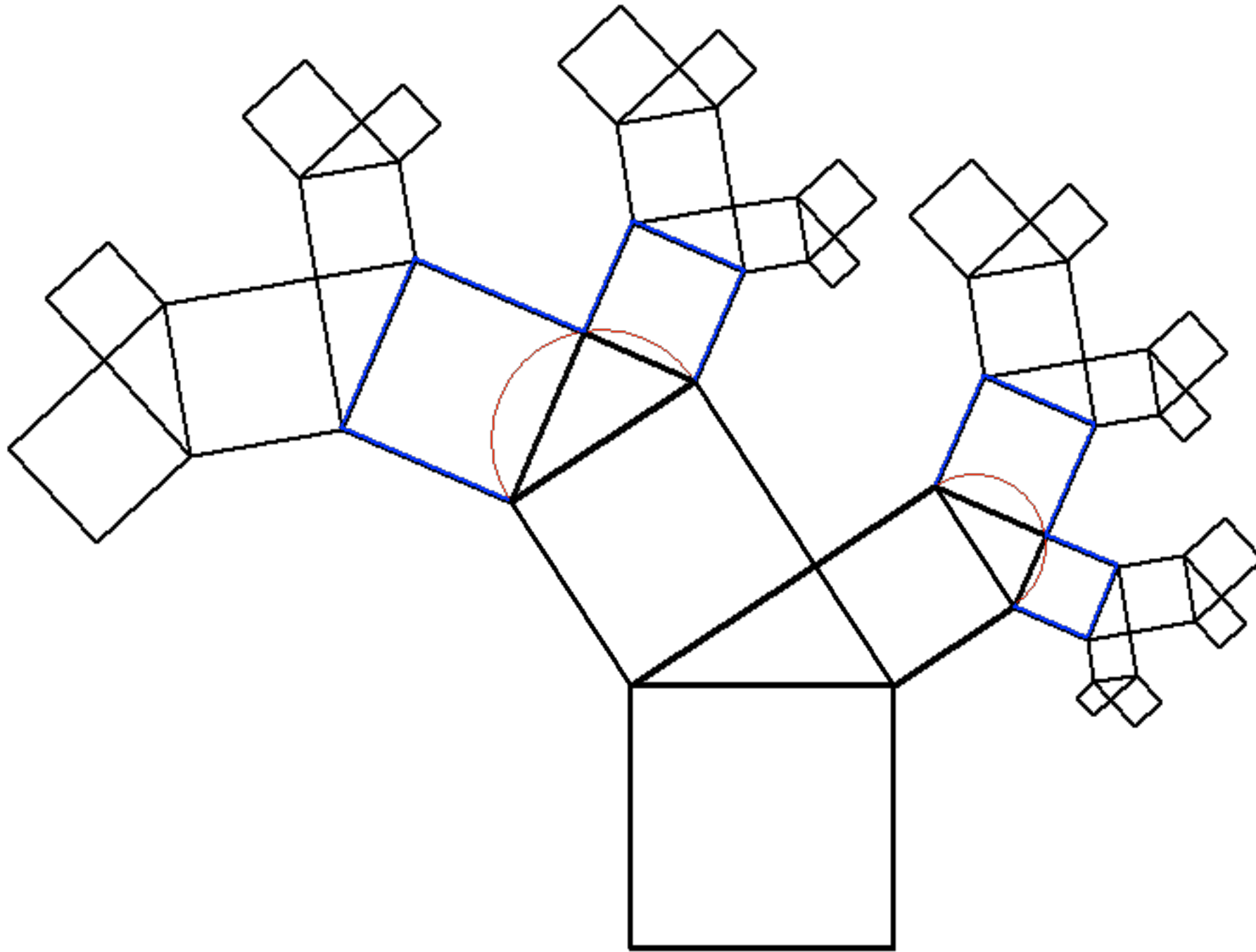


Beispiel **Pythagoras-Baum**

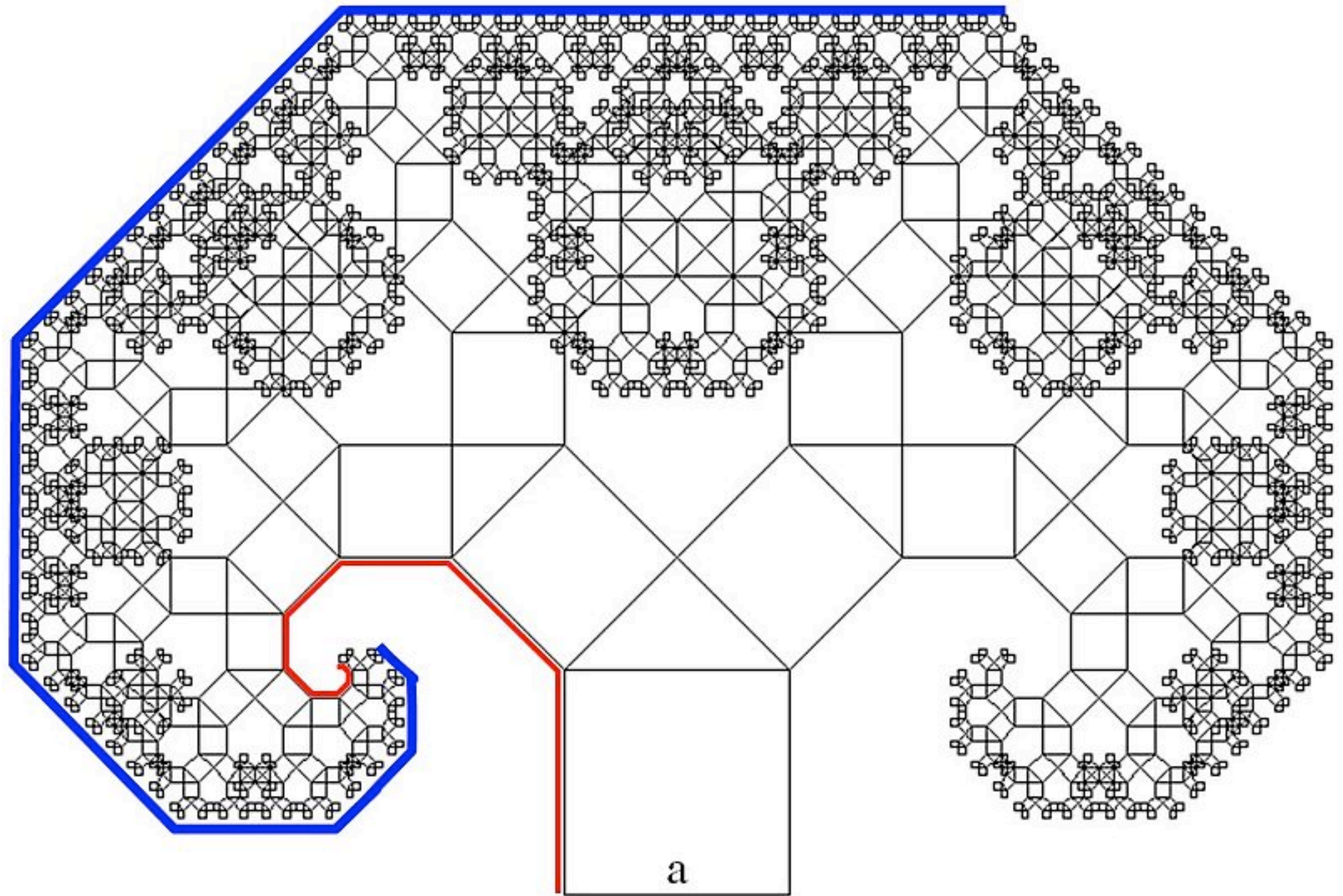
1. Basisfall: Gegebenes Quadrat
2. Regelwerk:
 - ▶ Zeichne Dreieck mit gegebener Höhe und Winkeln auf der Oberseite.
 - ▶ Errichte ein Quadrat über den Schenkeln des Dreiecks



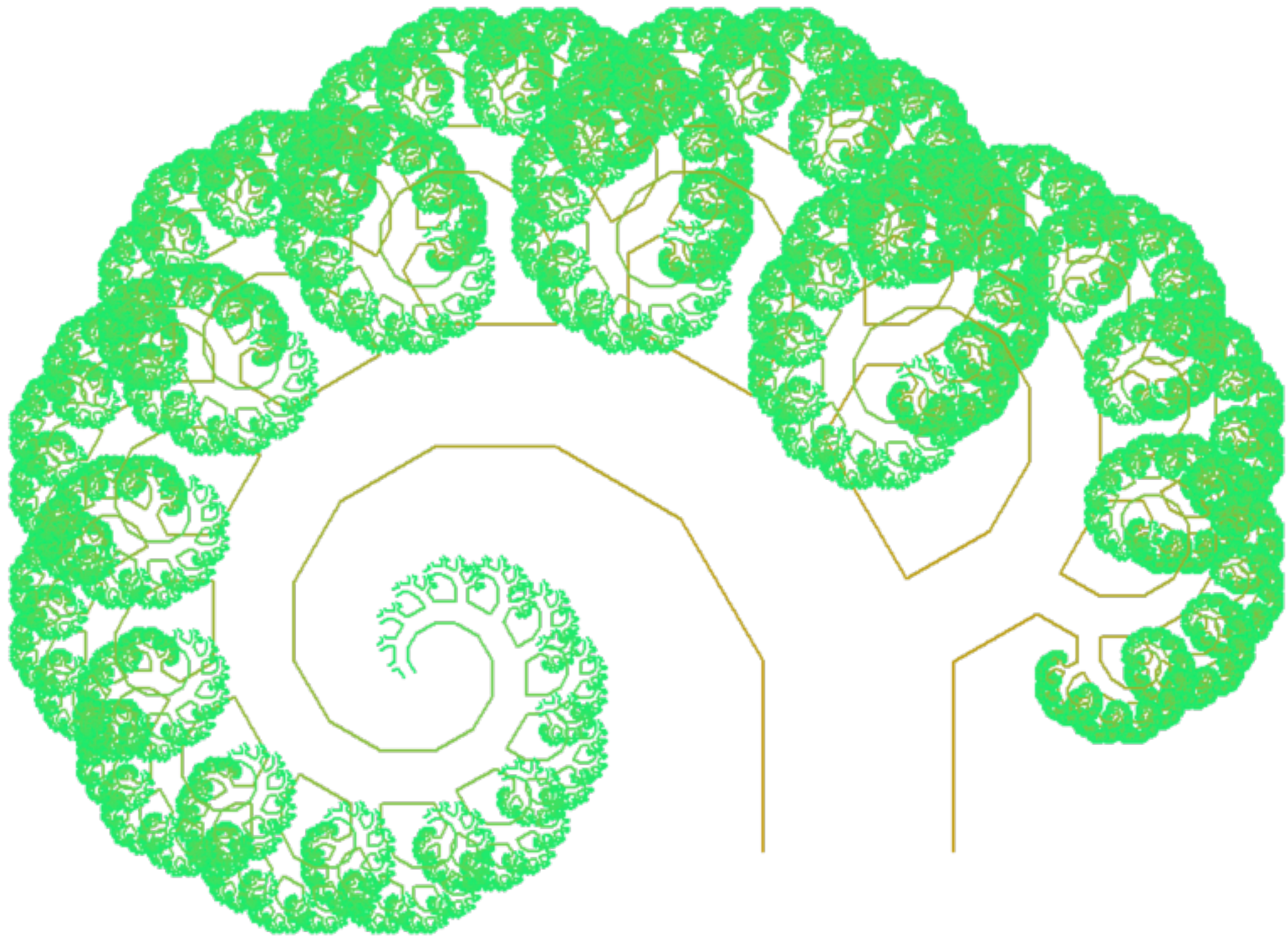
Beispiel **Pythagoras-Baum**



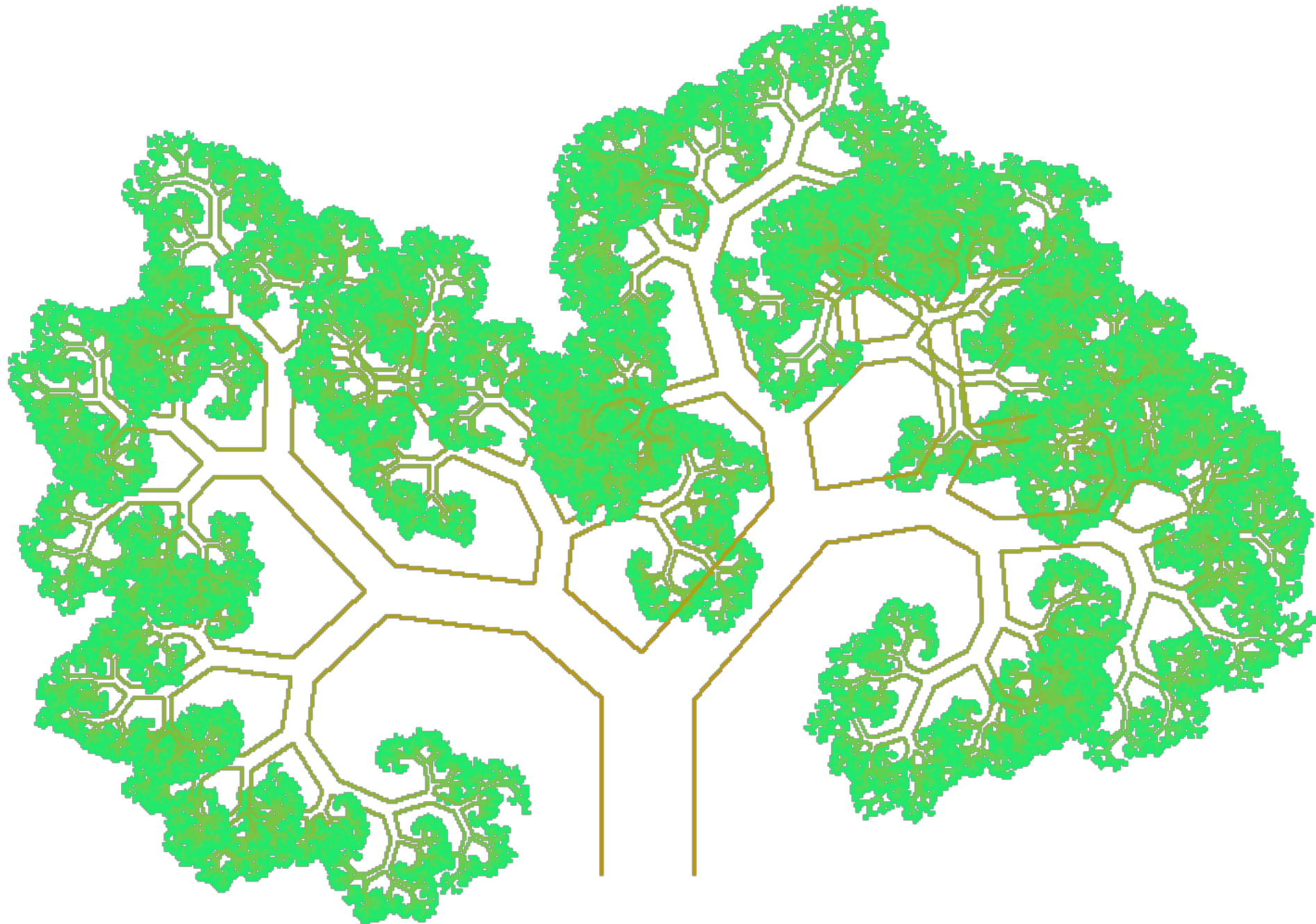
Beispiel **Pythagoras-Baum**



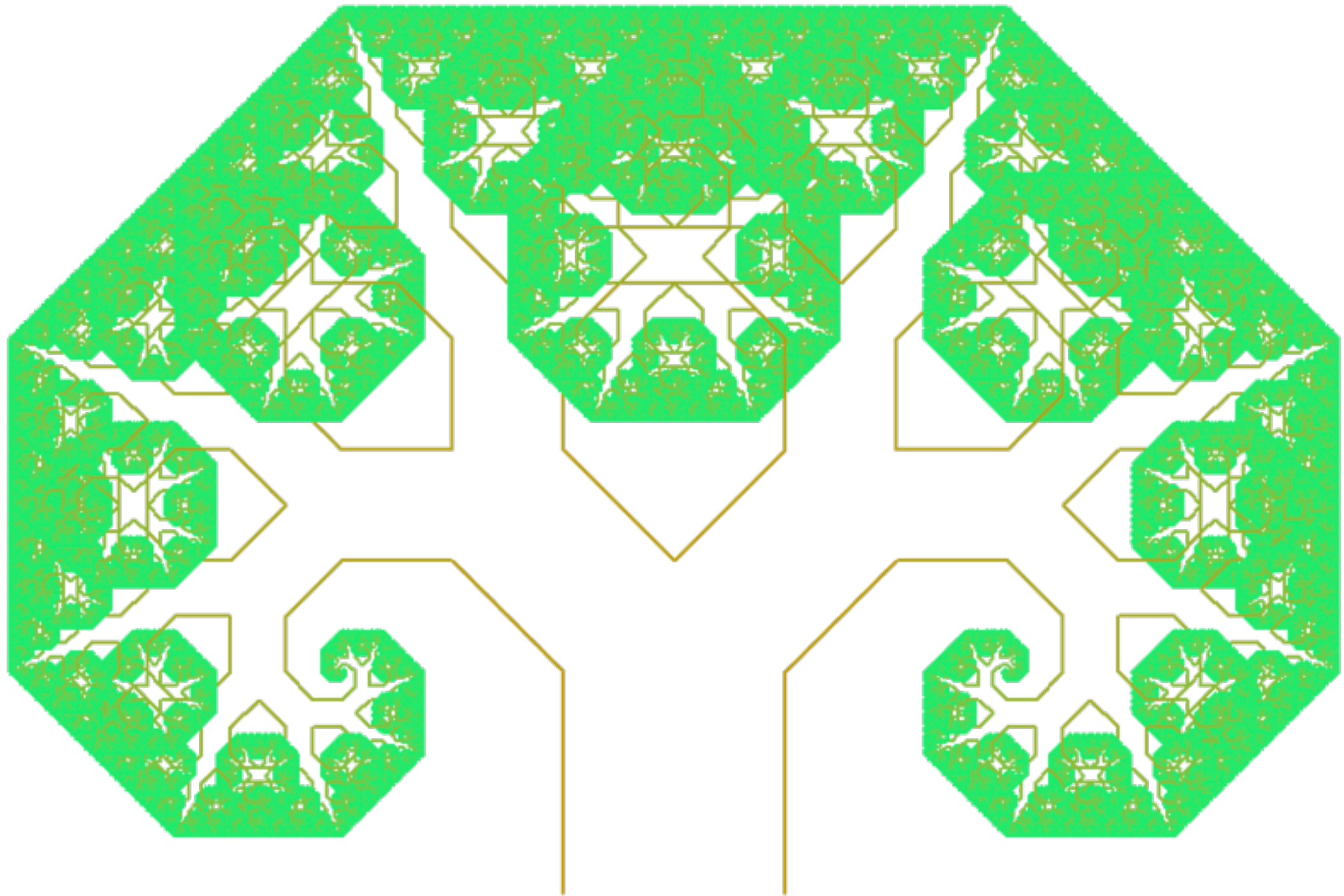
Beispiel **Pythagoras-Baum**



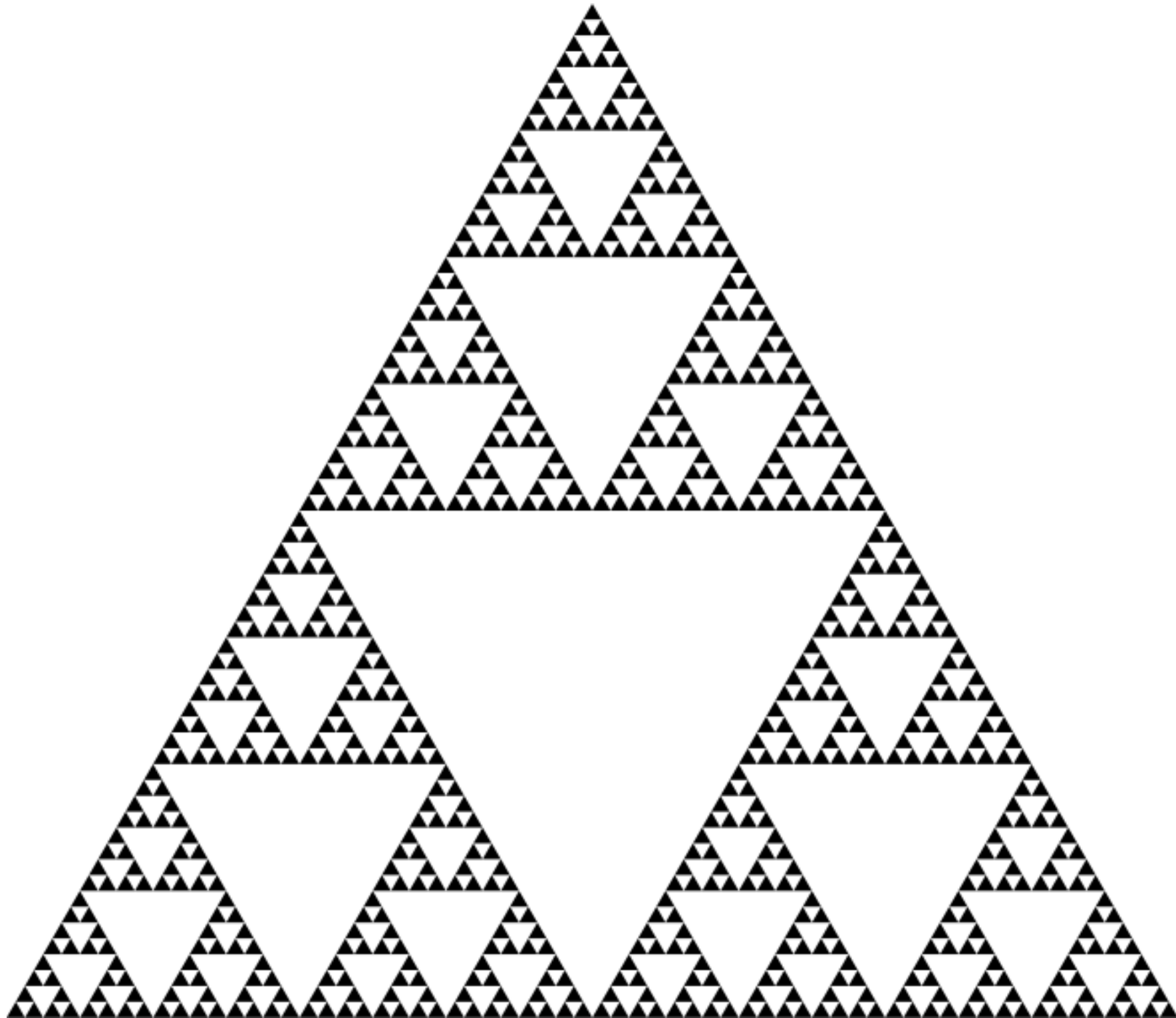
Beispiel **Pythagoras-Baum**



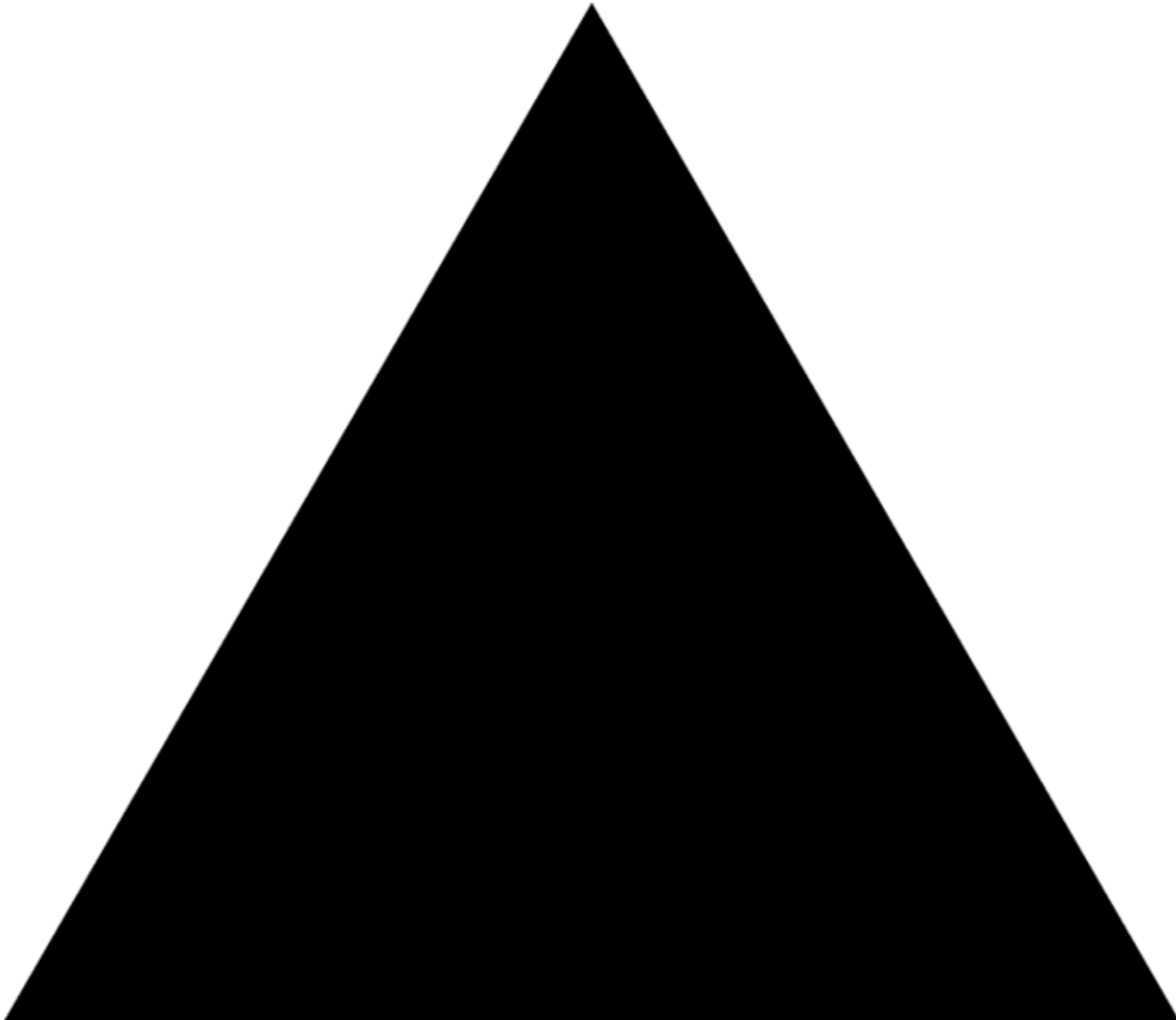
Beispiel **Pythagoras-Baum**



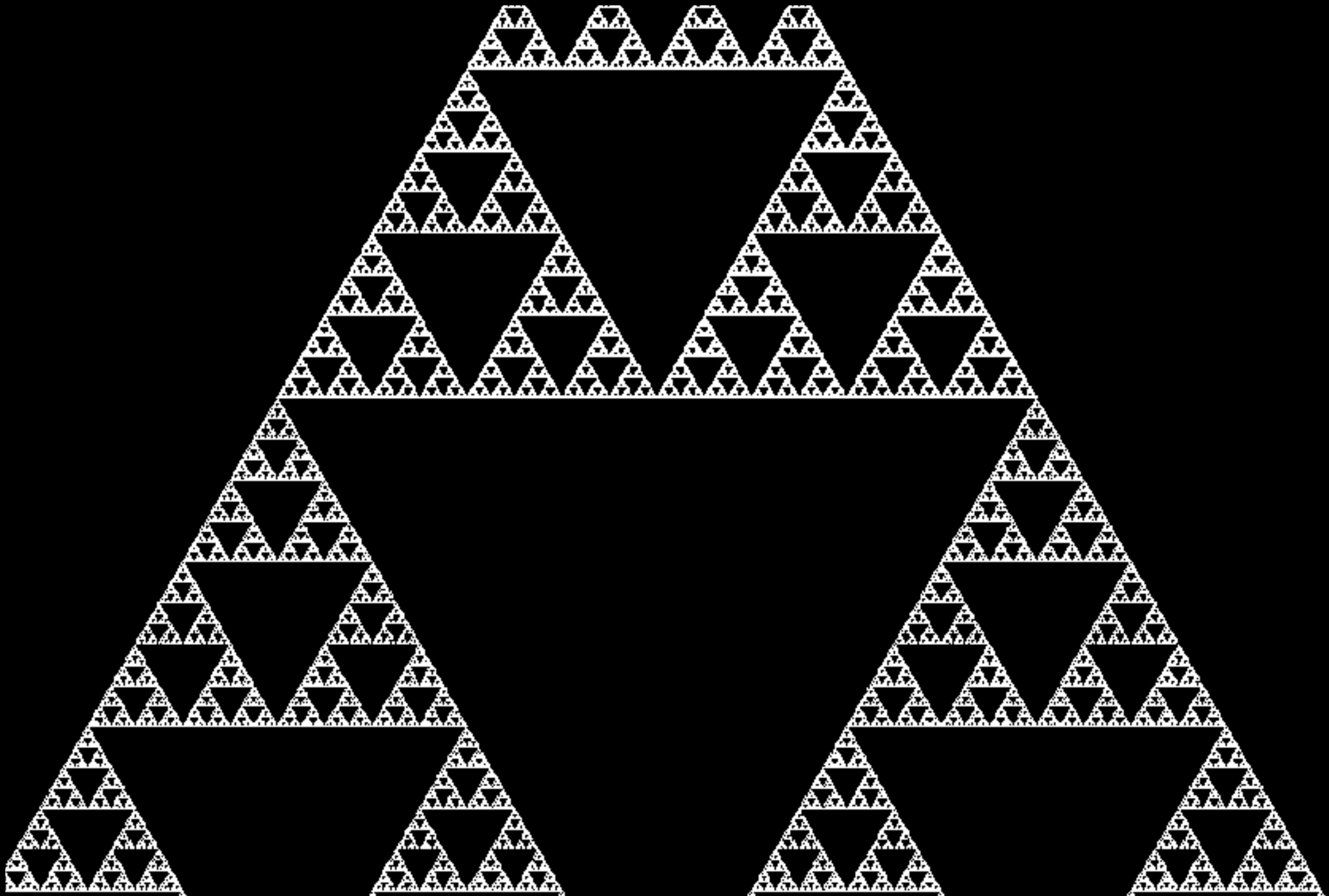
Beispiel **Sierpinski-Dreieck**



Beispiel **Sierpinski-Dreieck**



Selbstähnlichkeit



Rekursive **Funktionen**

- ▶ Beispiel: Summenfunktion

- ▶ Definition:

- ▶ $sum(n) = \sum_{i=0}^n i$

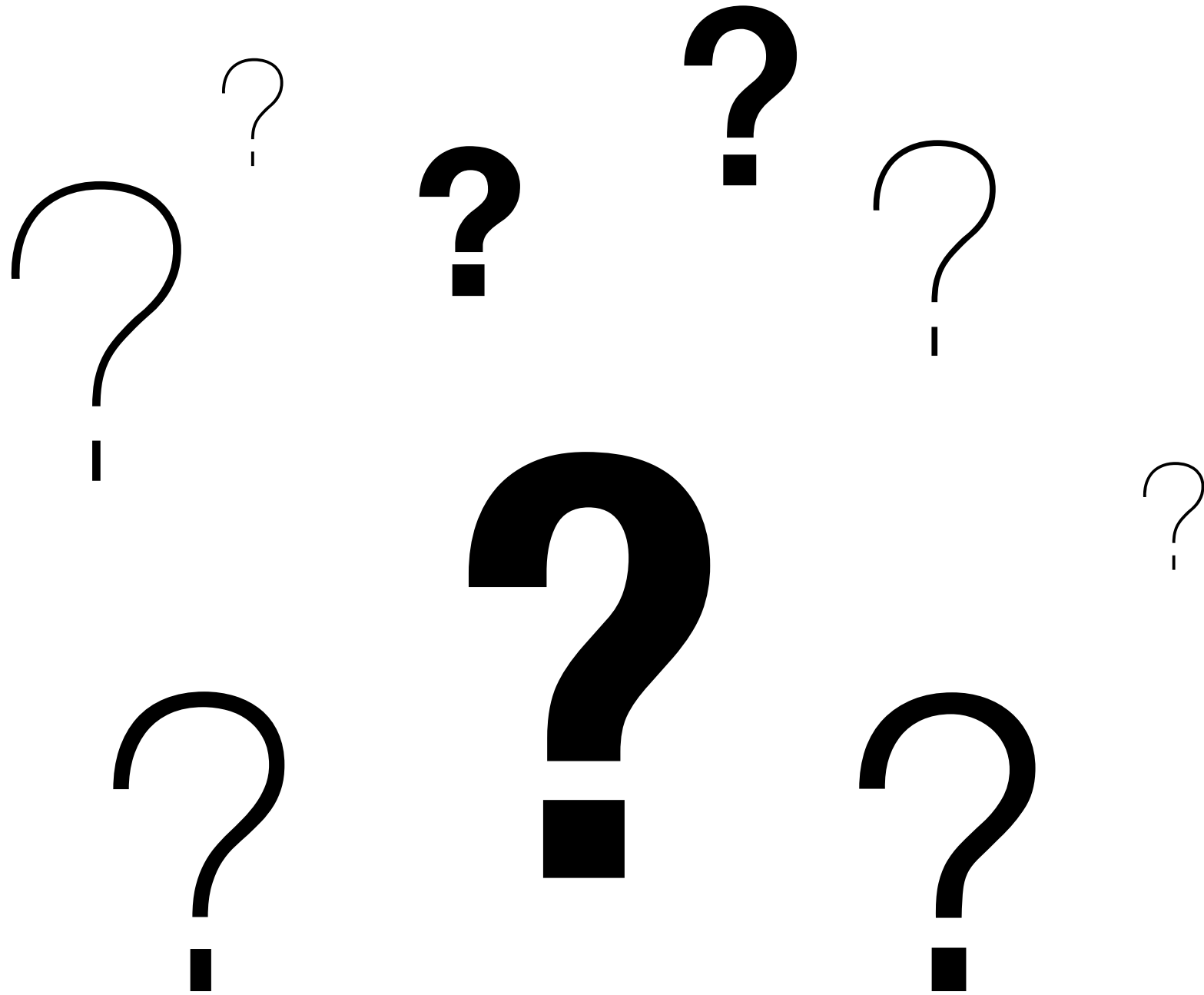
Rekursionsanfang

- ▶ rekursive Definition:

- ▶ $sum(n) = \begin{cases} 0 & \text{falls } n = 0 \\ sum(n - 1) + n & \text{sonst} \end{cases}$

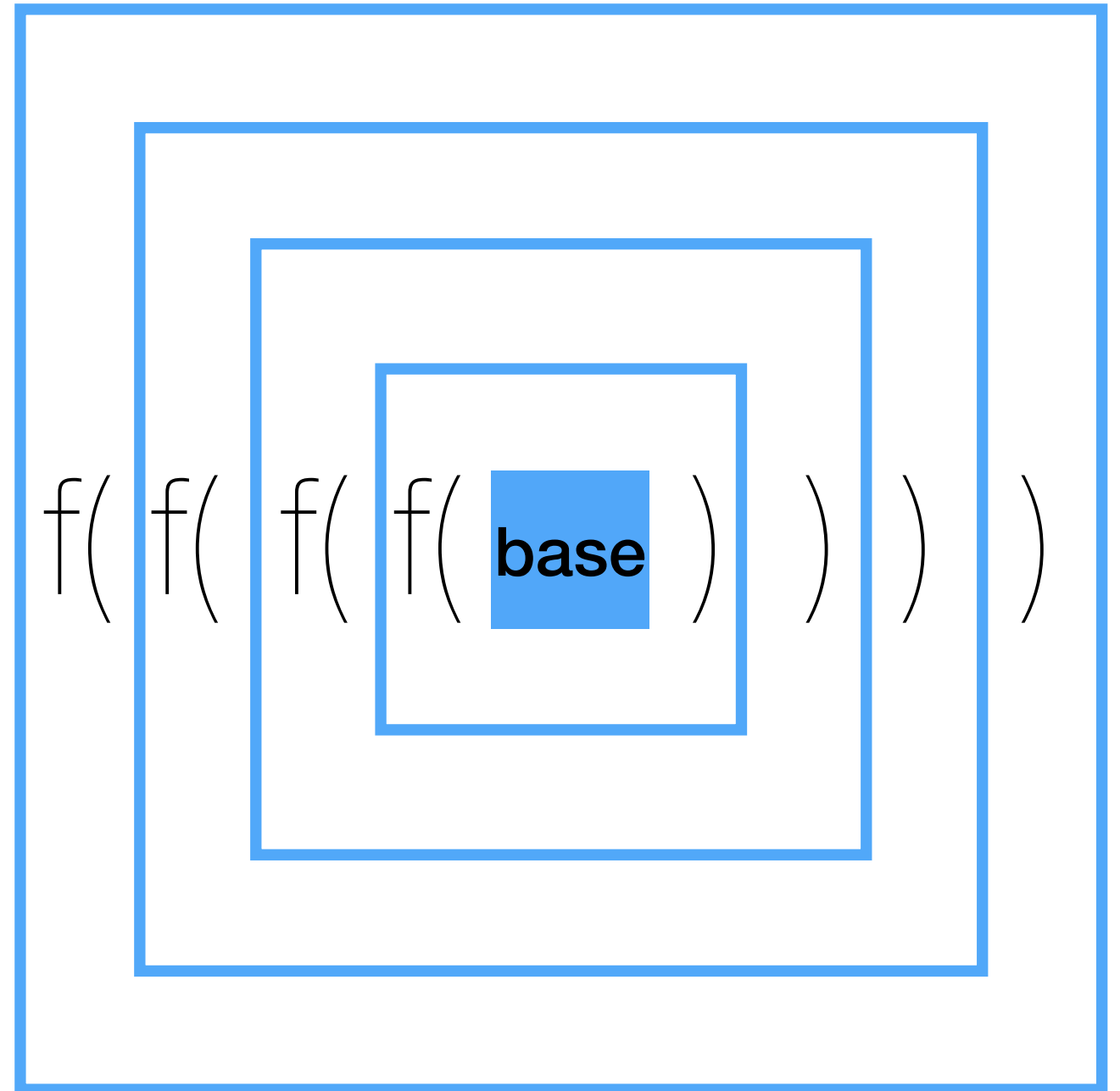
Rekursionsschritt

Berechnung **Rekursiver Funktionen**

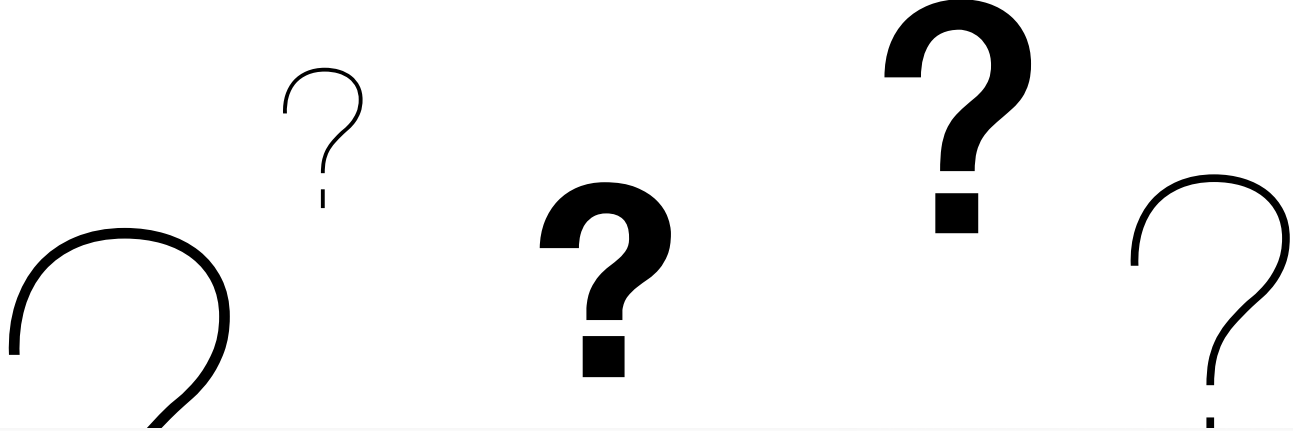


Berechnung **Rekursiver Funktionen**

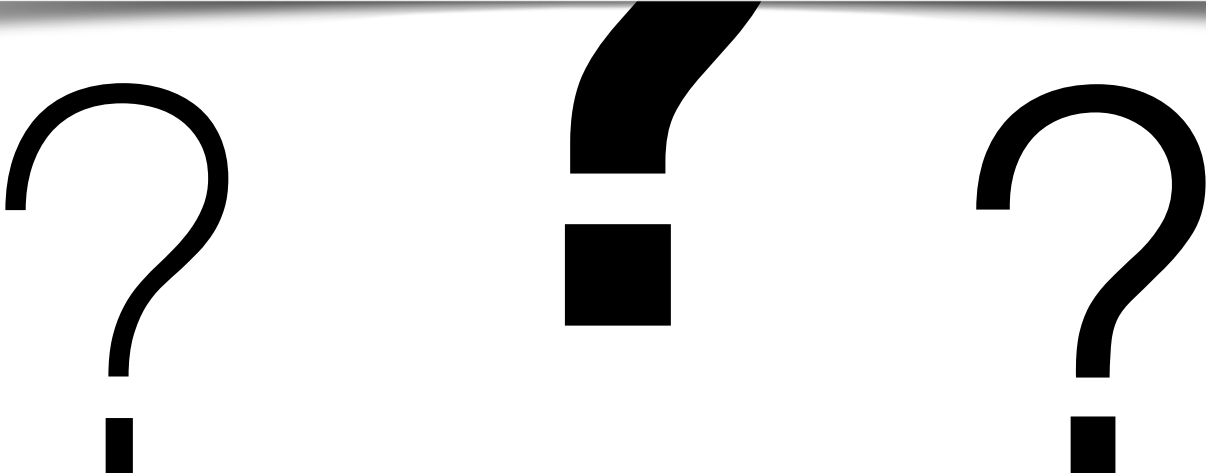
- Rückführung auf den Rekursionsanfang (Basisfall)



Berechnung **Rekursiver Funktionen**



Four question marks of varying sizes and styles are arranged in a row above the equation box.

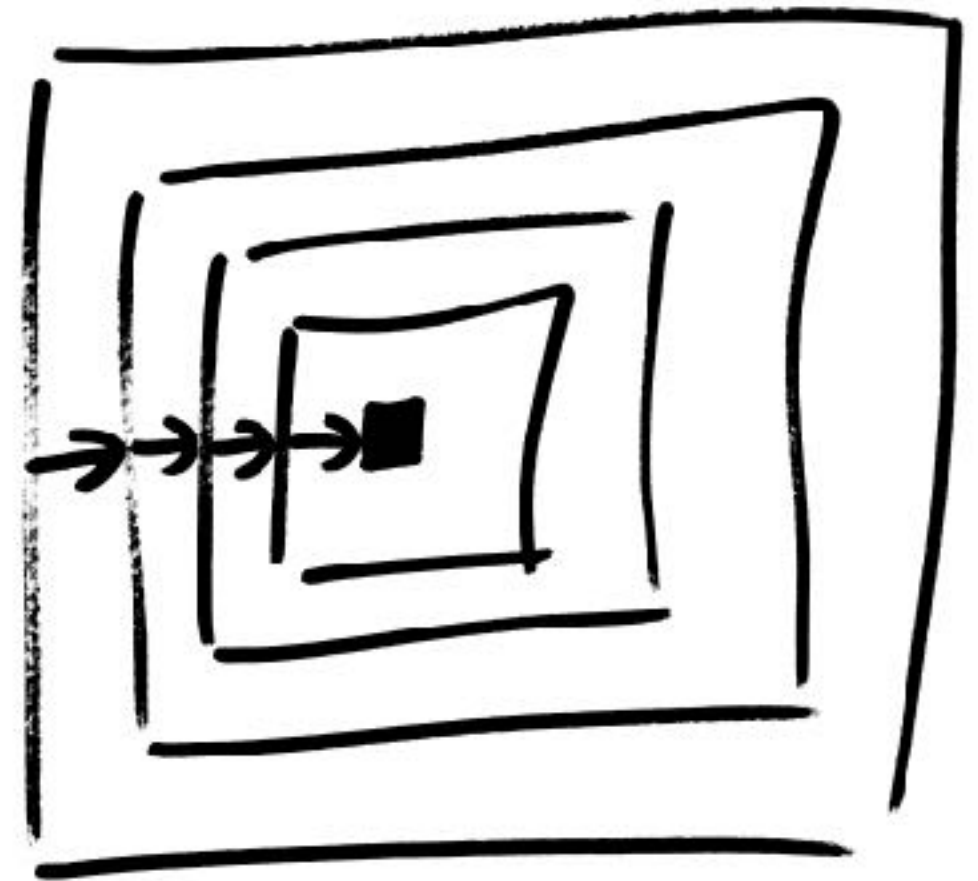
$$sum(n) = \begin{cases} 0 & \text{falls } n = 0 \\ sum(n - 1) + n & \text{sonst} \end{cases}$$


Three large, bold question marks are arranged in a row below the equation box.

Iterativ



VS.



Rekursiv

Iterative Implementierung

$$sum(n) = \begin{cases} 0 & \text{falls } n = 0 \\ sum(n - 1) + n & \text{sonst} \end{cases}$$

```
public int calculateSum(int n){
```

?

```
}
```



Iterative Implementierung

$$sum(n) = \begin{cases} 0 & \text{falls } n = 0 \\ sum(n - 1) + n & \text{sonst} \end{cases}$$

```
public int calculateSum(int n){  
    int result = 0;  
    for(int i = n; i>0; i--)  
    {  
        result += i;  
    }  
    return result;  
}
```


Iterative Implementierung

$$sum(n) = \begin{cases} 0 & \text{falls } n = 0 \\ sum(n - 1) + n & \text{sonst} \end{cases}$$

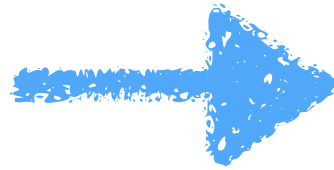
```
public int calculateSum(int n){  
    int result = 0;  
    for(int i = n; i>0; i--)  
    {  
        result += i;  
    }  
    return result;  
}
```

Rekursionsanfang

Rekursionsschritt

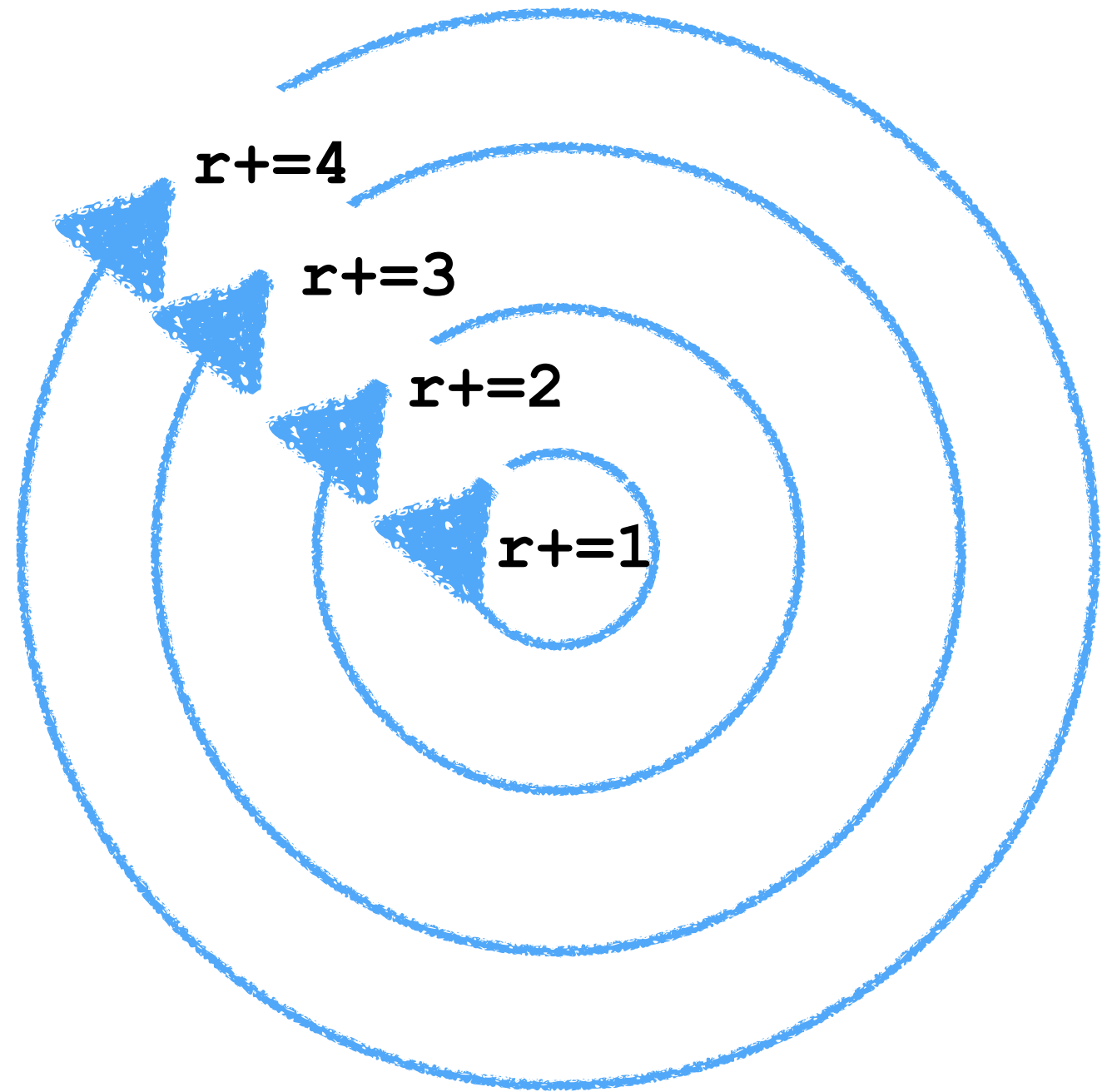
Iterative Implementierung

$r = 0$



$n = 4;$
 $r = 10;$

```
public int calculateSum(int n){  
    int result = 0;  
    for(int i = n; i>0; i--)  
    {  
        result += i;  
    }  
    return result;  
}
```



Rekursive Implementierung

$$sum(n) = \begin{cases} 0 & \text{falls } n = 0 \\ sum(n - 1) + n & \text{sonst} \end{cases}$$

```
public int calculateSum(int n){
```

?

```
}
```



Rekursive Implementierung

$$sum(n) = \begin{cases} 0 & \text{falls } n = 0 \\ sum(n - 1) + n & \text{sonst} \end{cases}$$

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

Rekursive Implementierung

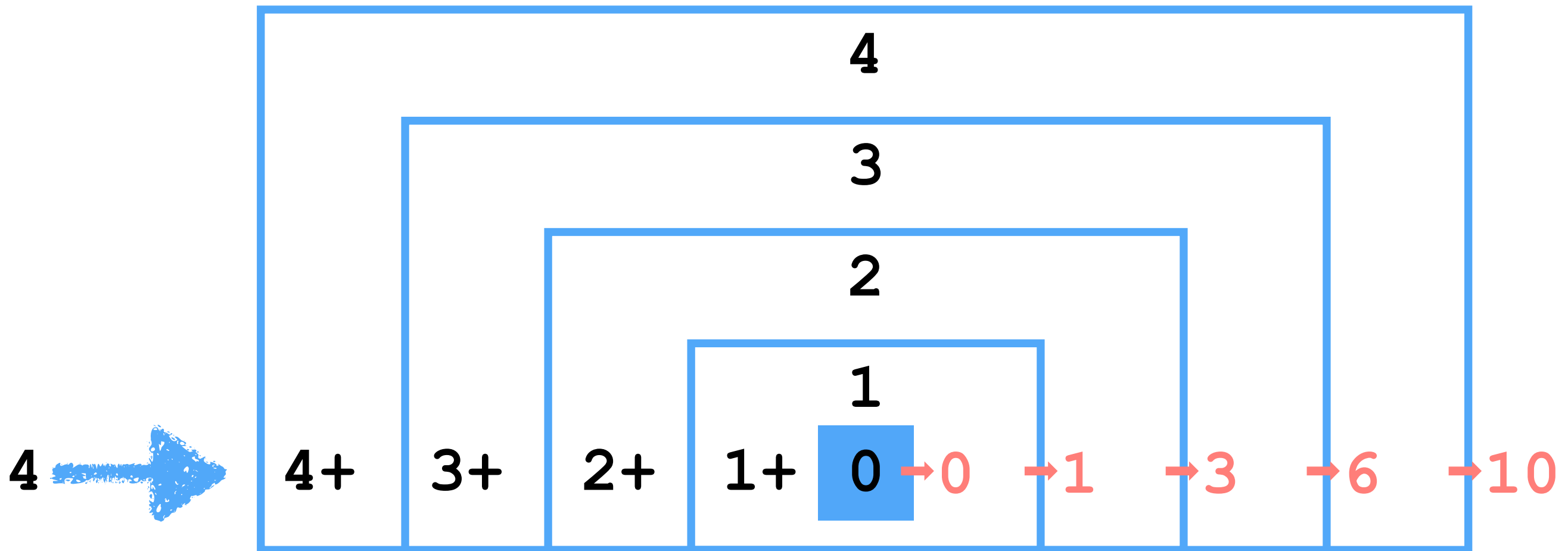
$$sum(n) = \begin{cases} 0 & \text{falls } n = 0 \\ sum(n - 1) + n & \text{sonst} \end{cases}$$

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

Rekursionsanfang

Rekursionsschritt

Rekursive Implementierung



```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

Rekursive Implementierung

Template

```
public int recursive(. . .){  
    if(Rekursionsanfang)  
        return Anfangswert;  
    else  
        return Rekursionsschritt;  
}
```

”Wo” findet die Berechnung statt?

Iterativ

eine Lokale Variable

```
public int calculateSum(int n){  
    int result = 0;  
    for(int i = n; i>0; i--)  
    {  
        result += i;  
    }  
    return result;  
}
```

”Wo” findet die Berechnung statt?



Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

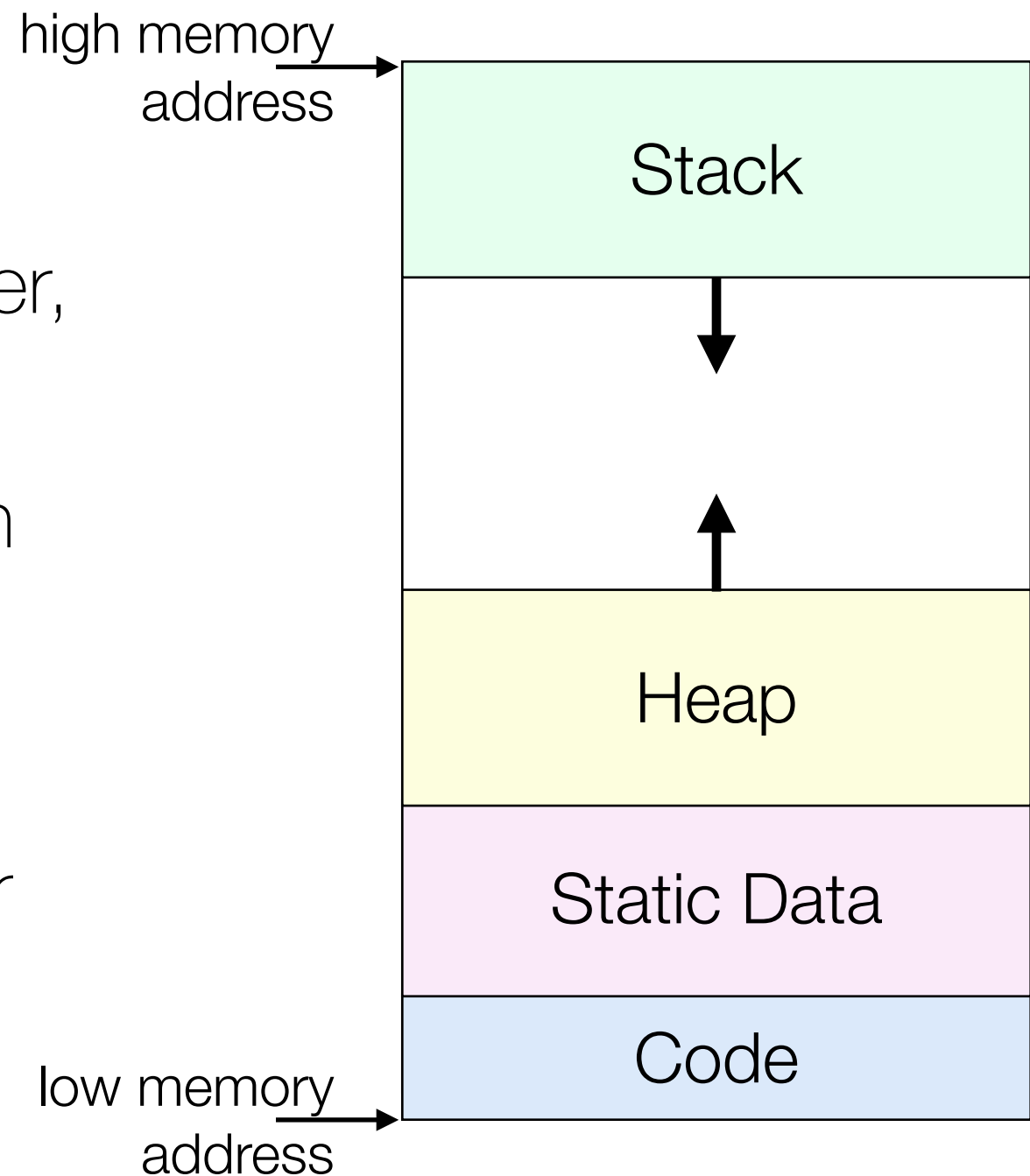
Exkurs: **Stack** und **Heap**

- ▶ Stack

- ▶ lokale Variablen, Parameter, Rückgabewerte, Rücksprungadressen von Methoden

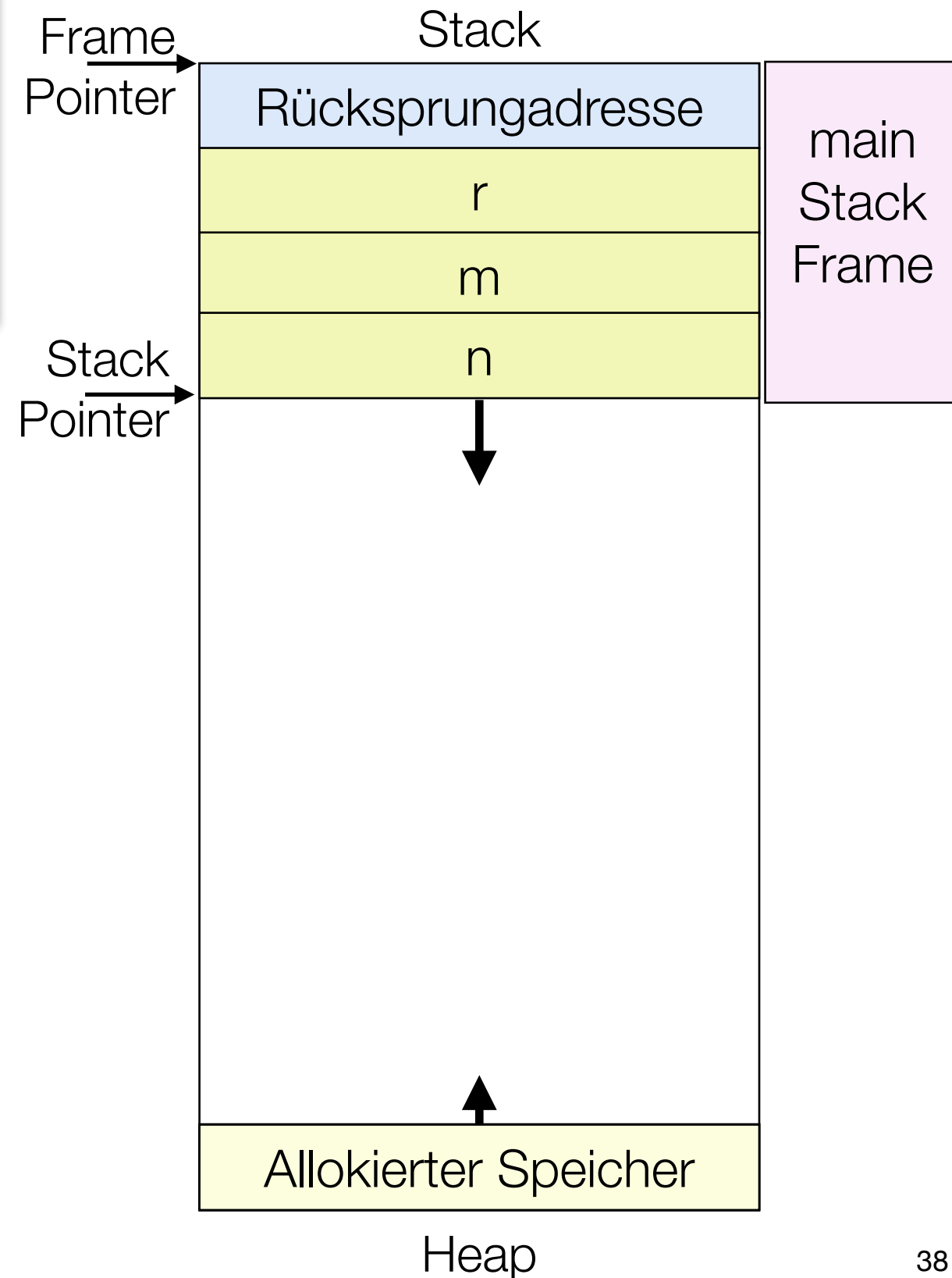
- ▶ Heap Space

- ▶ dynamischer Speicher für Referenzvariablen



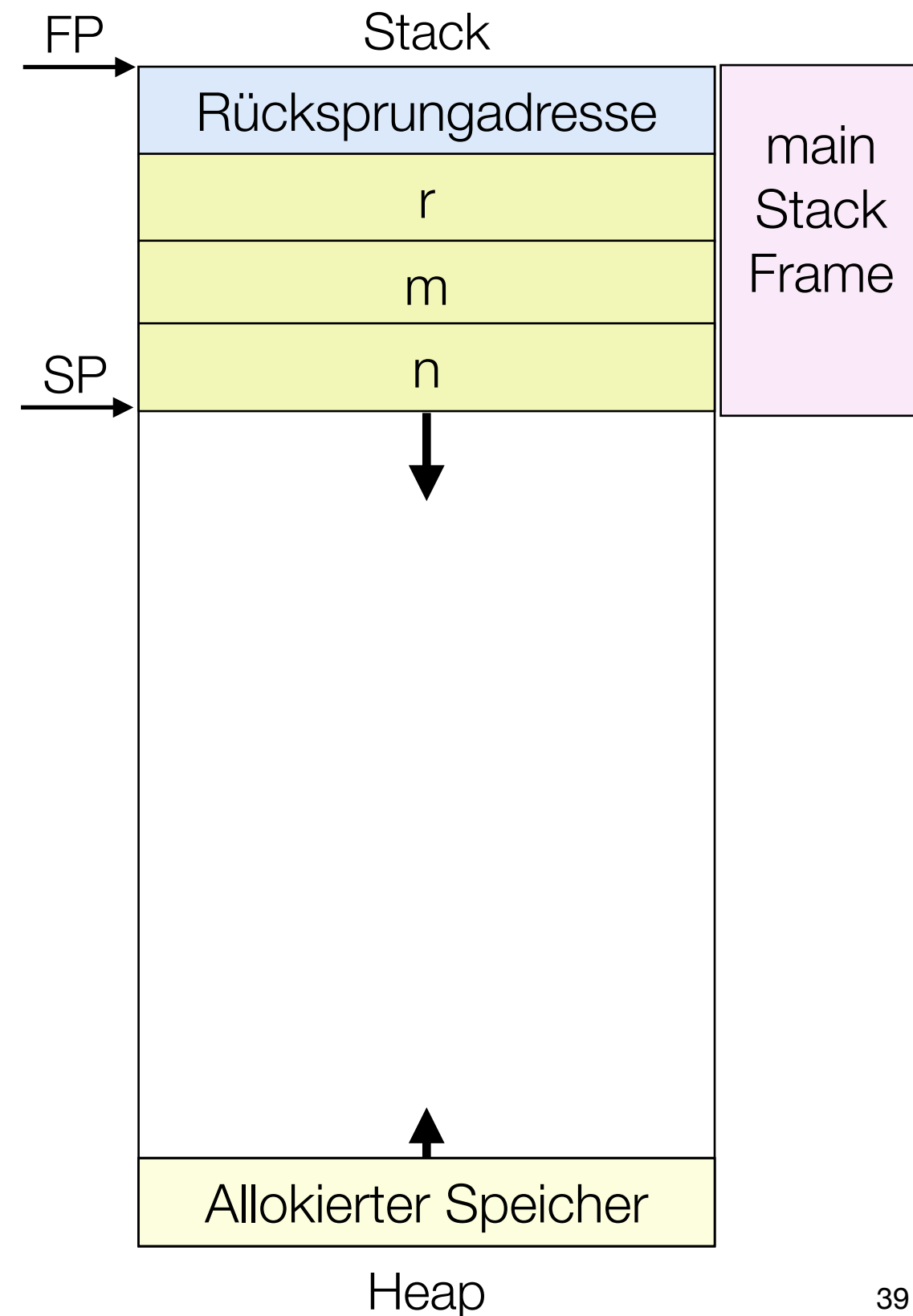
Exkurs: **Stack** und **Heap**

```
public static void main() {  
    int n = 21;  
    int m = 21;  
    int r = 0;  
}
```



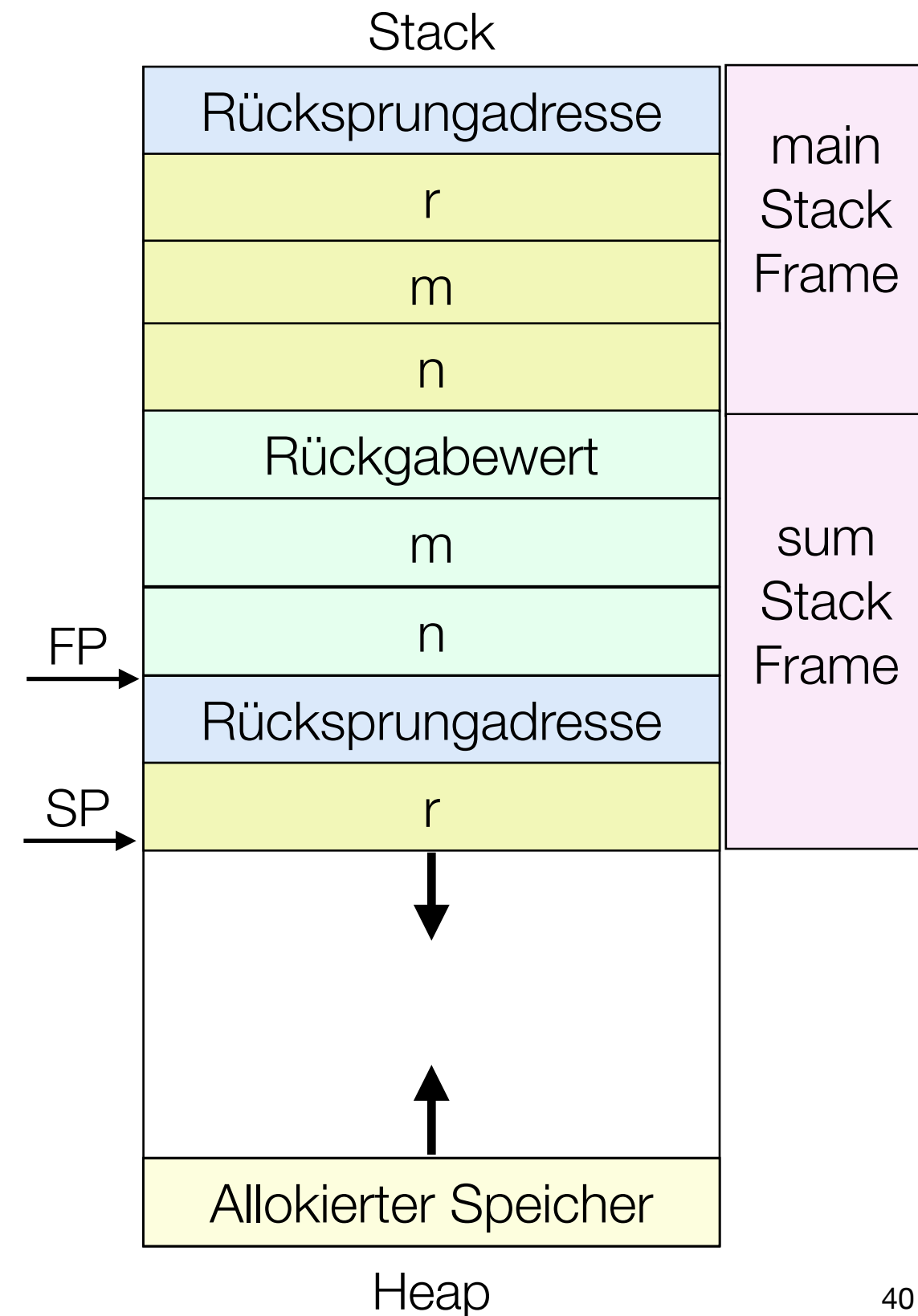
Exkurs: **Stack** und **Heap**

```
public static void main() {  
    int n = 21;  
    int m = 21;  
    int r = 0;  
    r = sum (n, m);  
}  
  
public static boolean truth(){  
    return true;  
}  
  
public static int sum(int n, int m){  
    int r = -1;  
    if(truth()){  
        r = n + m;  
    }  
    return r;  
}
```



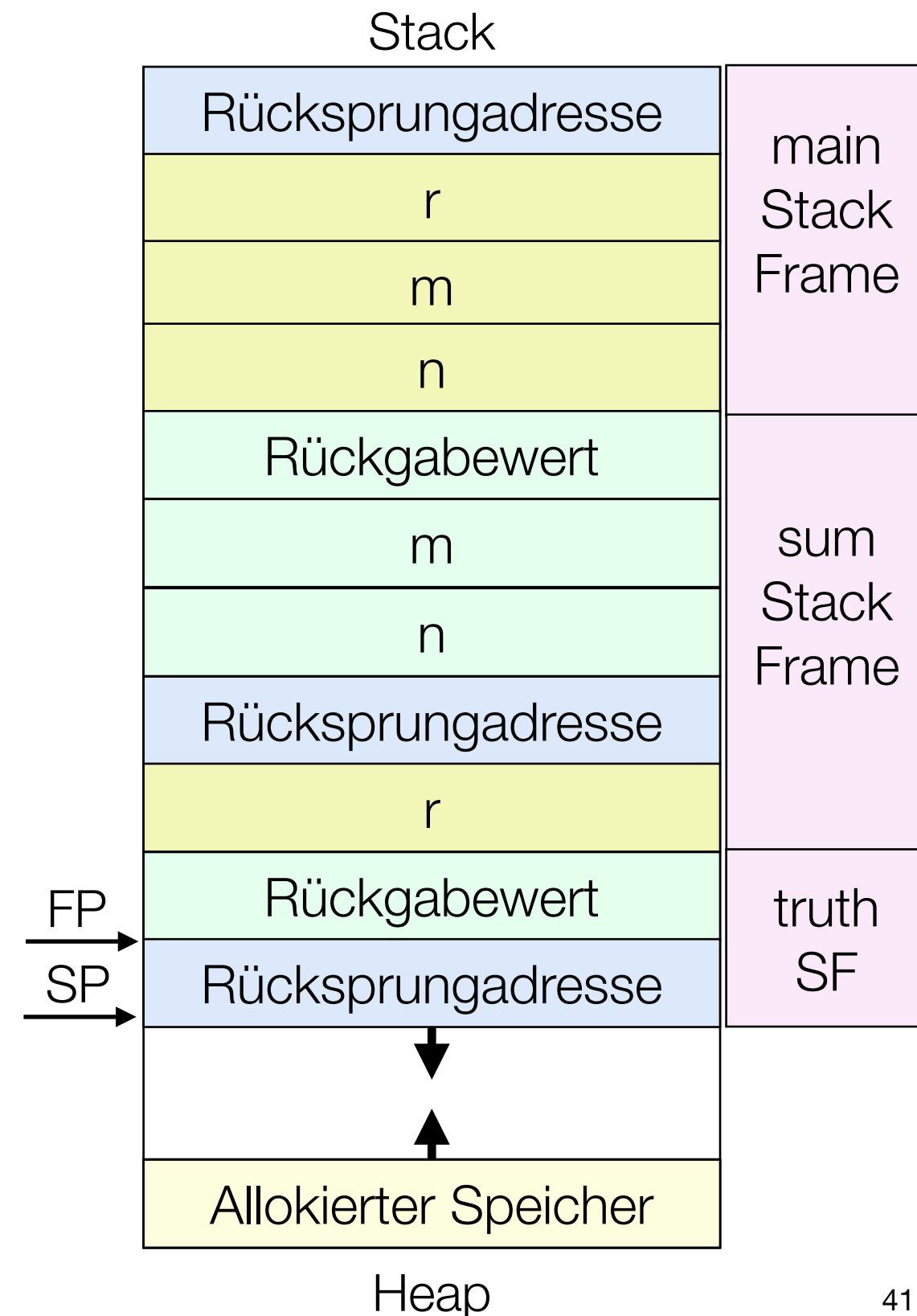
Exkurs: **Stack** und **Heap**

```
public static void main() {  
    int n = 21;  
    int m = 21;  
    int r = 0;  
    r = sum (n, m);  
}  
  
public static boolean truth(){  
    return true;  
}  
  
public static int sum(int n, int m){  
    int r = -1;  
    if(truth()){  
        r = n + m;  
    }  
    return r;  
}
```



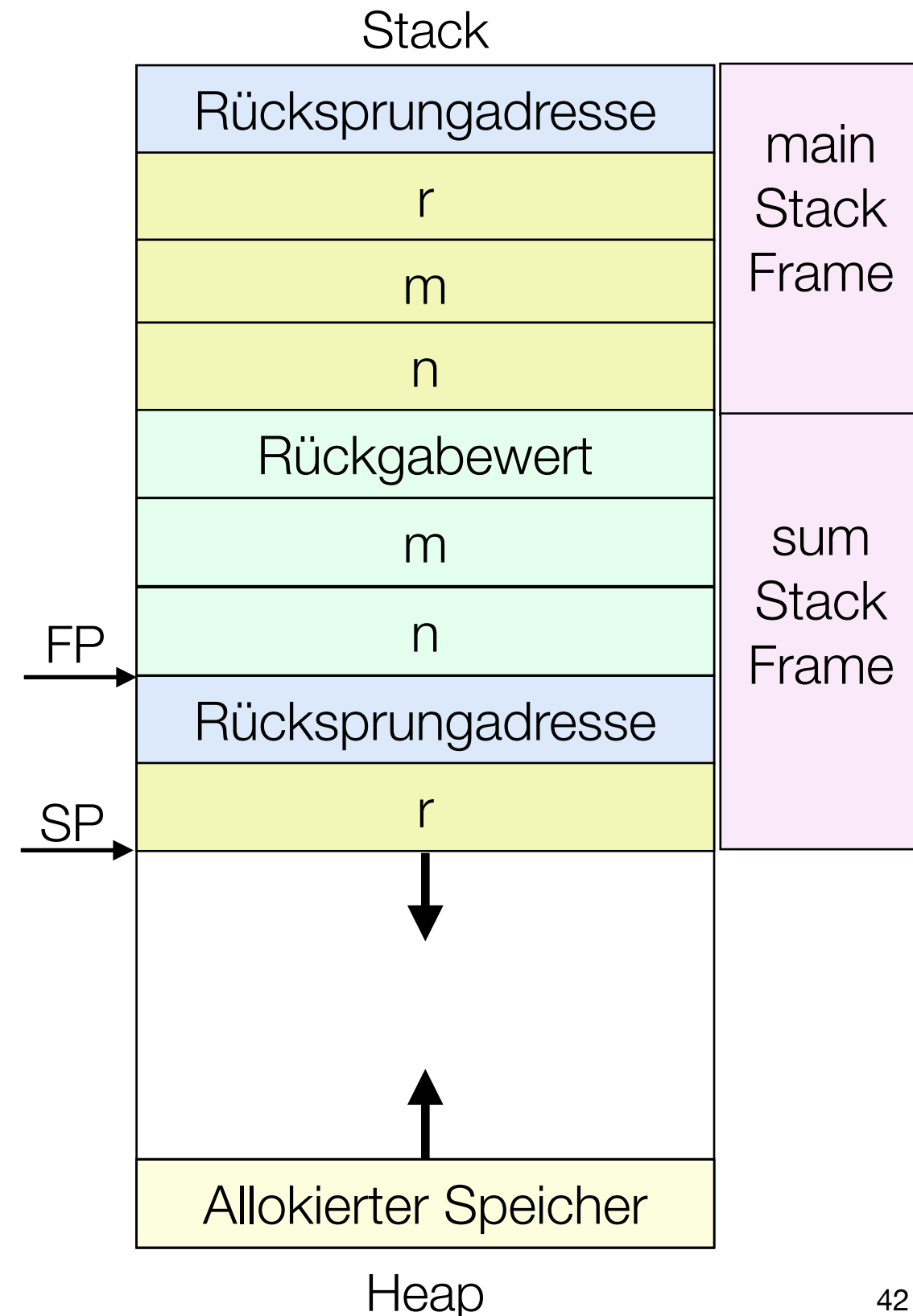
Exkurs: **Stack** und **Heap**

```
public static void main() {  
    int n = 21;  
    int m = 21;  
    int r = 0;  
    r = sum (n, m);  
}  
  
public static boolean truth(){  
    return true;  
}  
  
public static int sum(int n, int m){  
    int r = -1;  
    if(truth()){  
        r = n + m;  
    }  
    return r;  
}
```



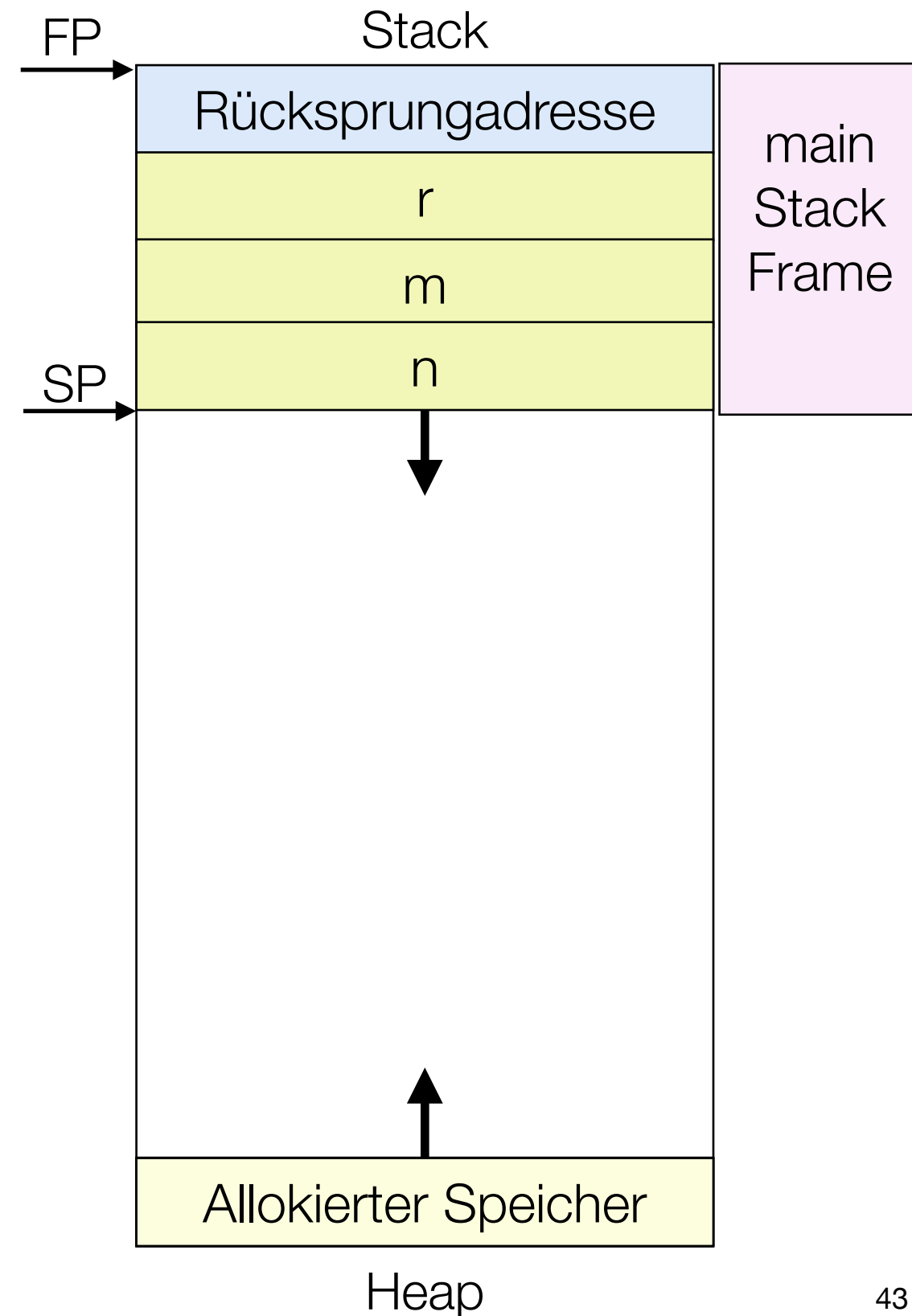
Exkurs: **Stack** und **Heap**

```
public static void main() {  
    int n = 21;  
    int m = 21;  
    int r = 0;  
    r = sum (n, m);  
}  
  
public static boolean truth(){  
    return true;  
}  
  
public static int sum(int n, int m){  
    int r = -1;  
    if(truth()){  
        r = n + m;  
    }  
    return r;  
}
```



Exkurs: **Stack** und **Heap**

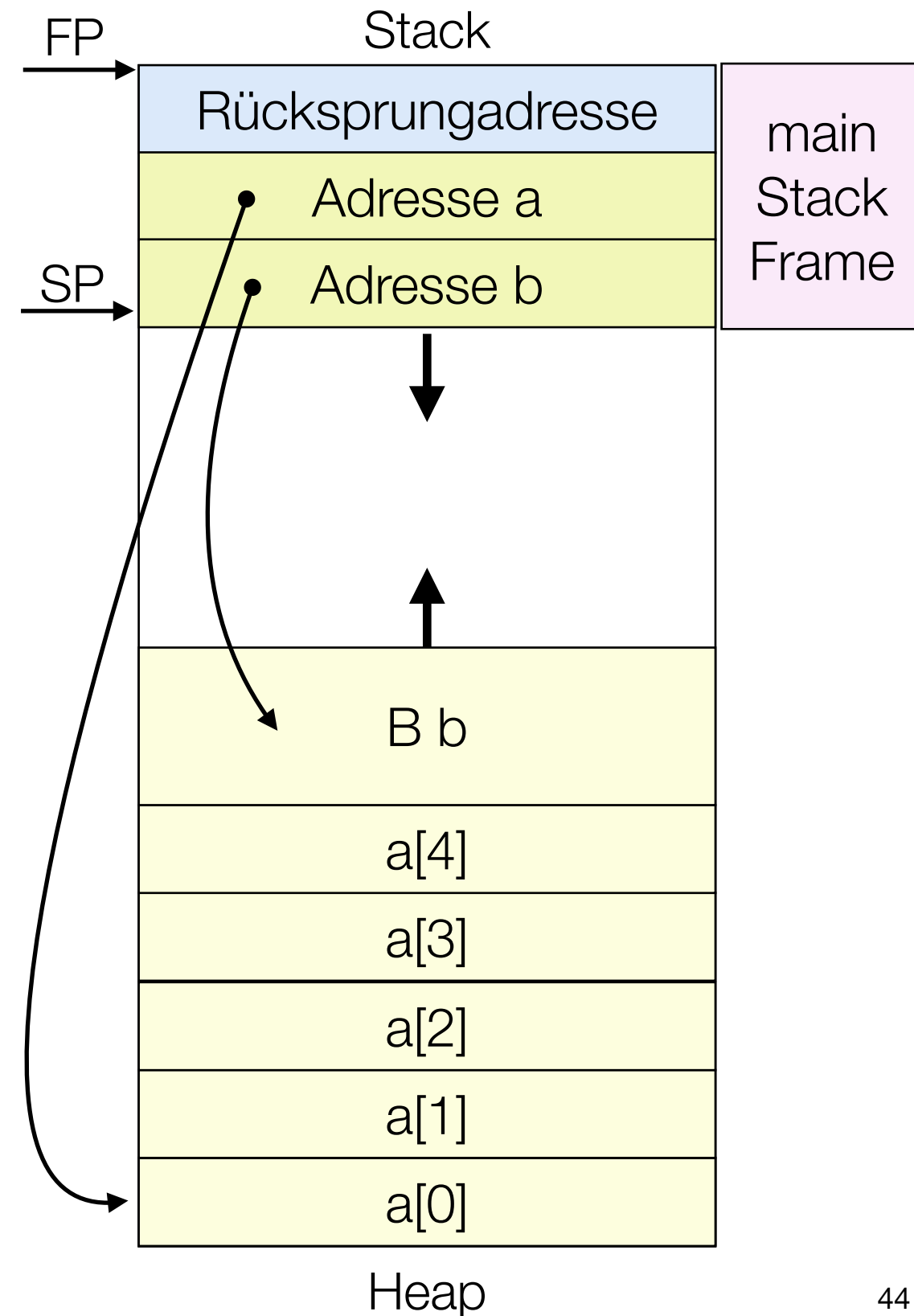
```
public static void main() {  
    int n = 21;  
    int m = 21;  
    int r = 0;  
    r = sum (n, m);  
}  
  
public static boolean truth(){  
    return true;  
}  
  
public static int sum(int n, int m){  
    int r = -1;  
    if(truth()){  
        r = n + m;  
    }  
    return r;  
}
```



Exkurs: **Stack** und **Heap**

```
public static void main() {  
    int[] a = new int[5];  
    B b = new B();  
}
```

```
public class B {  
    private int c;  
    public B()  
    {  
        this.c = 42;  
    }  
}
```



”Wo” findet die Berechnung statt?



Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

”Wo” findet die Berechnung statt?

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

Stack Frame	Rücksprungadresse
	n - 3
	Rückgabewert
Stack Frame	Rücksprungadresse
	n - 2
	Rückgabewert
Stack Frame	Rücksprungadresse
	n - 1
	Rückgabewert
Stack Frame	Rücksprungadresse
	n
	Rückgabewert
⋮	

Call Stack

”Wo” findet die Berechnung statt?

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

Stack Frame	Rücksprungadresse
	0
	Rückgabewert
Stack Frame	Rücksprungadresse
	1
	Rückgabewert
Stack Frame	Rücksprungadresse
	2
	Rückgabewert
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
⋮	

Call Stack

”Wo” findet die Berechnung statt?

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

Stack Frame	Rücksprungadresse
	0
	0
Stack Frame	Rücksprungadresse
	1
	Rückgabewert
Stack Frame	Rücksprungadresse
	2
	Rückgabewert
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
	⋮

Call Stack

”Wo” findet die Berechnung statt?

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

Stack Frame	Rücksprungadresse
	1
	1
Stack Frame	Rücksprungadresse
	2
	Rückgabewert
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
	⋮

Call Stack

”Wo” findet die Berechnung statt?

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

Stack Frame	Rücksprungadresse
	2
	3
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
	⋮

Call Stack

”Wo” findet die Berechnung statt?

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

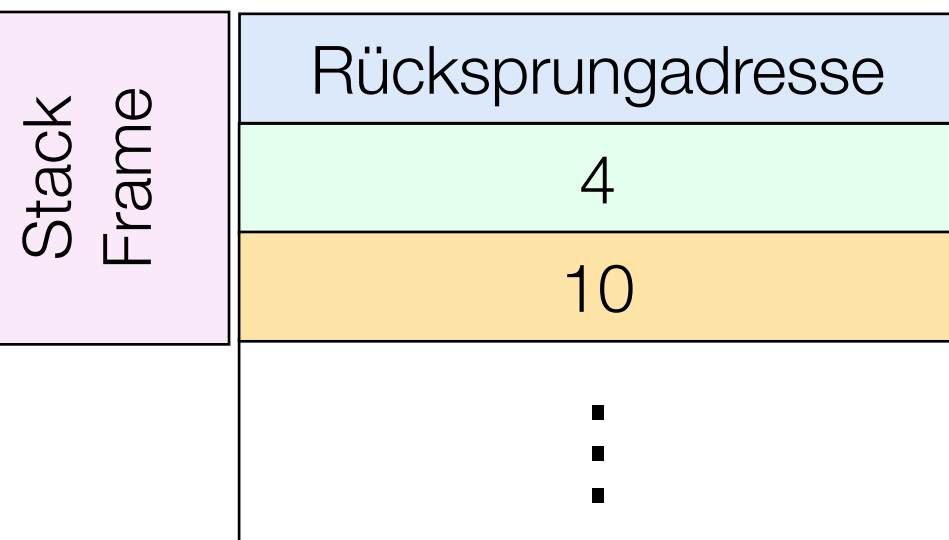
Stack Frame	Rücksprungadresse
	3
	6
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
⋮	

Call Stack

”Wo” findet die Berechnung statt?

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```



Call Stack

"Wo" findet die Berechnung statt?

Stack Frame	Rücksprungadresse
	n - 3
	Rückgabewert
Stack Frame	Rücksprungadresse
	n - 2
	Rückgabewert
Stack Frame	Rücksprungadresse
	n - 1
	Rückgabewert
Stack Frame	Rücksprungadresse
	n
	Rückgabewert
⋮	

Call Stack

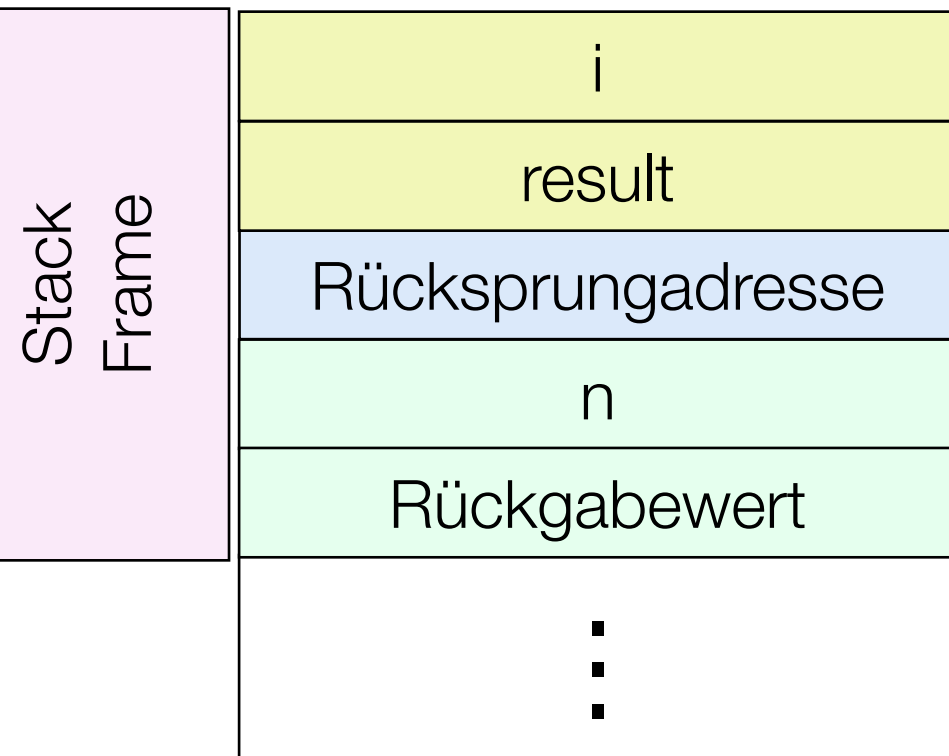
Iterativ

```
public int calculateSum(int n){  
    int result = 0;  
    for(int i = n; i>0; i--)  
    {  
        result += i;  
    }  
    return result;  
}
```

”Wo” findet die Berechnung statt?

Iterativ

```
public int calculateSum(int n){  
    int result = 0;  
    for(int i = n; i>0; i--)  
    {  
        result += i;  
    }  
    return result;  
}
```



Call Stack

Zeitkomplexität

Iterativ

```
public int calculateSum(int n){  
    int result = 0;  
    for(int i = n; i>0; i--)  
    {  
        result += i;  
    }  
    return result;  
}
```

linear: $O(n)$

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```



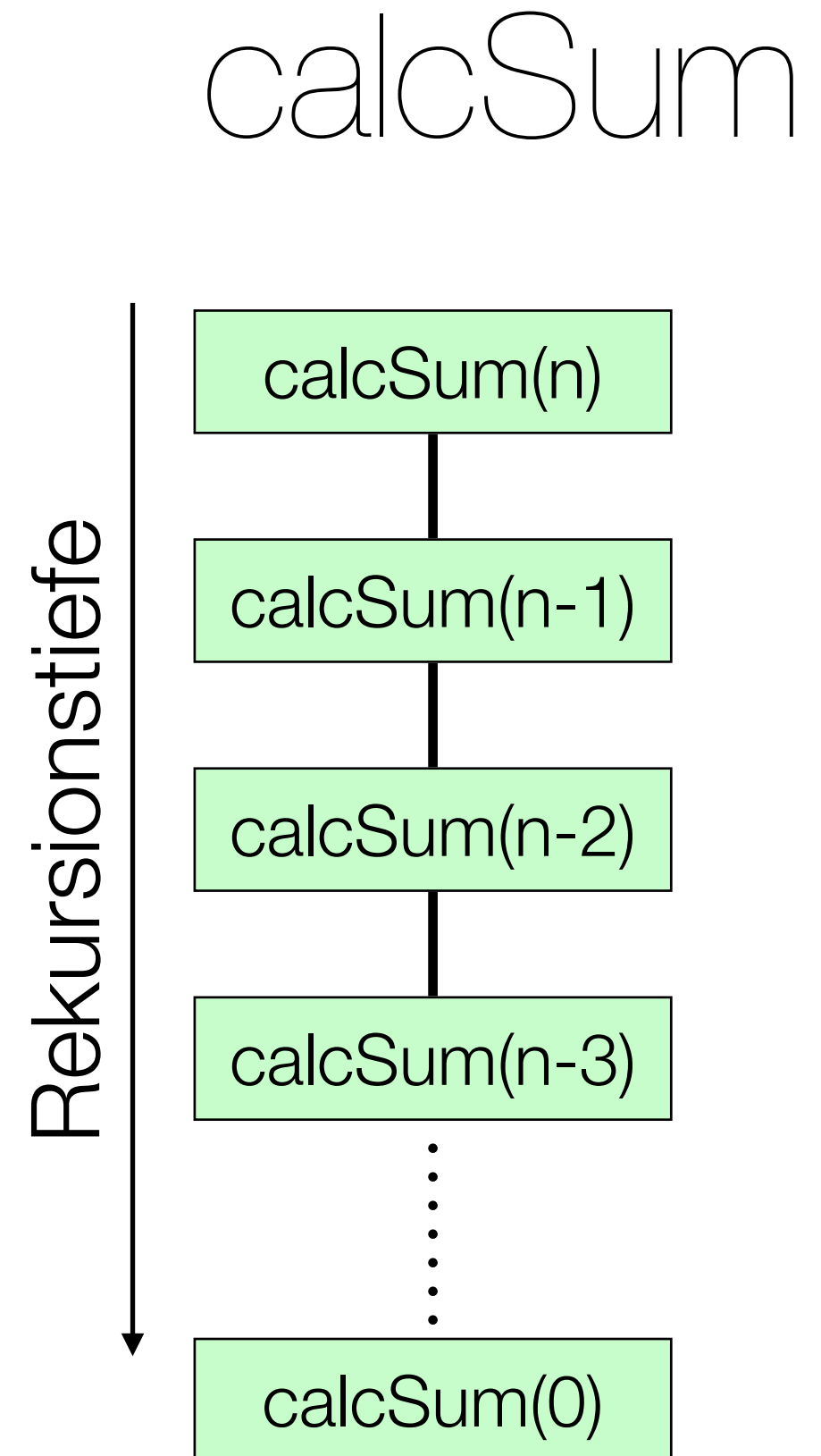
Rekursive Aufrufe von calcSum

Iterativ

```
public int calculateSum(int n){  
    int result = 0;  
    for(int i = n; i>0; i--)  
    {  
        result += i;  
    }  
    return result;  
}
```

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```



Zeitkomplexität

Iterativ

```
public int calculateSum(int n){  
    int result = 0;  
    for(int i = n; i>0; i--)  
    {  
        result += i;  
    }  
    return result;  
}
```

linear: $O(n)$

Rekursiv

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

linear: $O(n)$

Geschwindigkeit



Geschwindigkeit

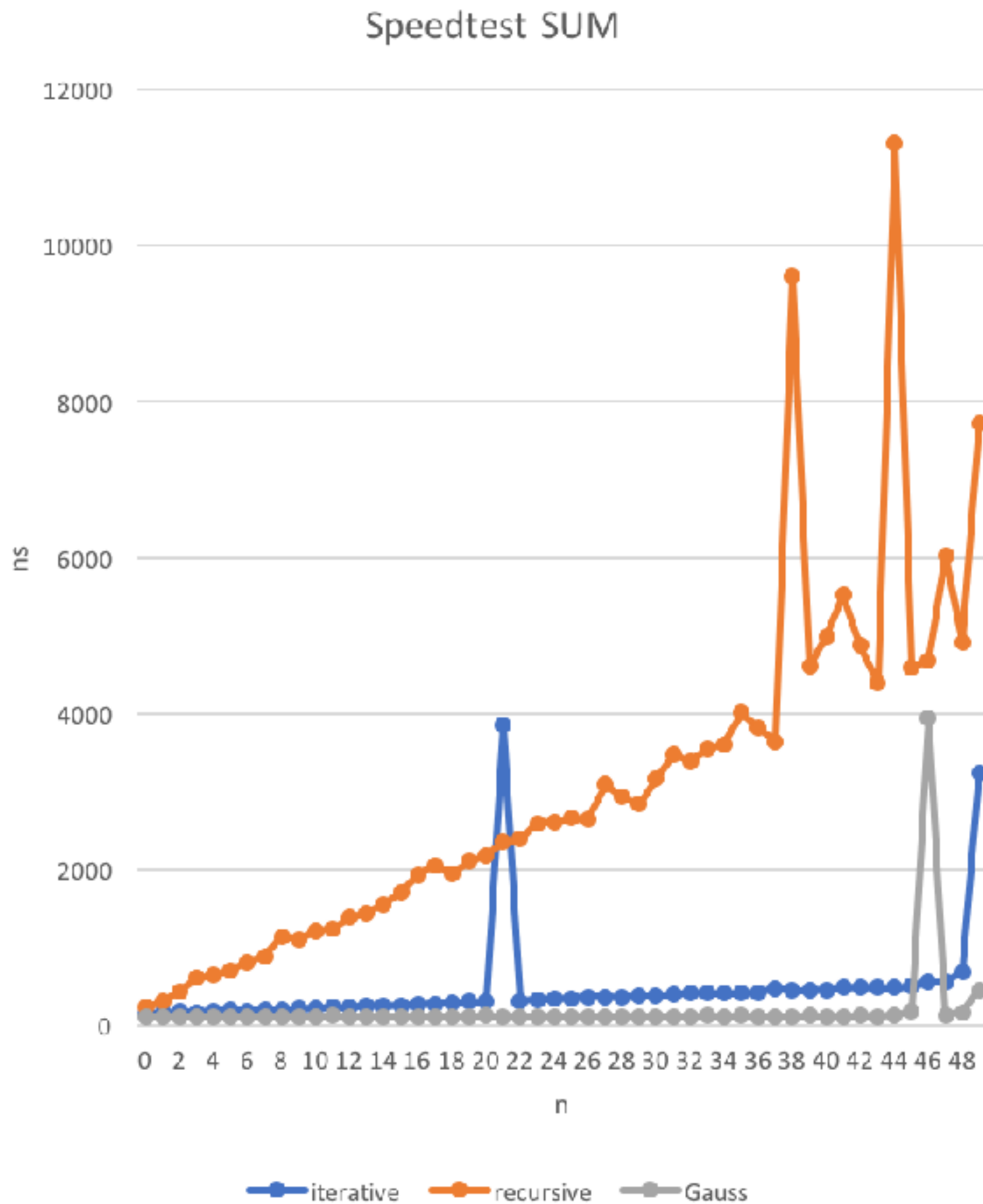
A speedometer with a black face and a red needle is shown. The needle is pointing towards the 10 mark. A large, red, rectangular stamp with the word "DEMO" in white, bold, sans-serif capital letters is overlaid on the speedometer. The stamp has a distressed, grungy texture. The speedometer has white markings for 10, 100, and 120, with smaller markings in between. The background is black.

DEMO

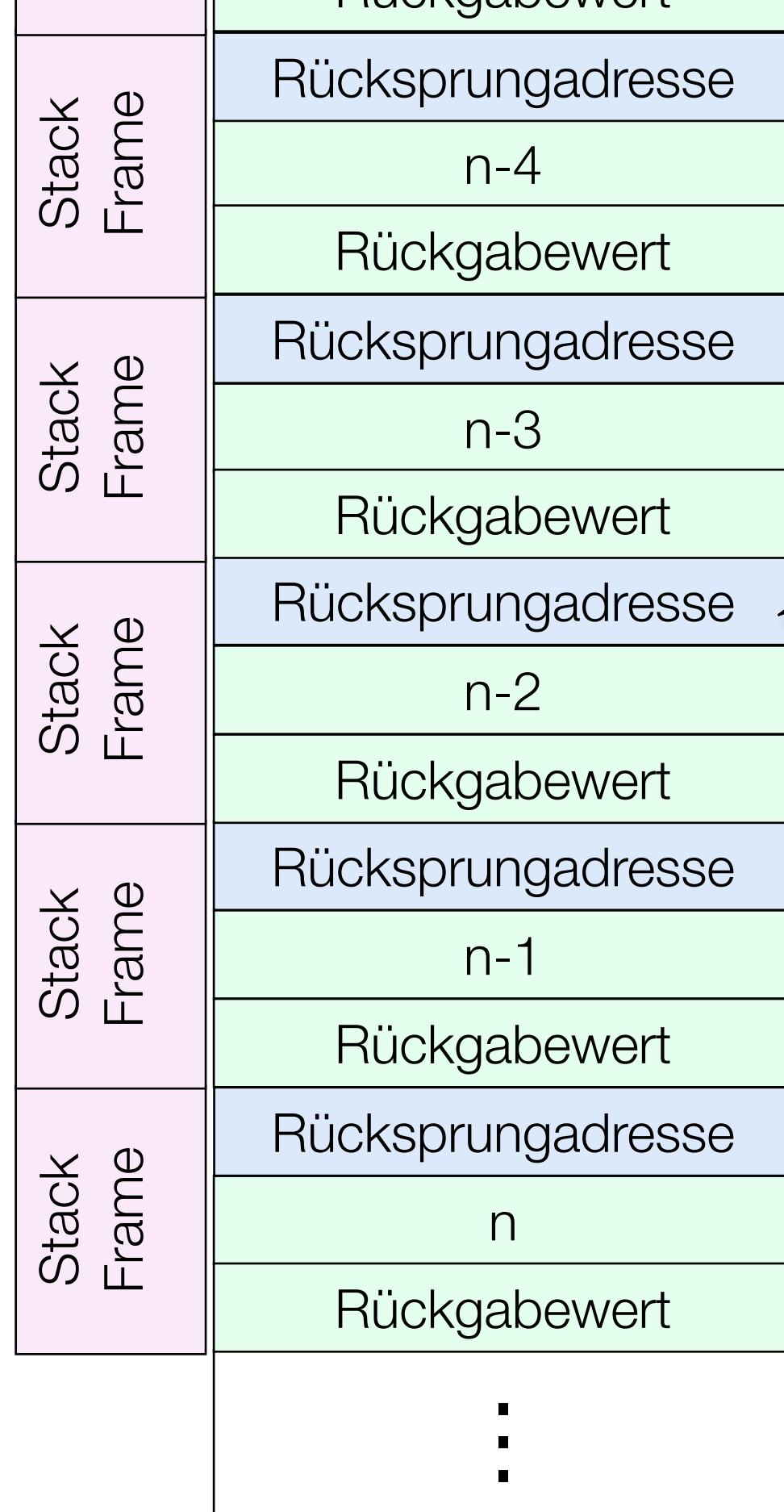
► Speedtest.java

Speedtest

SUM



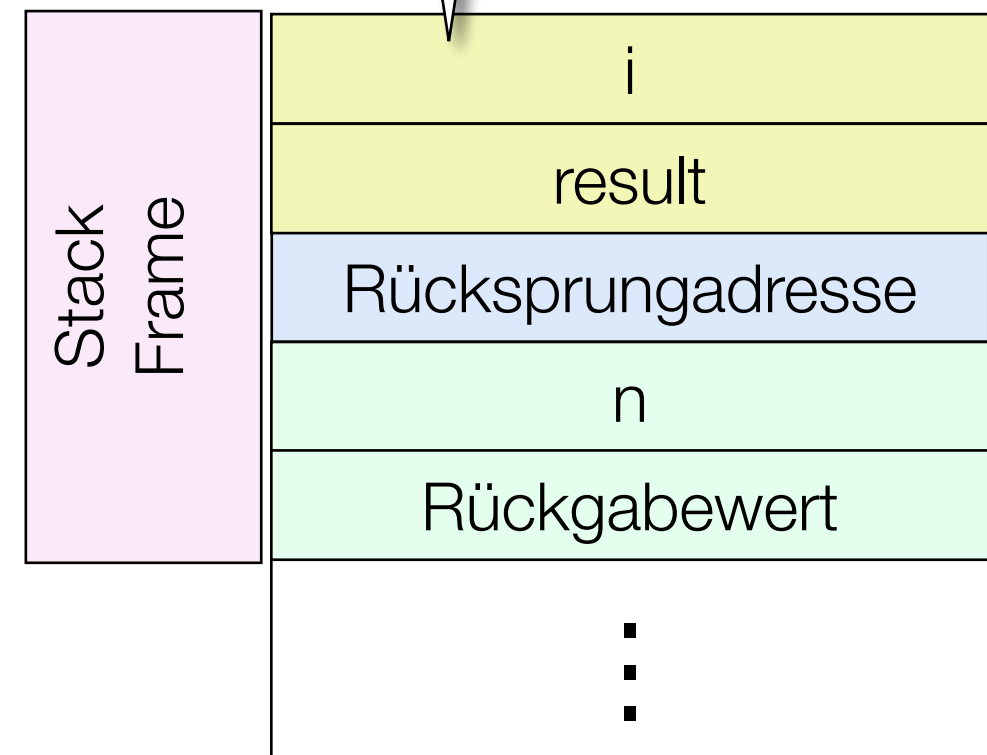
Speicherkomplexität



Rekursiv

linear: $O(n)$

konstant: $O(1)$



Iterativ

Details

Call Stack



Sum

Stack Frame	Rücksprungadresse
	0
Stack Frame	Rückgabewert
Stack Frame	Rücksprungadresse
	1
Stack Frame	Rückgabewert
Stack Frame	Rücksprungadresse
	2
Stack Frame	Rückgabewert
Stack Frame	Rücksprungadresse
	3
Stack Frame	Rückgabewert
Stack Frame	Rücksprungadresse
	4
Stack Frame	Rückgabewert
Stack Frame	Rücksprungadresse
	5
Stack Frame	Rückgabewert

Details

Call Stack



Sum

Stack Frame	Rücksprungadresse
	0
	0
Stack Frame	Rücksprungadresse
	1
	Rückgabewert
Stack Frame	Rücksprungadresse
	2
	Rückgabewert
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack



Sum

Stack Frame	Rücksprungadresse
	1
	1
Stack Frame	Rücksprungadresse
	2
	Rückgabewert
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack



Sum

Stack Frame	Rücksprungadresse
	2
	3
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack



Sum

Stack Frame	Rücksprungadresse
	3
	6
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

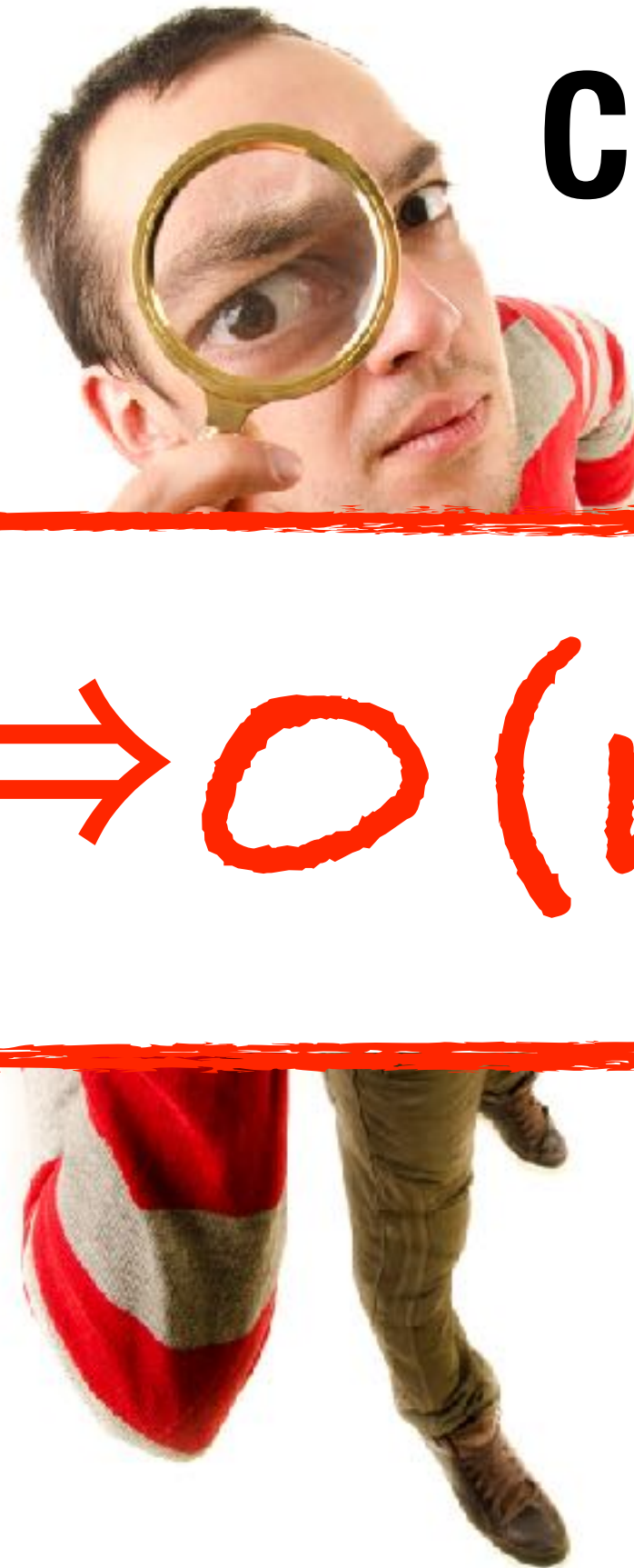


Sum

Stack Frame	Rücksprungadresse
	4
	10
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack



$$n + 1 \Rightarrow O(n)$$

Stack
Frame

Rücksprungadresse

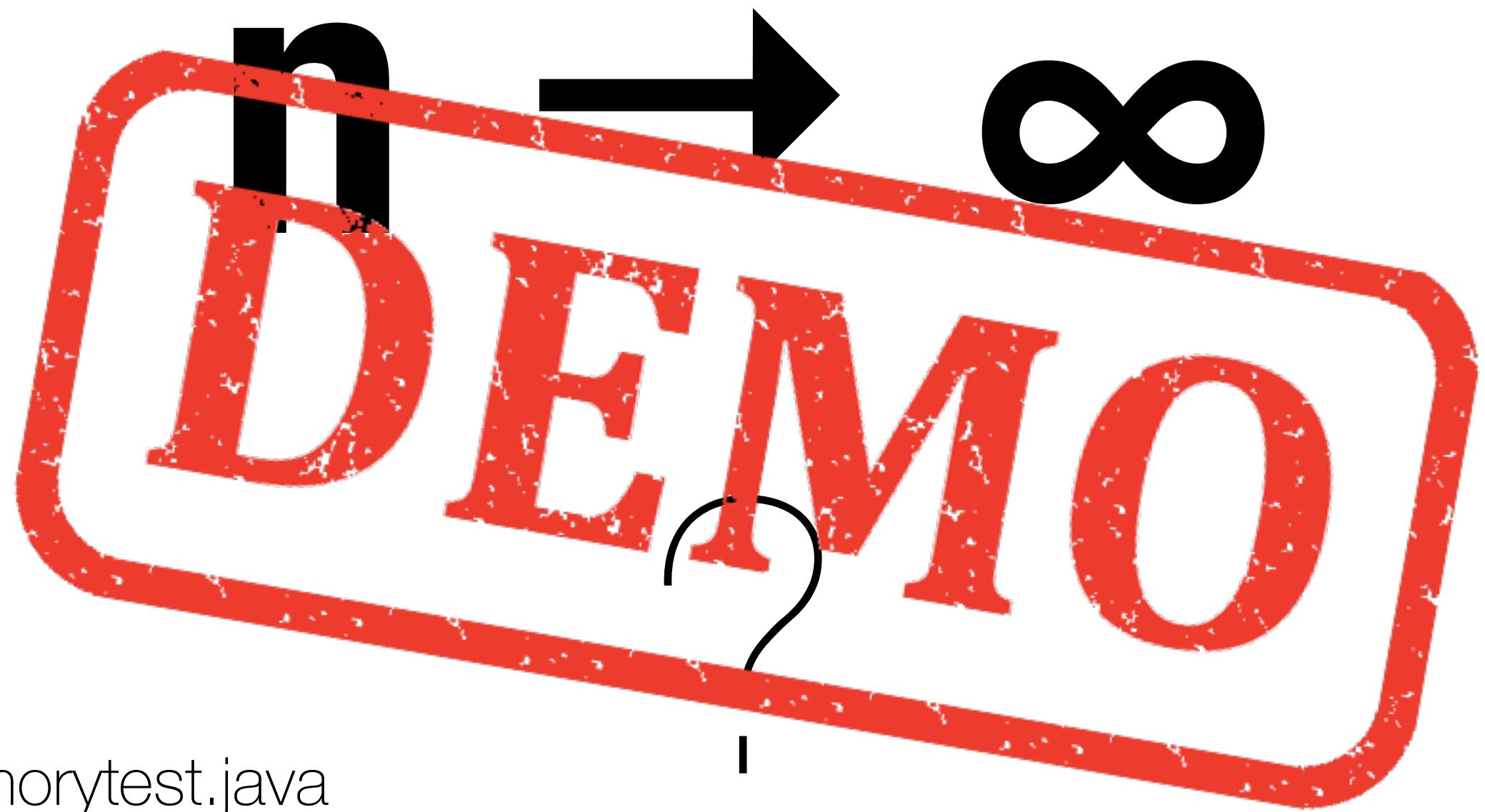
5

15

Sum

n → **∞**

?



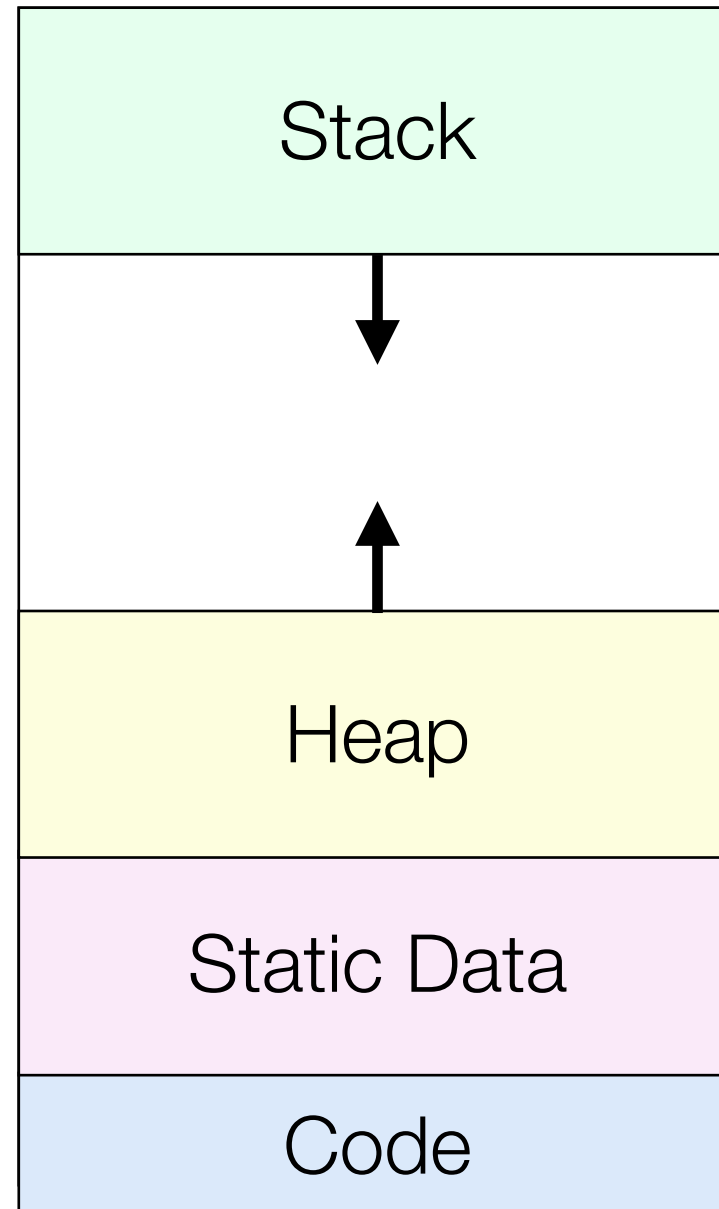
► Memorytest.java

Stack Overflow

java.lang.StackOverflowError



n $\rightarrow$ ∞



Fibonacci-Folge

► Definition:

$$\text{► } fib(n) = \begin{cases} 1 & \text{falls } n \leq 2 \\ fib(n-1) + fib(n-2) & \text{sonst} \end{cases}$$

Rekursionsanfang

Rekursionsschritt

Fibonacci-Folge

n	f_n	n	f_n	n	f_n	n	f_n	n	f_n
1	1	11	89	21	10946	31	1346269	41	165580141
2	1	12	144	22	17711	32	2178309	42	267914296
3	2	13	233	23	28657	33	3524578	43	433494437
4	3	14	377	24	46368	34	5702887	44	701408733
5	5	15	610	25	75025	35	9227465	45	1134903170
6	8	16	987	26	121393	36	14930352	46	1836311903
7	13	17	1597	27	196418	37	24157817	47	2971215073
8	21	18	2584	28	317811	38	39088169	48	4807526976
9	34	19	4181	29	514229	39	63245986	49	7778742049
10	55	20	6765	30	832040	40	102334155	50	12586269025

$$fib(n) = \begin{cases} 1 & \text{falls } n \leq 2 \\ fib(n-1) + fib(n-2) & \text{sonst} \end{cases}$$

Iterative Implementierung

$$fib(n) = \begin{cases} 1 & \text{falls } n \leq 2 \\ fib(n-1) + fib(n-2) & \text{sonst} \end{cases}$$

```
public long calcFib(int n){
```

?

```
}
```

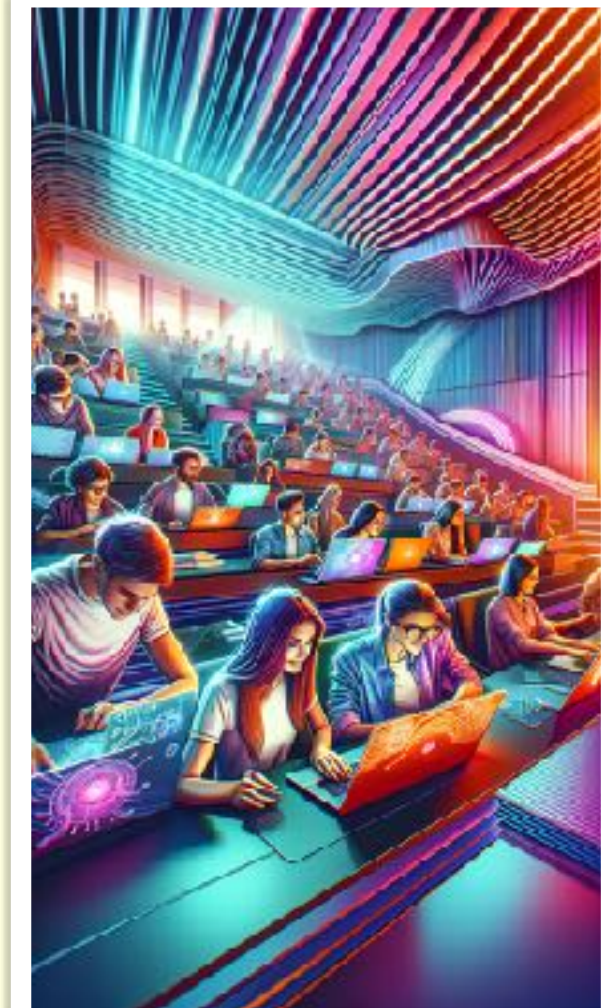
Iterative Implementierung

$$fib(n) = \begin{cases} 1 & \text{falls } n \leq 2 \\ fib(n-1) + fib(n-2) & \text{sonst} \end{cases}$$

```
public long calcFib(int n){
```

?

```
}
```



Iterative Implementierung

$$fib(n) = \begin{cases} 1 & \text{falls } n \leq 2 \\ fib(n-1) + fib(n-2) & \text{sonst} \end{cases}$$

```
public long calcFib(int n){  
    long fibN = 1;  
    long fibN1 = 1; // fib(n-1);  
    long fibN2 = 1; // fib(n-2);  
    for (int i=3; i <= n; i++) {  
        fibN = fibN1 + fibN2;  
        fibN2 = fibN1;  
        fibN1 = fibN;  
    }  
    return fibN;  
}
```

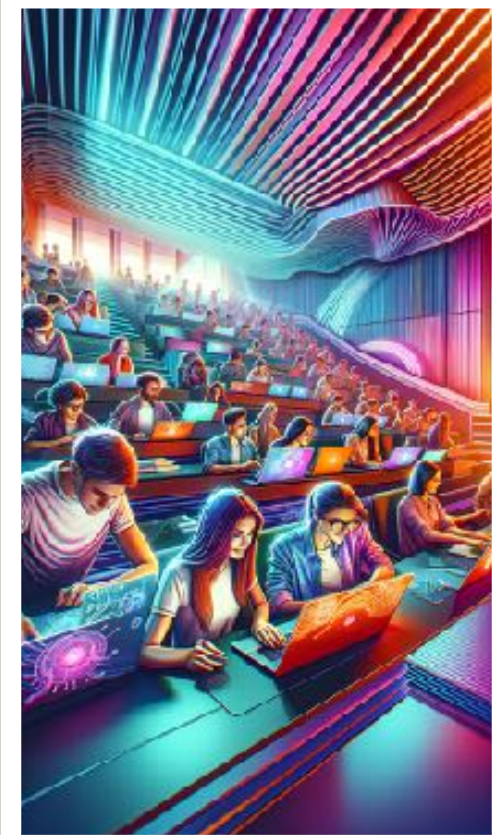
Rekursive Implementierung

$$fib(n) = \begin{cases} 1 & \text{falls } n \leq 2 \\ fib(n-1) + fib(n-2) & \text{sonst} \end{cases}$$

```
public int calcFib(int n)  
{
```

?

```
}
```



Rekursive Implementierung

$$fib(n) = \begin{cases} 1 & \text{falls } n \leq 2 \\ fib(n-1) + fib(n-2) & \text{sonst} \end{cases}$$

```
public int calcFib(int n)
{
    if(n <= 2)
        return 1;
    else
        return calcFib(n-1) + calcFib (n-2);
}
```

Zeitkomplexität Fibonacci

Iterativ

```
public long calcFib(int n){  
    long fibN = 0;  
    long fibN1 = 1; // Fib(n-1);  
    long fibN2 = 1; // Fib(n-2)  
    for (int i=3; i <= n; i++) {  
        fibN = fibN1 + fibN2;  
        fibN2 = fibN1;  
        fibN1 = fibN;  
    }  
    return fibN;  
}
```

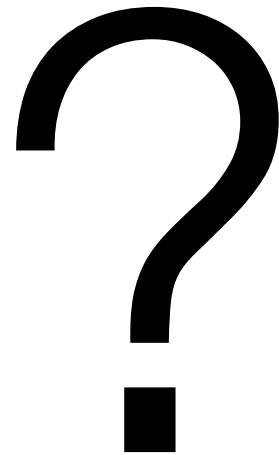

$$\triangleright C_n = (n-2) \times 3$$

$$\Rightarrow O(n)$$

Zeitkomplexität

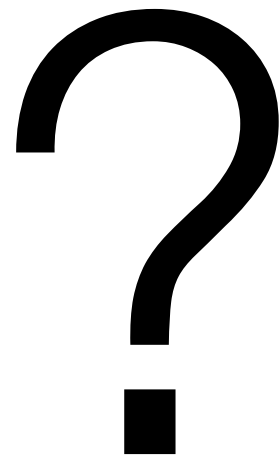
Fibonacci

Rekursiv



Zeitkomplexität Fibonacci

Rekursiv

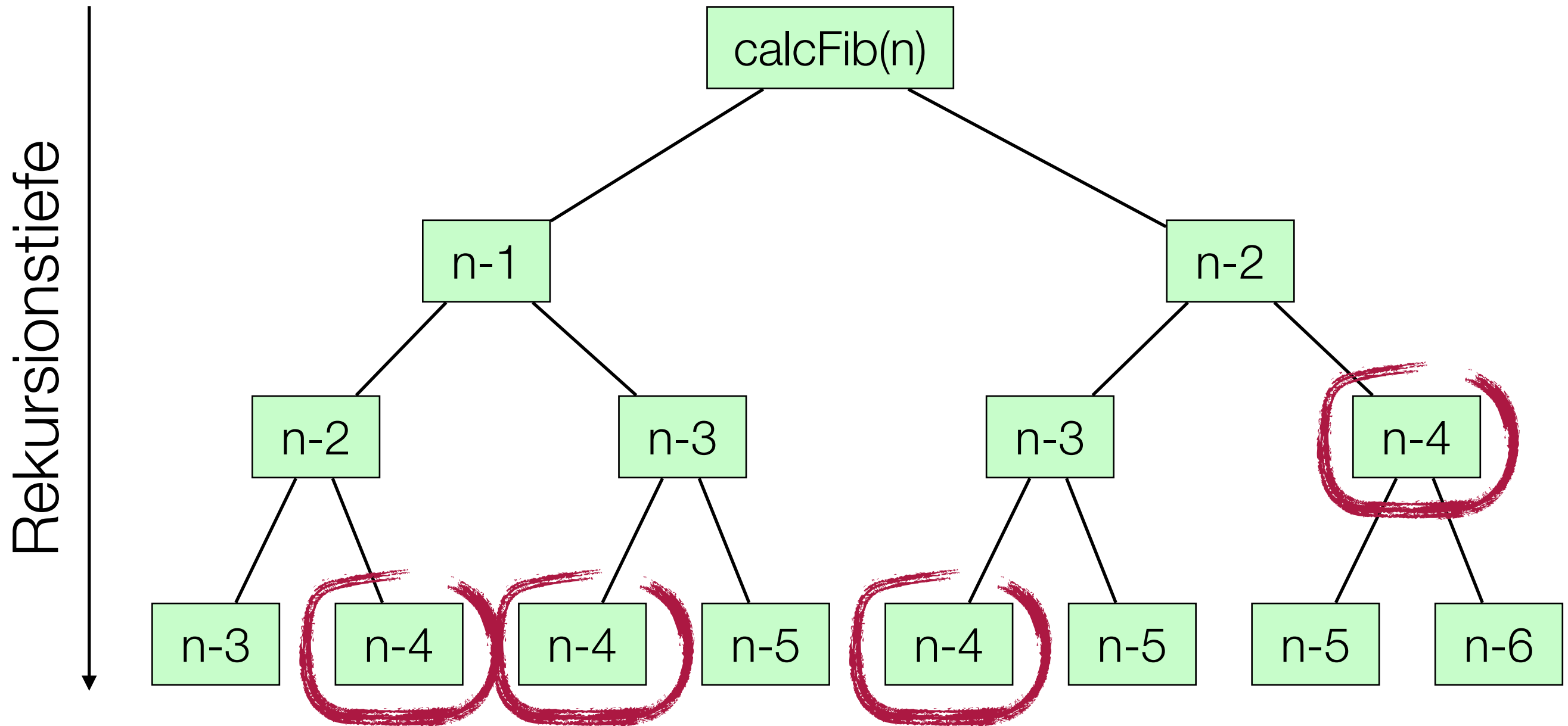


```
public int calcFib(int n)
{
    if(n <= 2)
        return 1;
    else
        return calcFib(n-1) + calcFib (n-2);
}
```



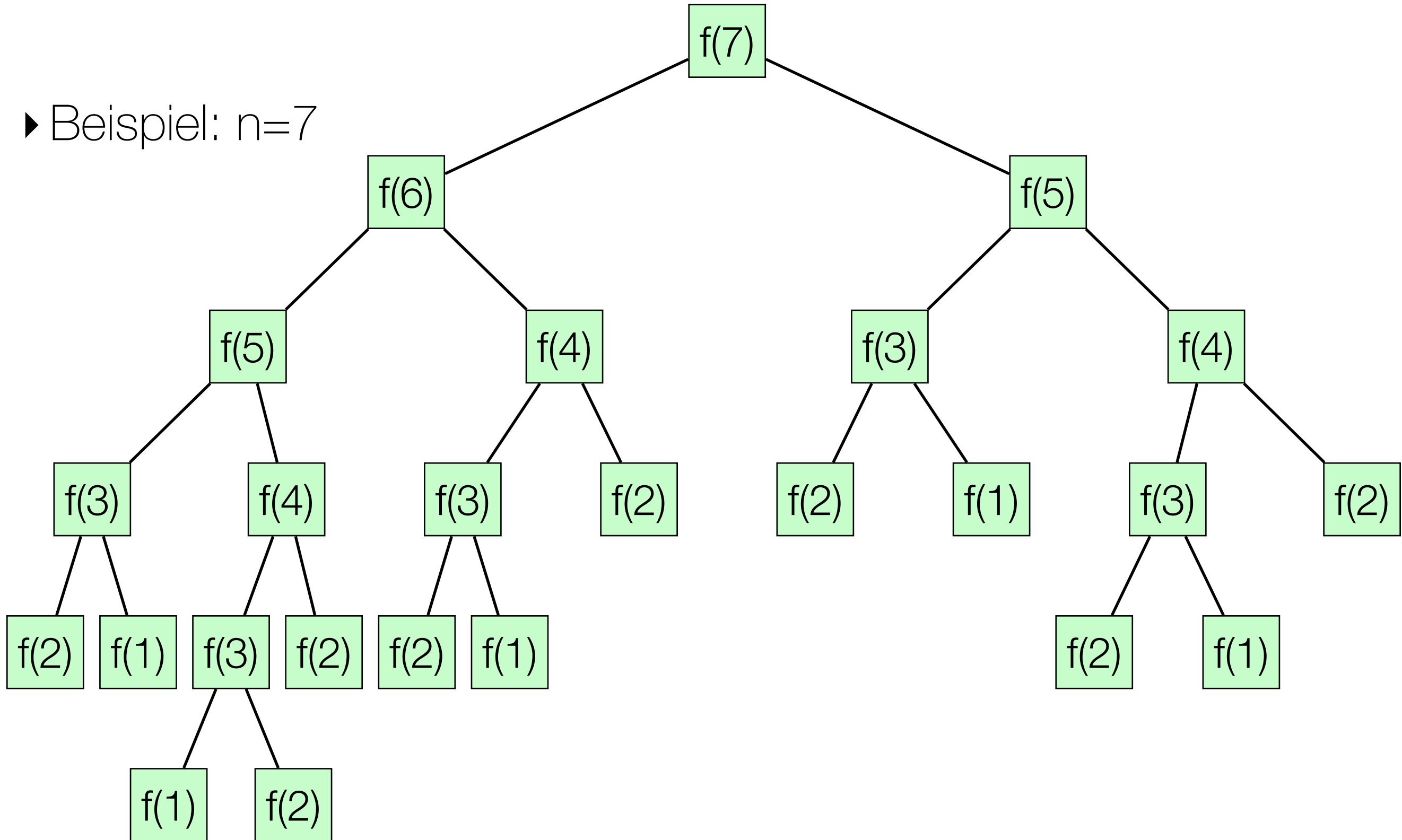
30 Sek.

Redundante Berechnungen



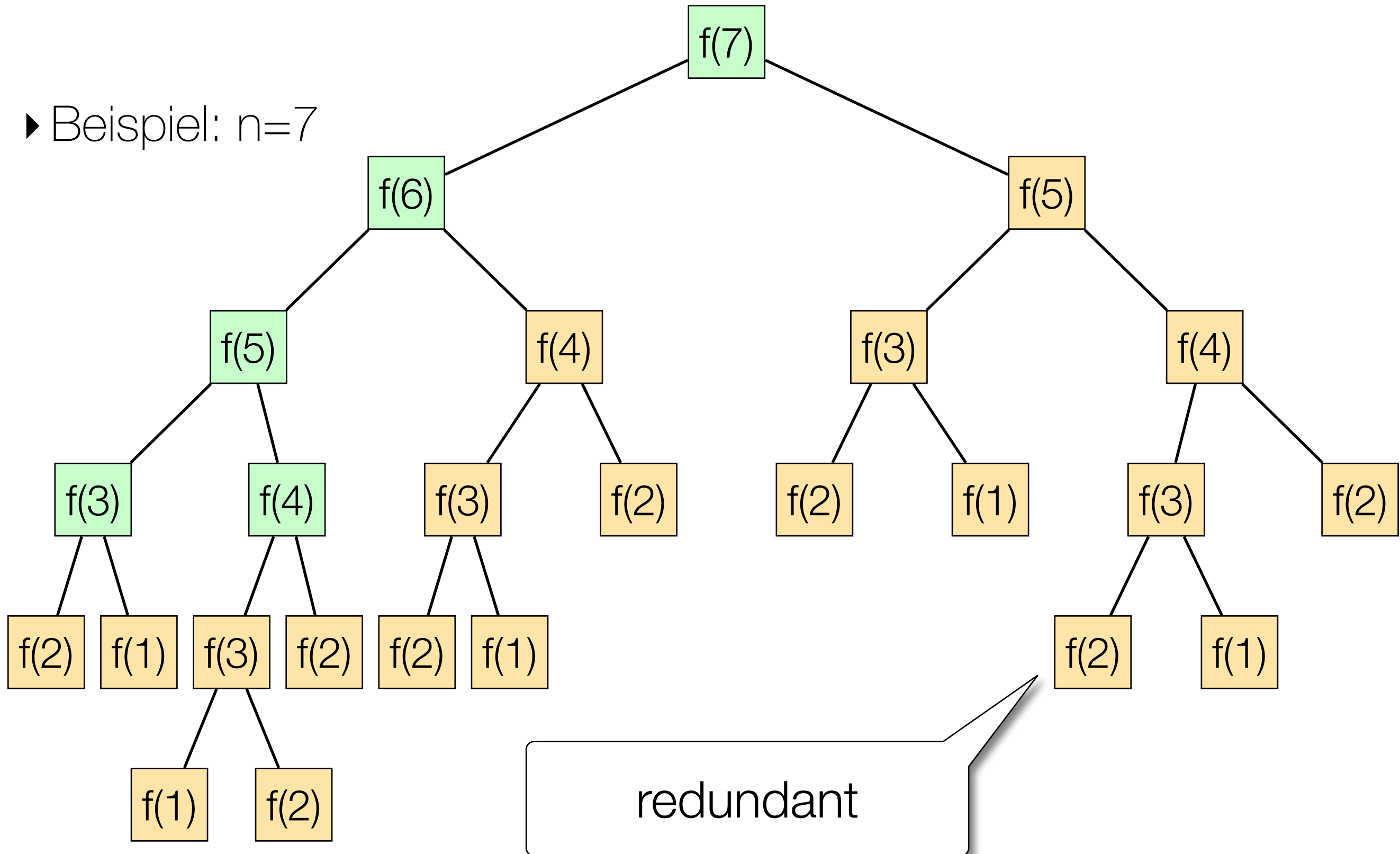
Redundante Berechnungen

► Beispiel: $n=7$



Redundante Berechnungen

► Beispiel: $n=7$



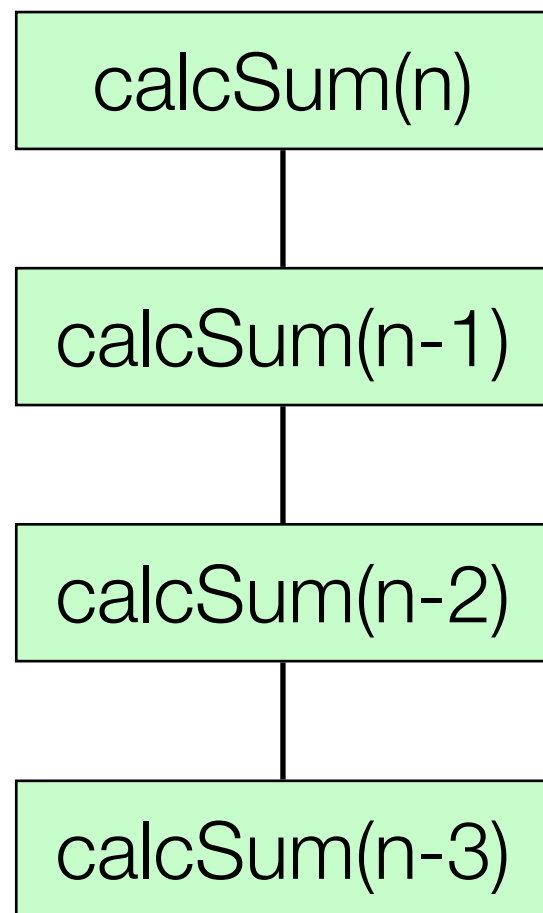
Zeitkomplexität Fibonacci

Rekursiv

- ▶ Sei c_i der Aufwand zur Berechnung von $\text{fib}(i)$
 - ▶ Es gilt $c_1 = c_2 = 1$
 - ▶ für $n > 2$: $c_n = 1 + c_{n-1} + c_{n-2}$
 - ▶ bei $n > 3$: $c_{n-1} = 1 + c_{n-2} + c_{n-3} \Rightarrow c_{n-2} = c_{n-1} - 1 - c_{n-3}$
 - ▶ Durch einsetzen von c_{n-2} :
 - ▶ $c_n = 1 + c_{n-1} + c_{n-2} = 2c_{n-1} - c_{n-3}$
 $< 2c_{n-1} < 2^2c_{n-2} < \dots 2^{n-1}c_1$
- $\Rightarrow O(2^n)$

Zeitkomplexität

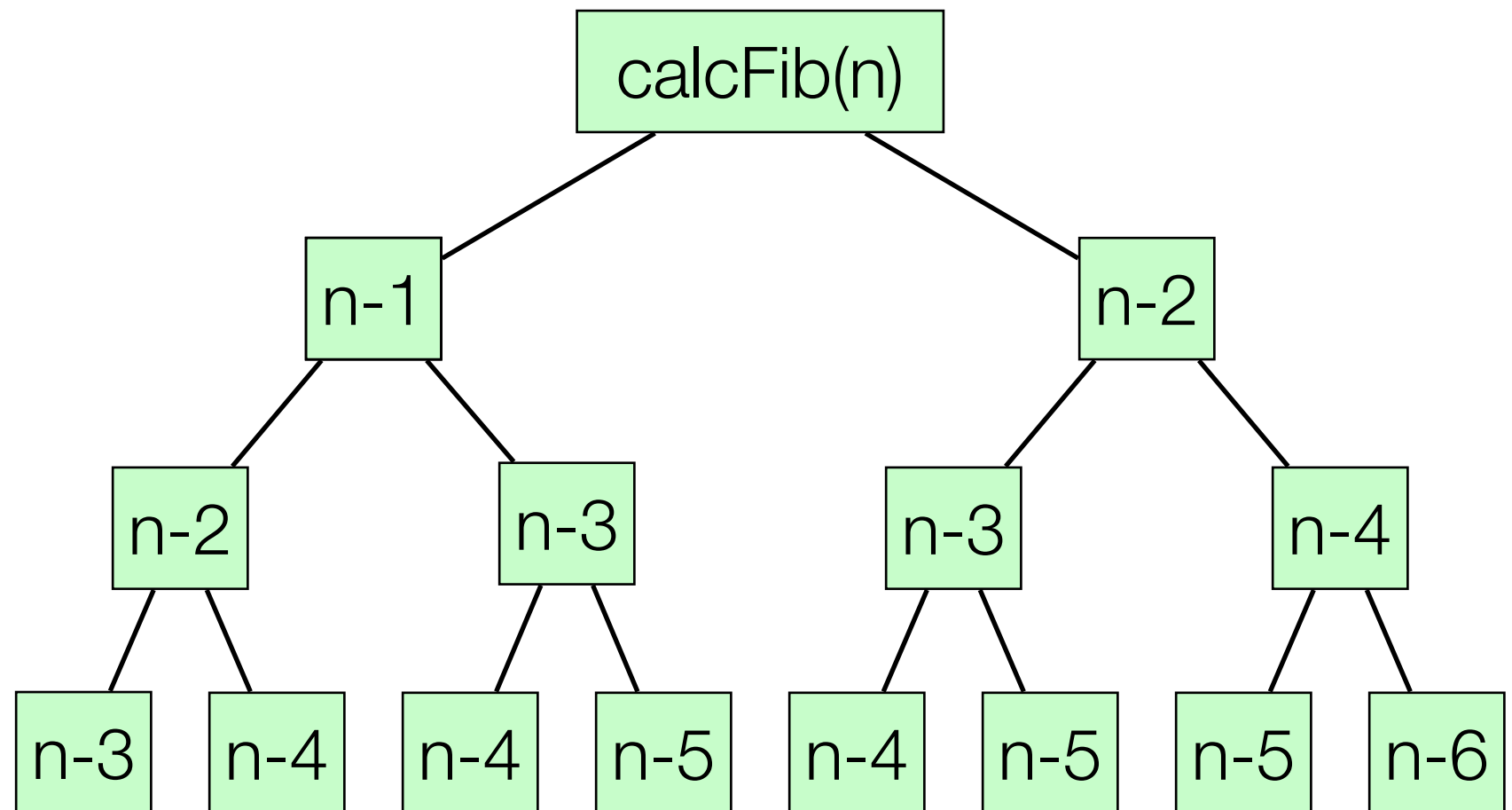
sum



$O(n)$

VS.

fib



$O(2^n)$

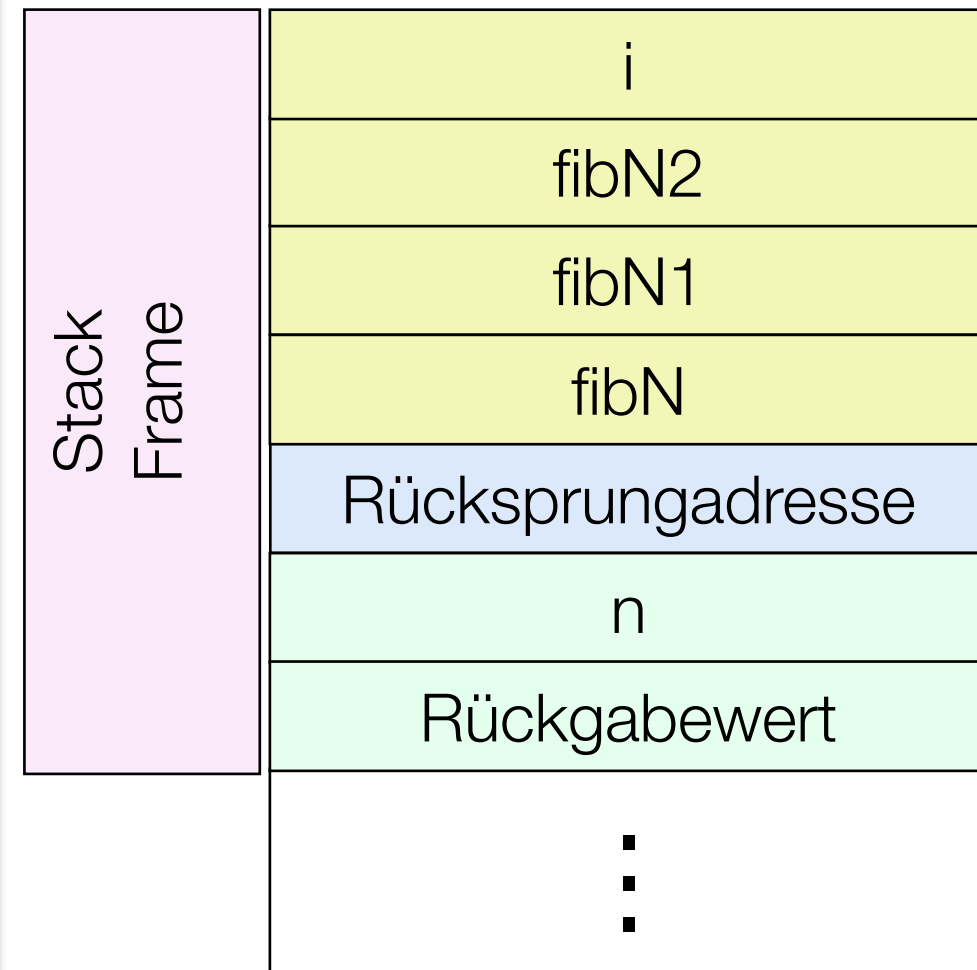
DEMO

► **FibonacciSpeedtest.java**

Speicherkomplexität Fibonacci

Iterativ

```
public long calcFib(int n){  
    long fibN = 0;  
    long fibN1 = 1; // Fib(n-1);  
    long fibN2 = 1; // Fib(n-2);  
    for (int i=3; i <= n; i++) {  
        fibN = fibN1 + fibN2;  
        fibN2 = fibN1;  
        fibN1 = fibN;  
    }  
    return fibN;  
}
```



$\Rightarrow O(1)$

sum

```
public int calculateSum(int n){  
    if(n == 0)  
        return 0;  
    else  
        return n + calculateSum(n-1);  
}
```

VS.

fib

```
public int calcFib(int n){  
    if(n <= 2)  
        return 1;  
    else  
        return calcFib(n-1) + calcFib (n-2);  
}
```

Speicher**komplexität**

rekursiv

Details

Call Stack



Fibonacci

Stack Frame	Rücksprungadresse
	2
	Rückgabewert
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	2
	1
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	1
	Rückgabewert
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	1
	1
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	3
	2
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	4
Stack Frame	Rückgabewert
Stack Frame	Rücksprungadresse
	5
Stack Frame	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	2
Stack Frame	Rückgabewert
Stack Frame	Rücksprungadresse
	4
Stack Frame	Rückgabewert
Stack Frame	Rücksprungadresse
	5
Stack Frame	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	2
	1
Stack Frame	Rücksprungadresse
	4
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	4
	3
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	2
	Rückgabewert
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	2
	1
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	1
	Rückgabewert
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	1
	1
Stack Frame	Rücksprungadresse
	3
	Rückgabewert
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4



Fibonacci

Stack Frame	Rücksprungadresse
	3
	2
Stack Frame	Rücksprungadresse
	5
	Rückgabewert

Details

Call Stack

4

$$n-1 \Rightarrow O(n)$$

Stack
Frame

Rücksprungadresse

5

5

Fibonacci

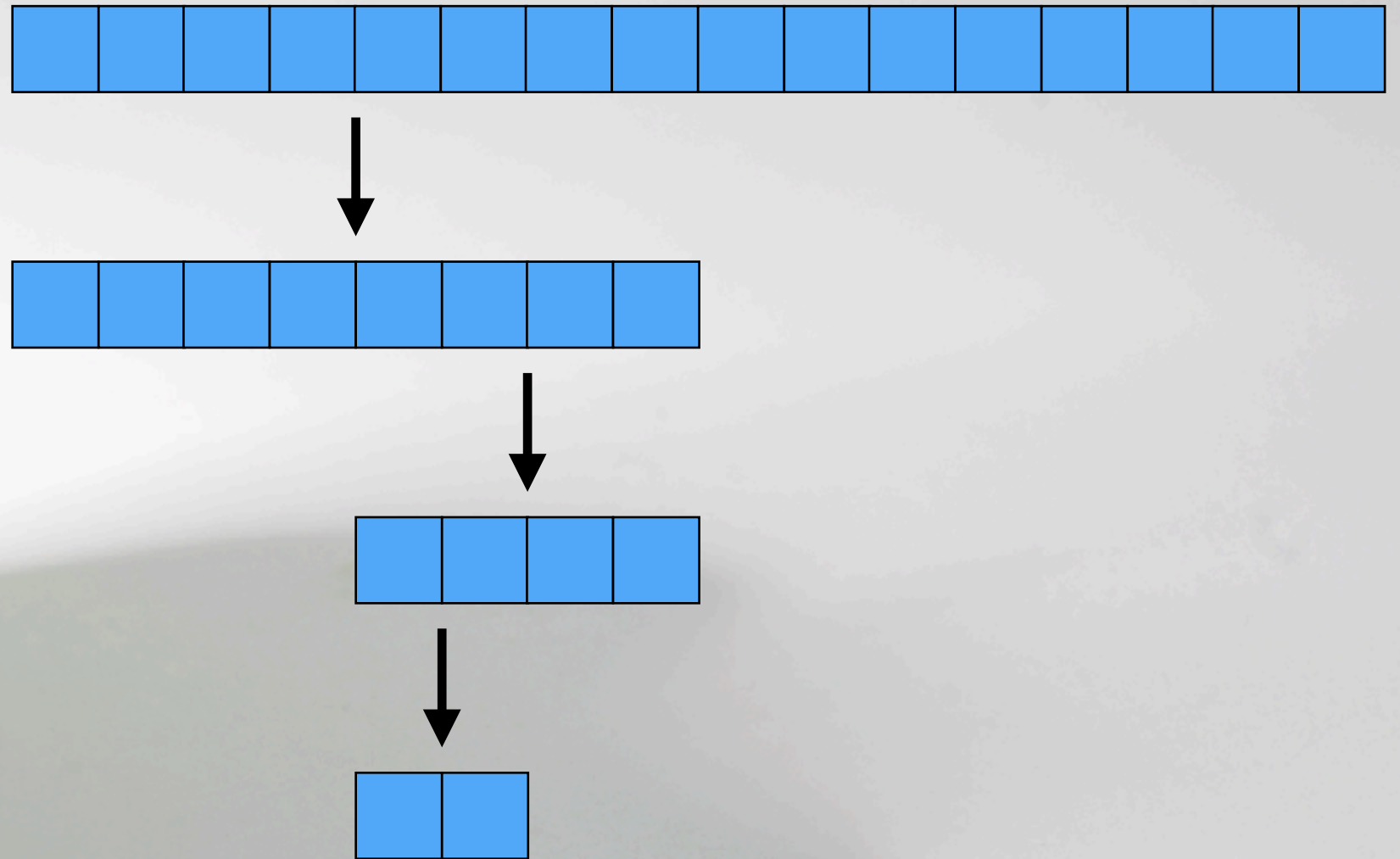
DEMO

► **FibonacciLauncher.java**

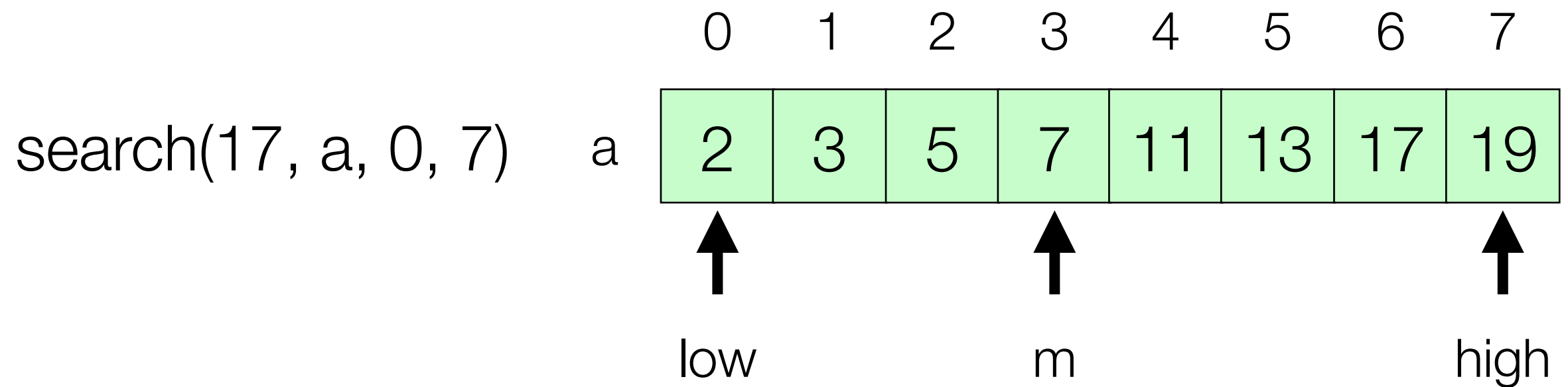
Beispiel **Binäre Suche**



Beispiel **Binäre Suche**



Binäre Suche - **Iterativ**



```
public int search(int elem, int[] a, int low, int high) {
```

?

```
}
```

Binäre Suche - **Iterativ**

search(17, a, 0, 7)

	0	1	2	3	4	5	6	7
a	2	3	5	7	11	13	17	19
	↑			↑				↑
	low			m				high

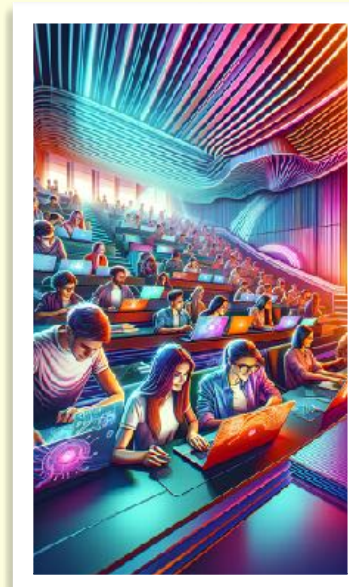
```
public int search(int elem, int[] a, int low, int high) {
    while(low <= high){
        int m = (low + high) / 2;
        if (elem == a[m]) return m;
        if (elem < a[m])
            high = m-1;
        else
            low = m+1;
    }
    return -1;
}
```

Binäre Suche - **Rekursiv**

search(17, a, 0, 7)

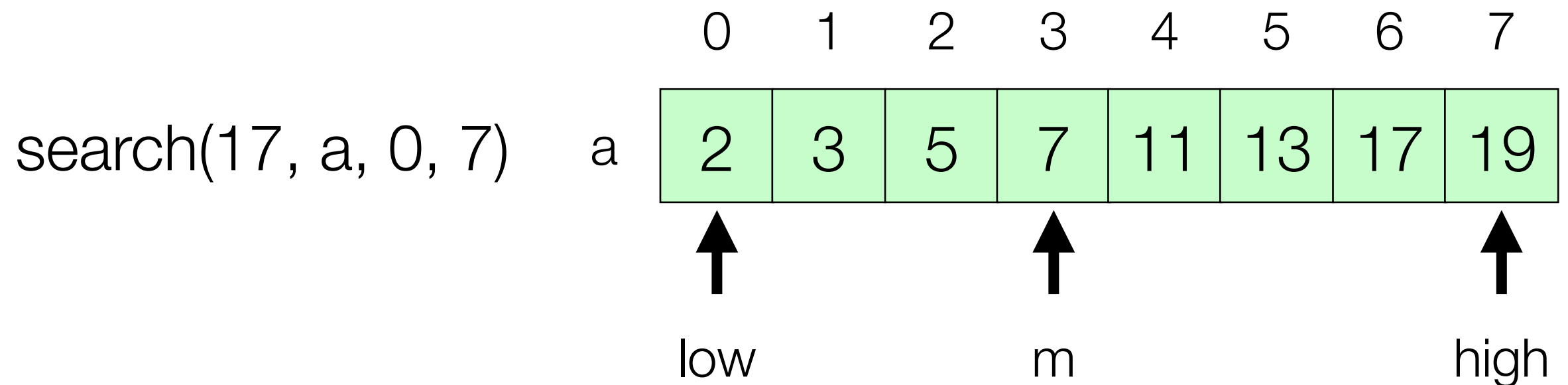
	0	1	2	3	4	5	6	7
a	2	3	5	7	11	13	17	19
	↑			↑				↑
	low			m				high

```
public int search(int elem, int[] a, int low, int high) {
```



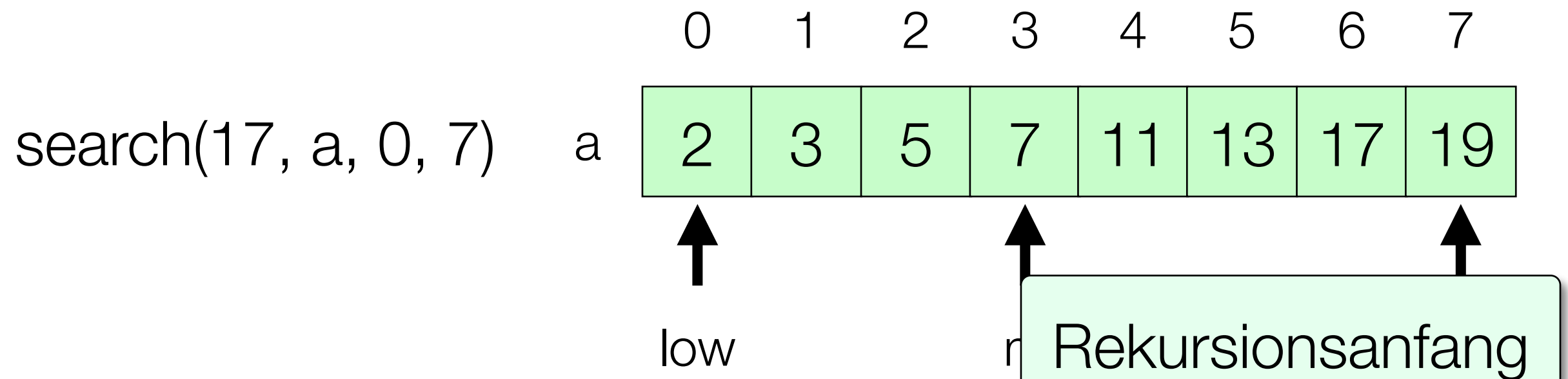
}

Binäre Suche - **Rekursiv**



```
public int search(int elem, int[] a, int low, int high) {
    if (low > high) return -1; // empty
    int m = (low + high) / 2;
    if (elem == a[m]) return m;
    if (elem < a[m])
        return search(elem, a, low, m-1);
    return search(elem, a, m+1, high);
}
```

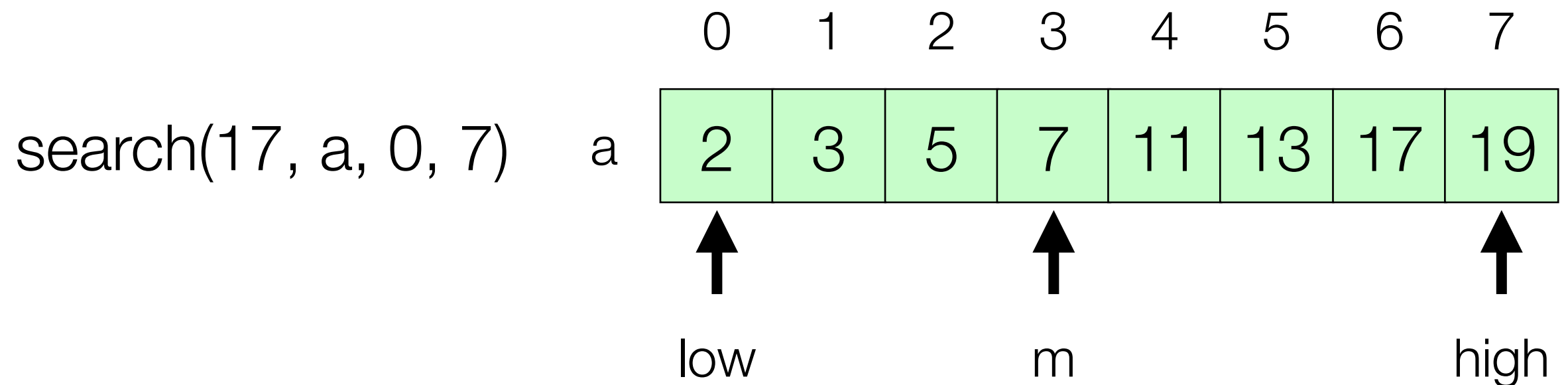
Binäre Suche - **Rekursiv**



```
public int search(int elem, int[] a, int low, int high) {  
    if (low > high) return -1; // empty  
    int m = (low + high) / 2;  
    if (elem == a[m]) return m;  
    if (elem < a[m])  
        return search(elem, a, low, m-1);  
    return search(elem, a, m+1, high);  
}
```

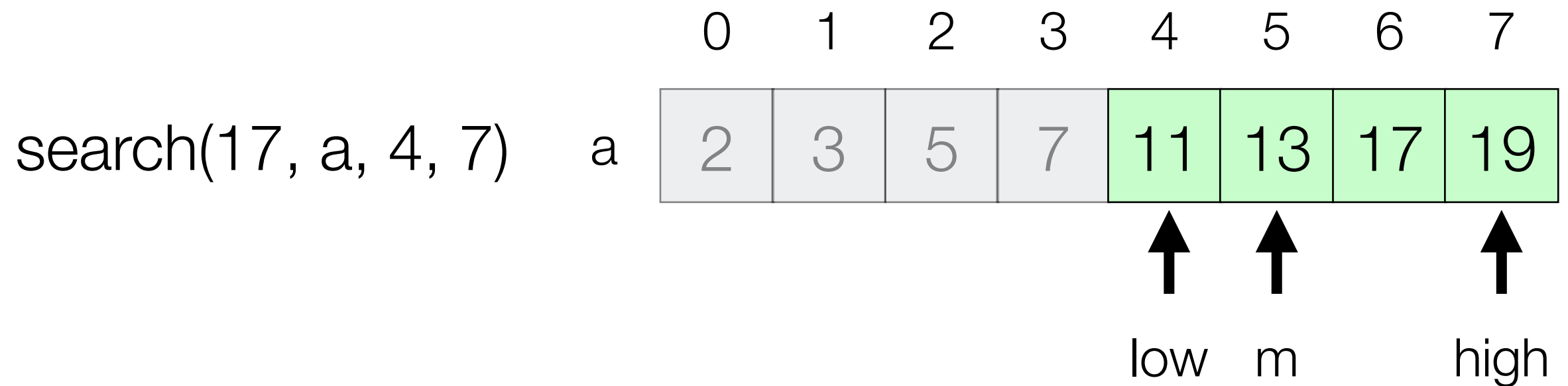
Rekursionsschritt

Binäre Suche - **Rekursiv**



```
public int search(int elem, int[] a, int low, int high) {  
    if (low > high) return -1; // empty  
    int m = (low + high) / 2;  
    if (elem == a[m]) return m;  
    if (elem < a[m])  
        return search(elem, a, low, m-1);  
    return search(elem, a, m+1, high);  
}
```

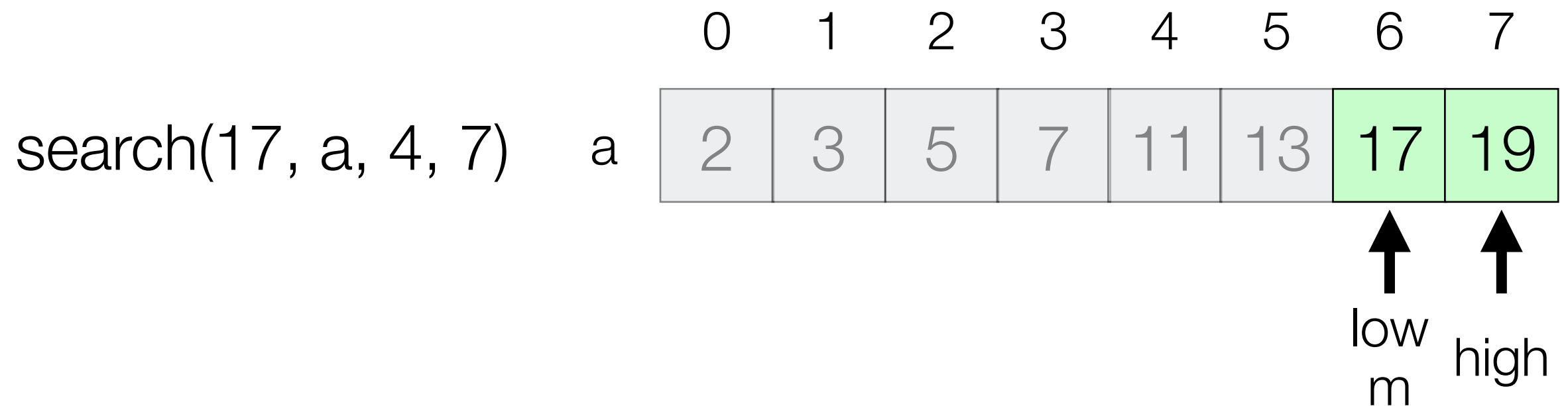
Binäre Suche - **Rekursiv**



```
public int search(int elem, int[] a, int low, int high) {  
    if (low > high) return -1; // empty  
    int m = (low + high) / 2;  
    if (elem == a[m]) return m;  
    if (elem < a[m])  
        return search(elem, a, low, m-1);  
    return search(elem, a, m+1, high);  
}
```



Binäre Suche - **Rekursiv**



```
public int search(int elem, int[] a, int low, int high) {  
    if (low > high) return -1; // empty  
    int m = (low + high) / 2;  
    if (elem == a[m]) return m; ←  
    if (elem < a[m])  
        return search(elem, a, low, m-1);  
    return search(elem, a, m+1, high);  
}
```

Zeitkomplexität

```
public int search(int elem, int[] a, int low, int high) {  
    if (low > high) return -1; // empty  
    int m = (low + high) / 2;  
    if (elem == a[m]) return m;  
    if (elem < a[m])  
        return search(elem, a, low, m-1);  
    return search(elem, a, m+1, high);  
}
```

$\Rightarrow O(\log n)$

```
public int search(int elem, int[] a, int low, int high) {  
    while(low <= high){  
        int m = (low + high) / 2;  
        if (elem == a[m]) return m;  
        if (elem < a[m])  
            high = m-1;  
        else  
            low = m+1;  
    }  
    return -1;  
}
```

$\Rightarrow O(\log n)$

Zeitkomplexität

Rekursion i.d.R.
dennoch
langsamer.
Siehe oben.



```
int low, int high) {  
    empty
```

```
    m-1);  
    h);
```

$\Rightarrow O(\log n)$

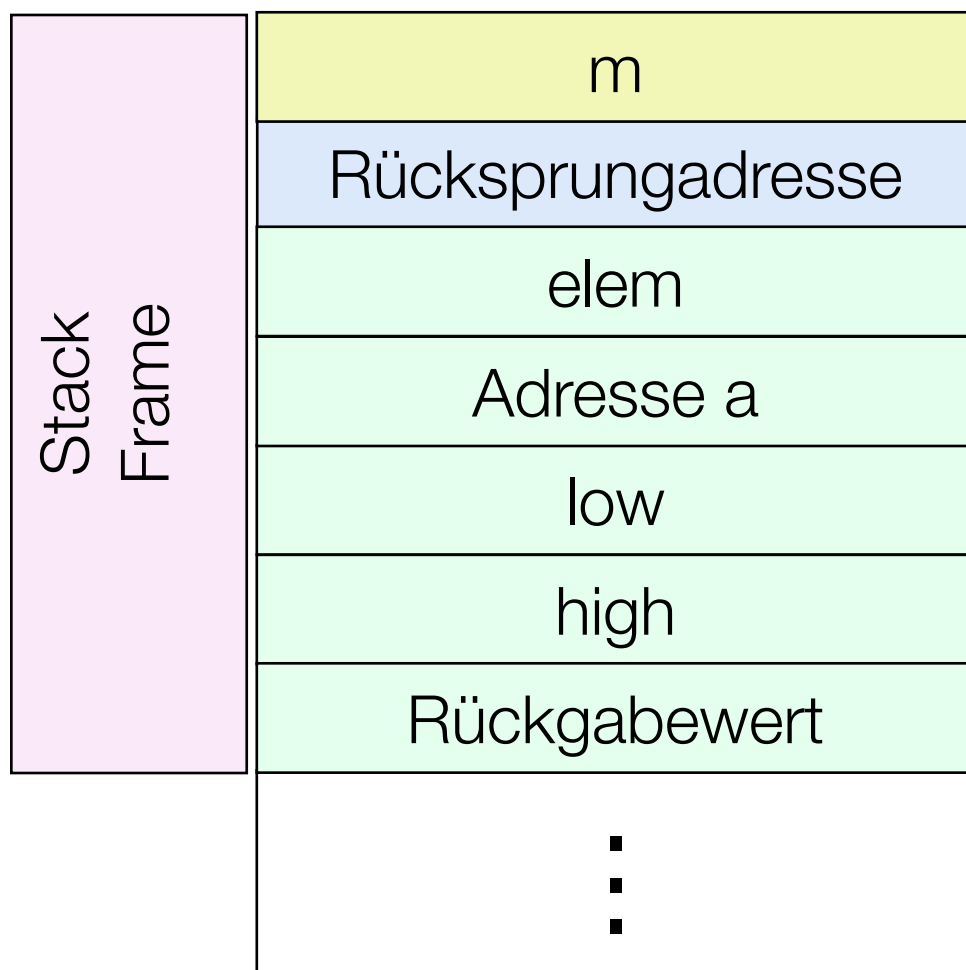
```
int low, int high) {
```

$O(\log n)$

Speicherkomplexität - Iterativ

$7 + \text{size}(a)$

$\Rightarrow O(1)$

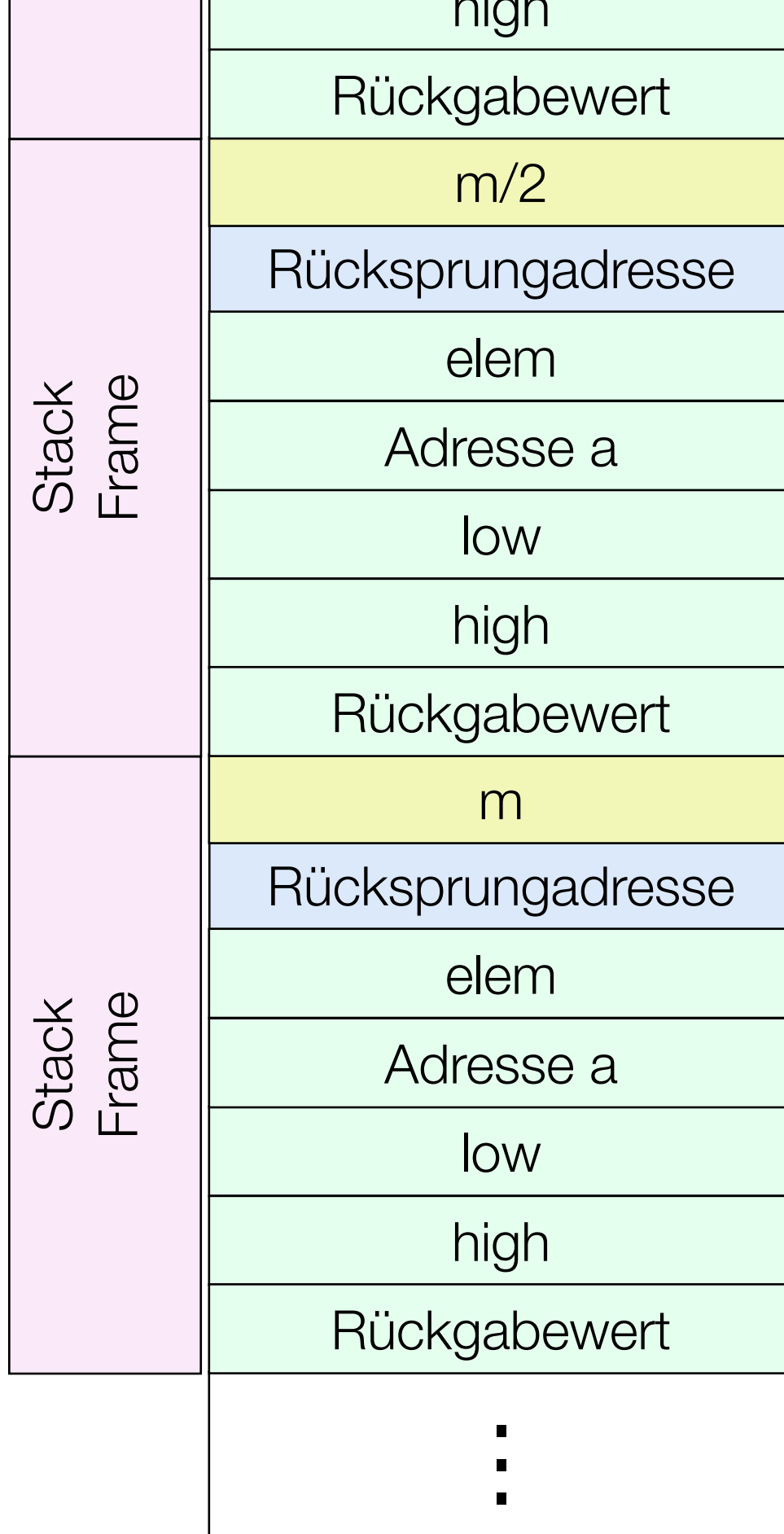


Call Stack

```
public int search(int elem, int[] a, int
low, int high) {
    while(low <= high){
        int m = (low + high) / 2;
        if (elem == a[m]) return m;
        if (elem < a[m])
            high = m-1;
        else
            low = m+1;
    }
    return -1;
}
```


Speicherkomplexität

-Rekursiv



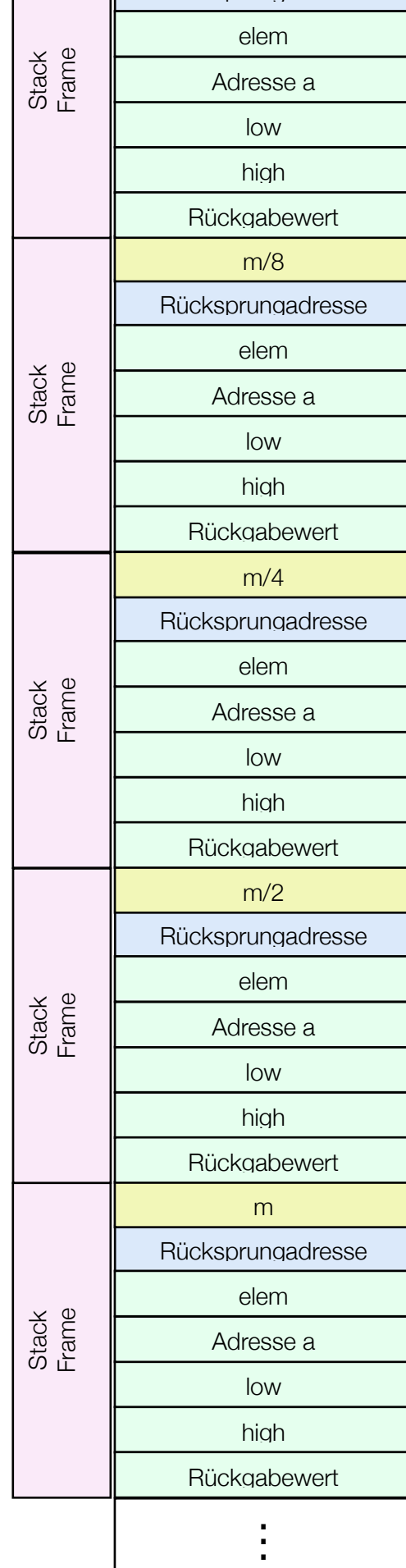
Call Stack

```
public int search(int elem, int[] a, int
low, int high) {
    if (low > high) return -1; // empty
    int m = (low + high) / 2;
    if (elem == a[m]) return m;
    if (elem < a[m])
        return search(elem, a, low, m-1);
    return search(elem, a, m+1, high);
}
```

Speicherkomplexität

-Rekursiv

$O(\text{Rekursionstiefe})$
 $\Rightarrow O(\log n)$

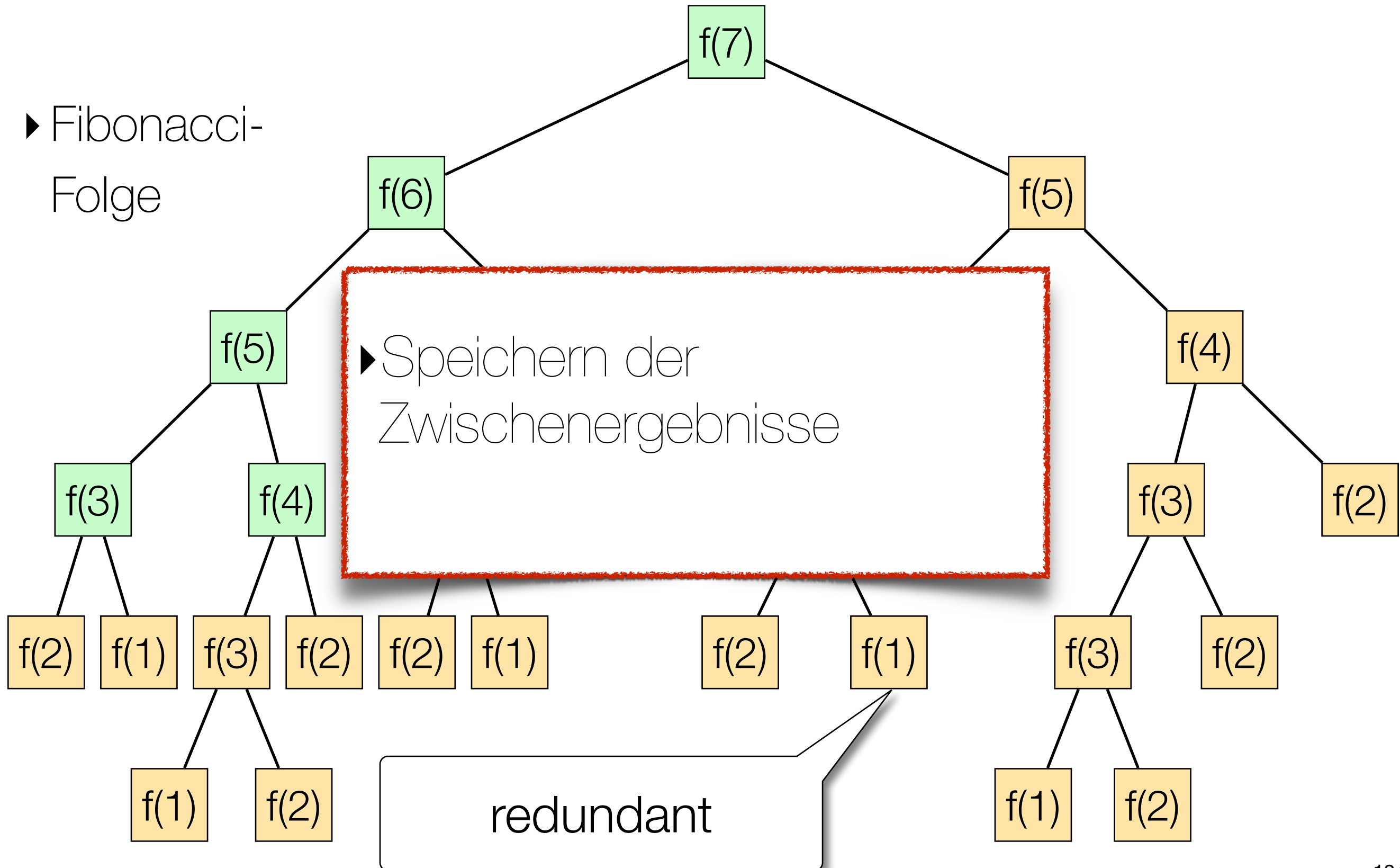


Call Stack

```
public int search(int elem, int[] a, int
low, int high) {
    if (low > high) return -1; // empty
    int m = (low + high) / 2;
    if (elem == a[m]) return m;
    if (elem < a[m])
        return search(elem, a, low, m-1);
    return search(elem, a, m+1, high);
}
```

Dynamische Programmierung

► Fibonacci-Folge



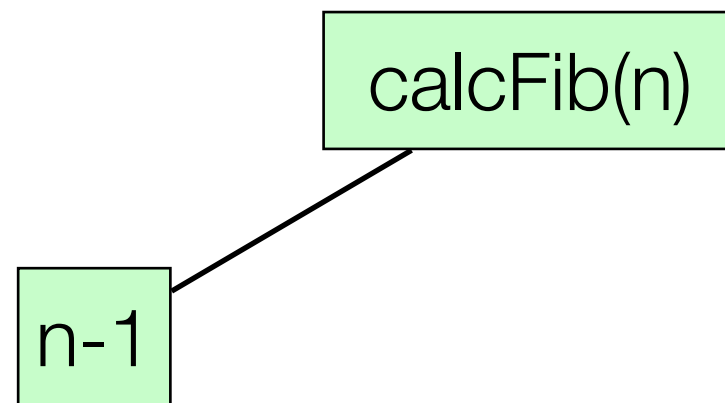
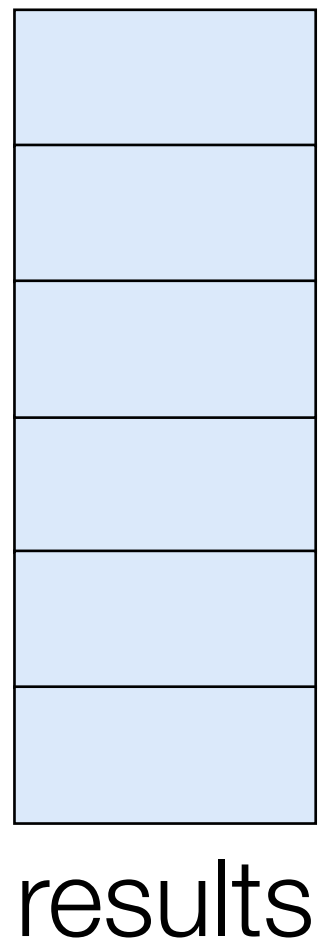
Fibonacci-Folge

Dynamisch

```
public int calcFib(int n)
{
    if(n <= 2)
        return 1;
    else
        return calcFib(n-1) + calcFib (n-2);
}
```

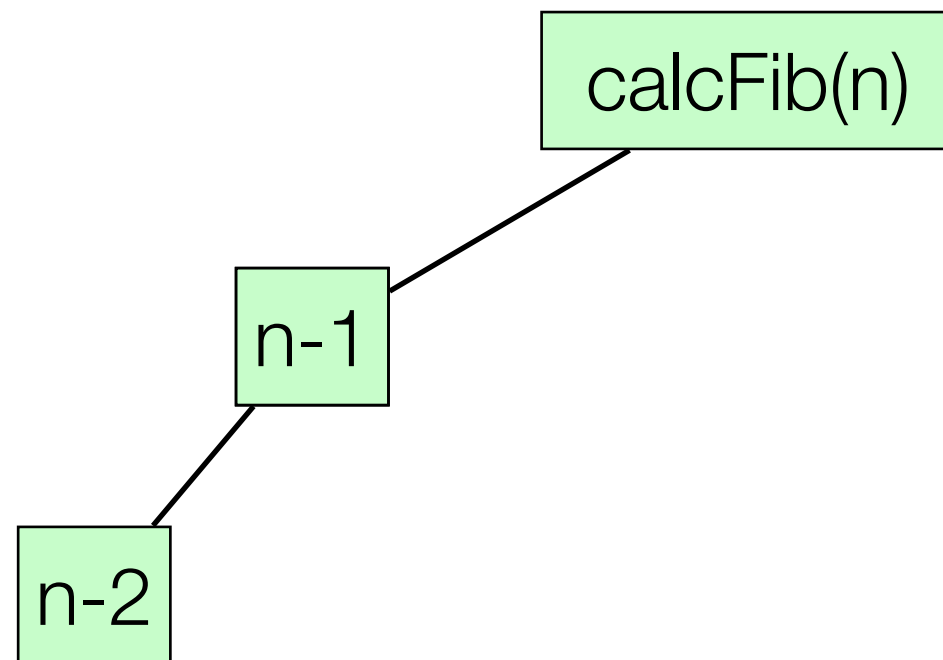
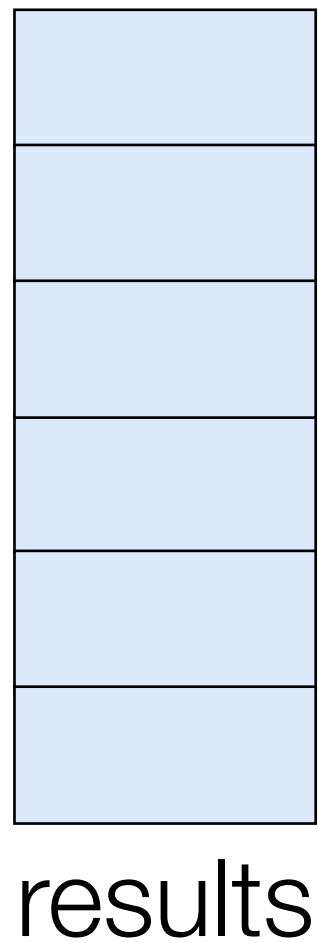
Fibonacci-Folge

Dynamisch



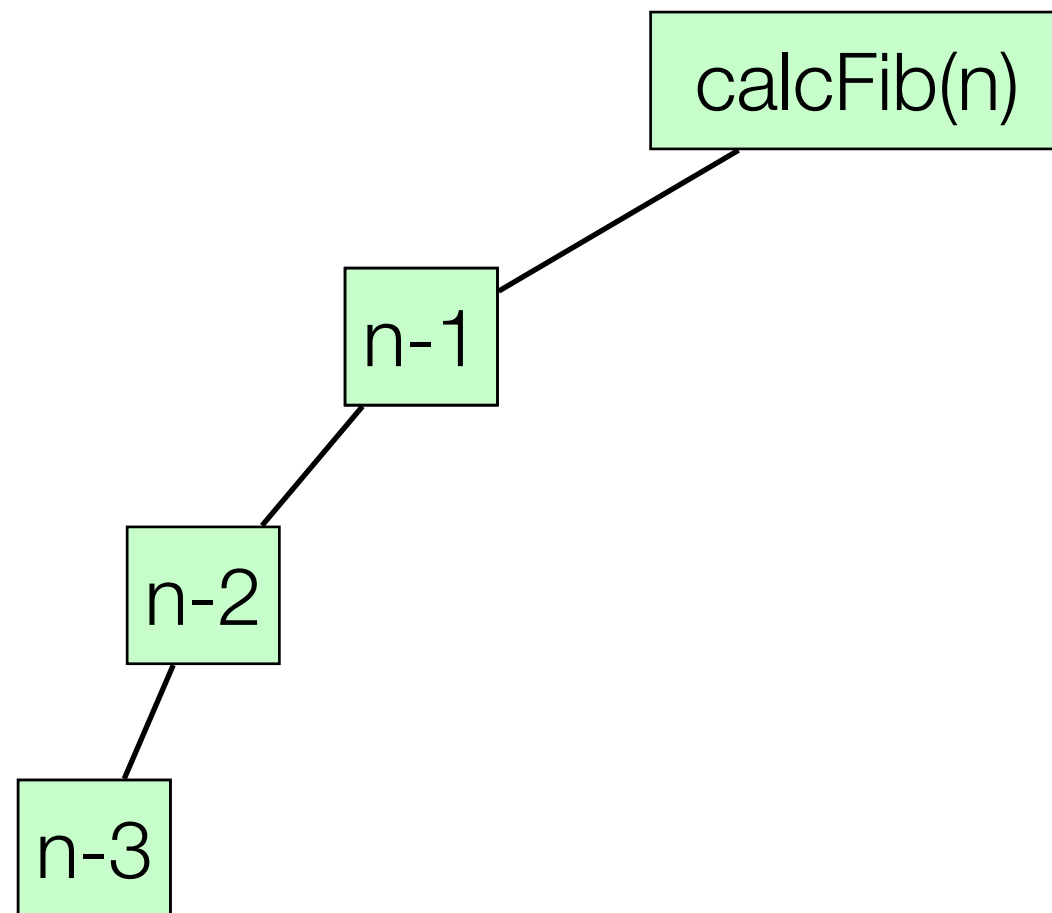
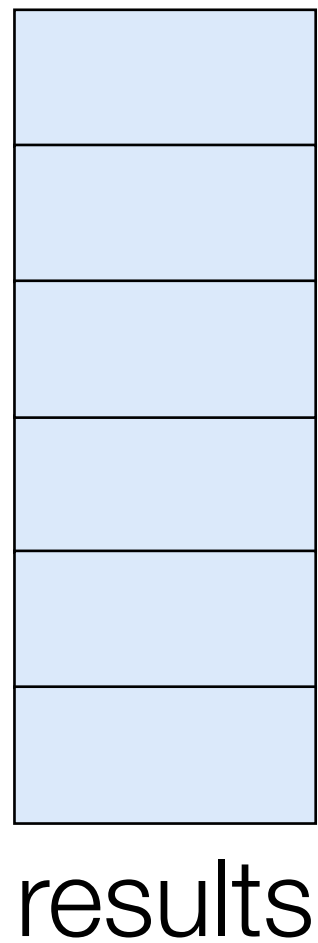
Fibonacci-Folge

Dynamisch



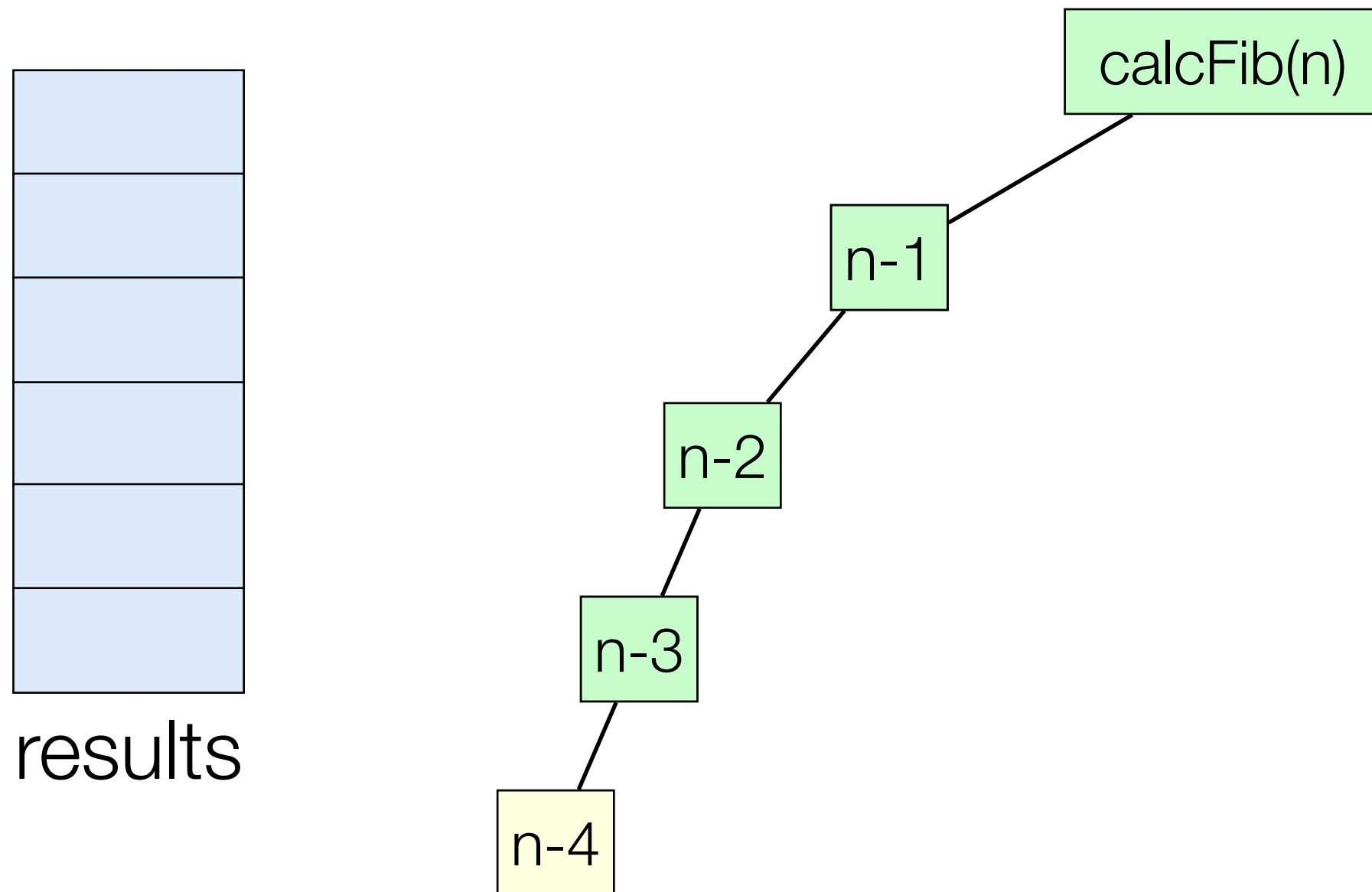
Fibonacci-Folge

Dynamisch



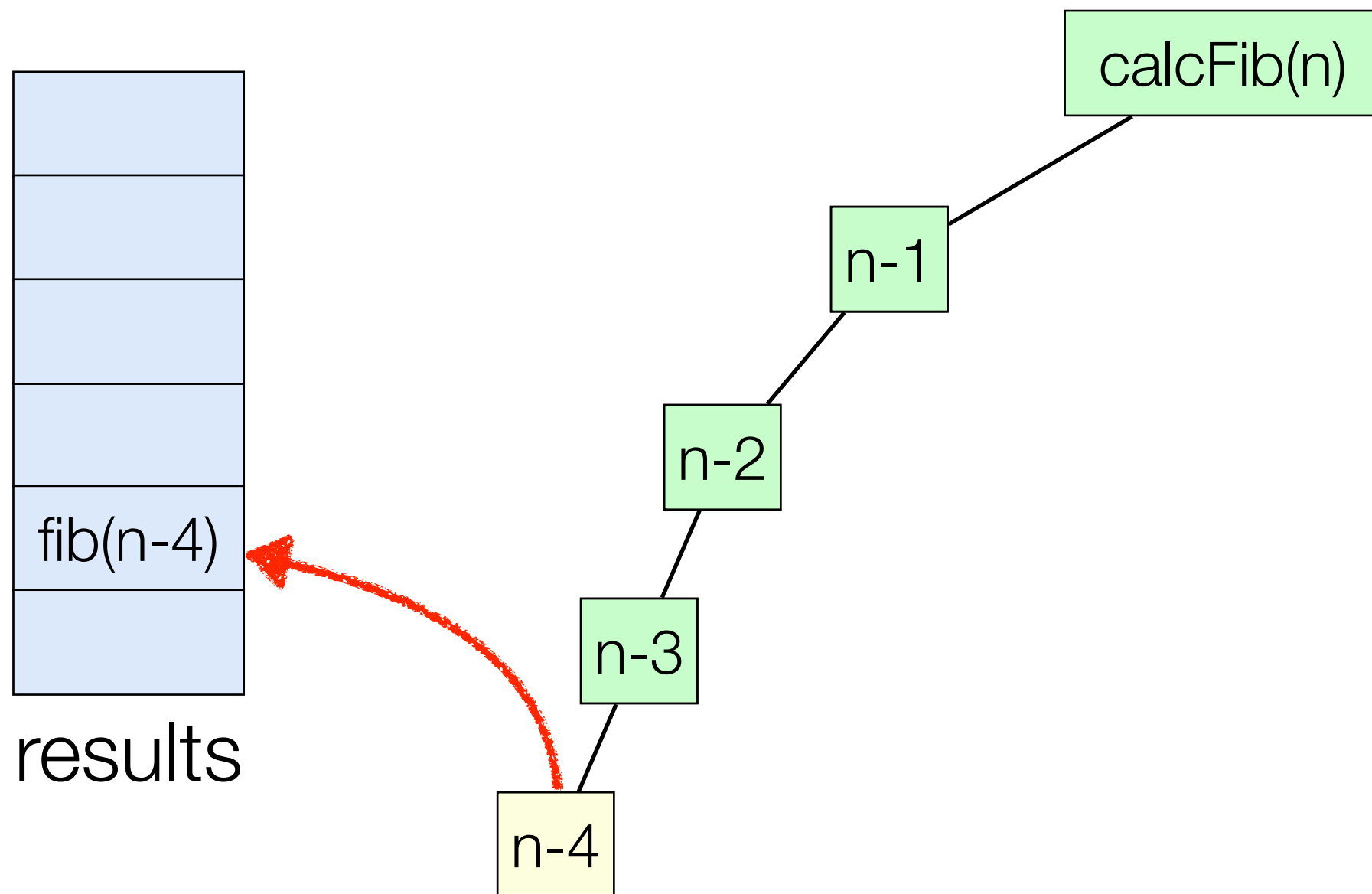
Fibonacci-Folge

Dynamisch



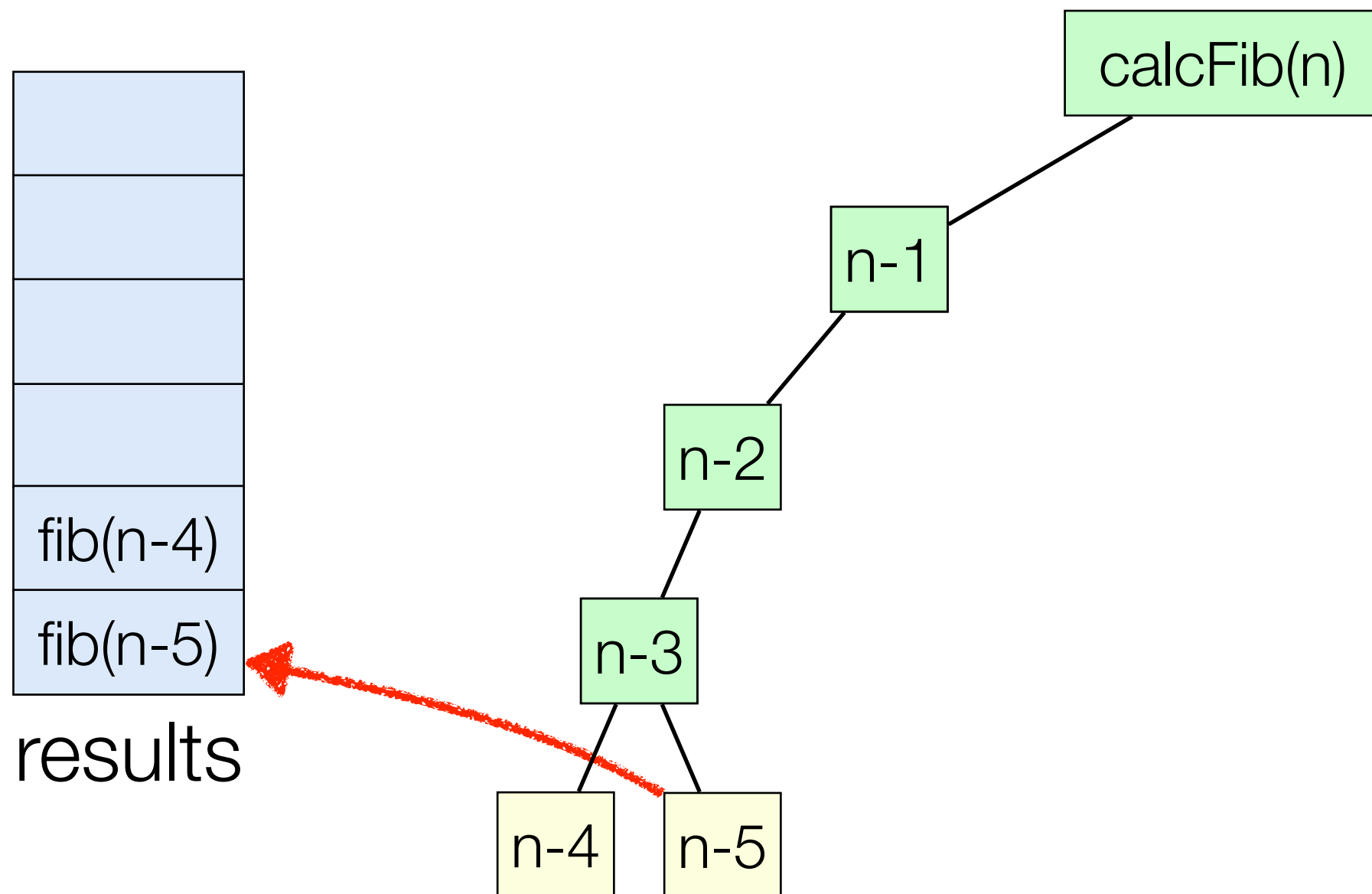
Fibonacci-Folge

Dynamisch



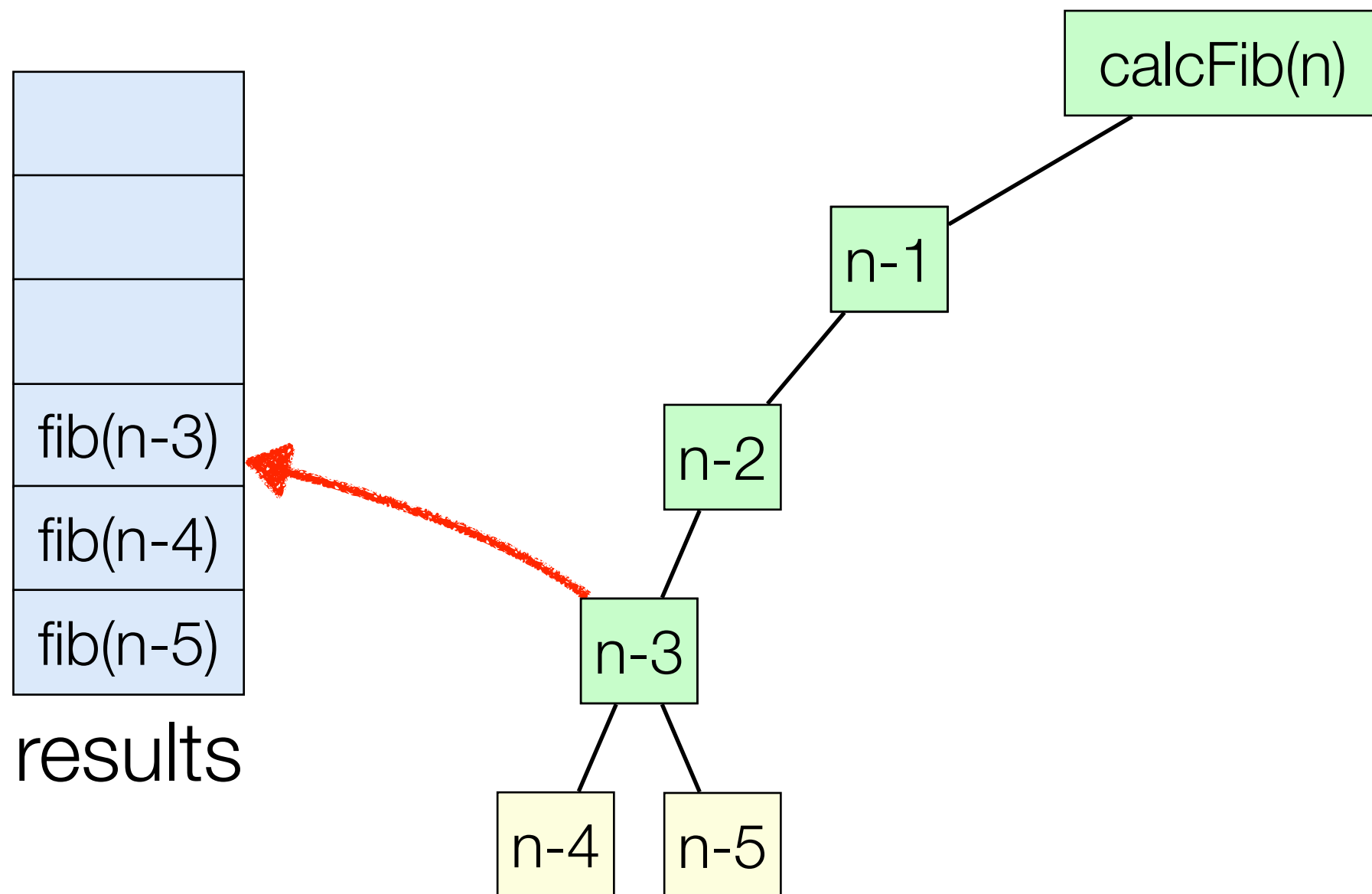
Fibonacci-Folge

Dynamisch



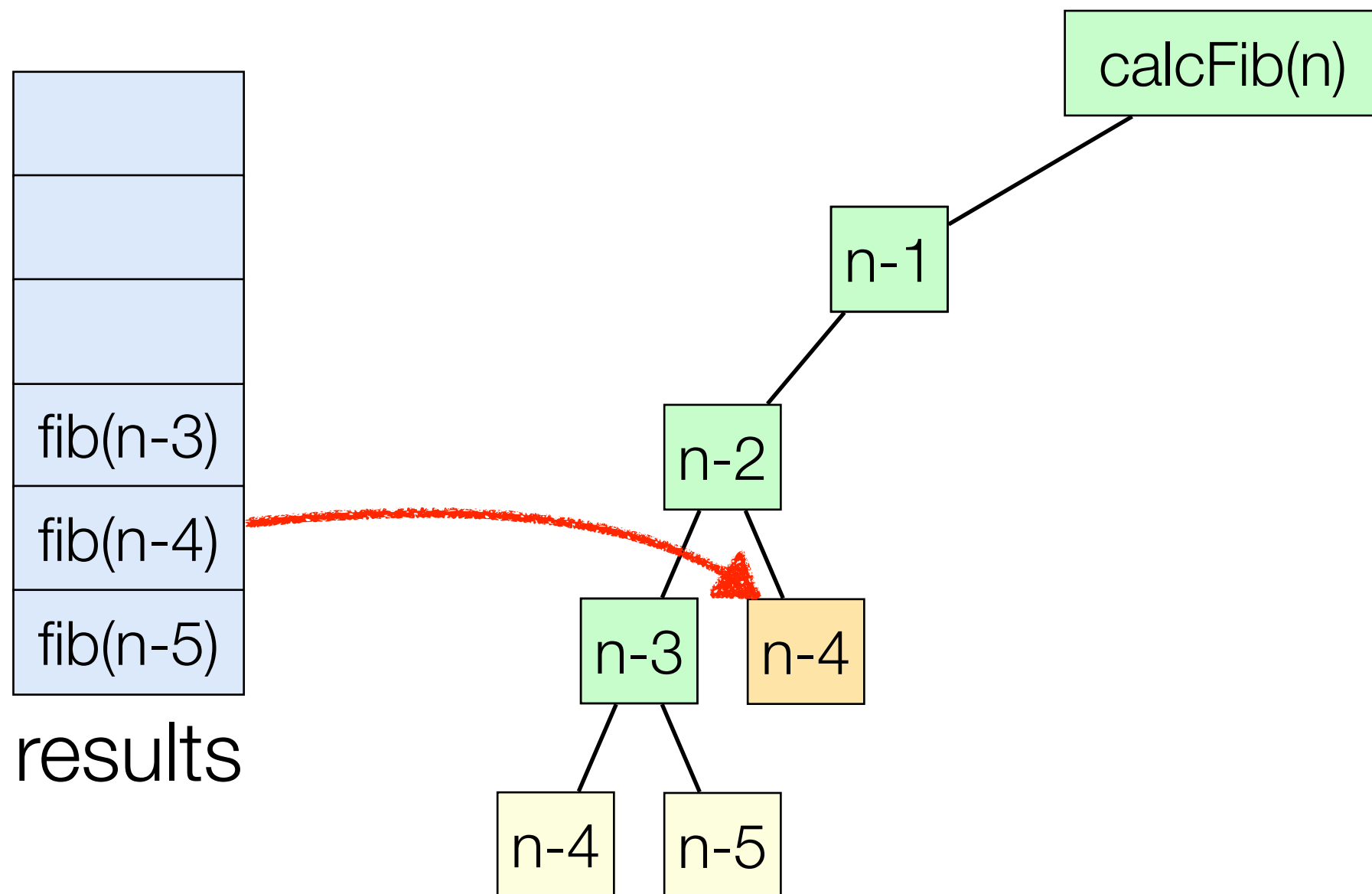
Fibonacci-Folge

Dynamisch



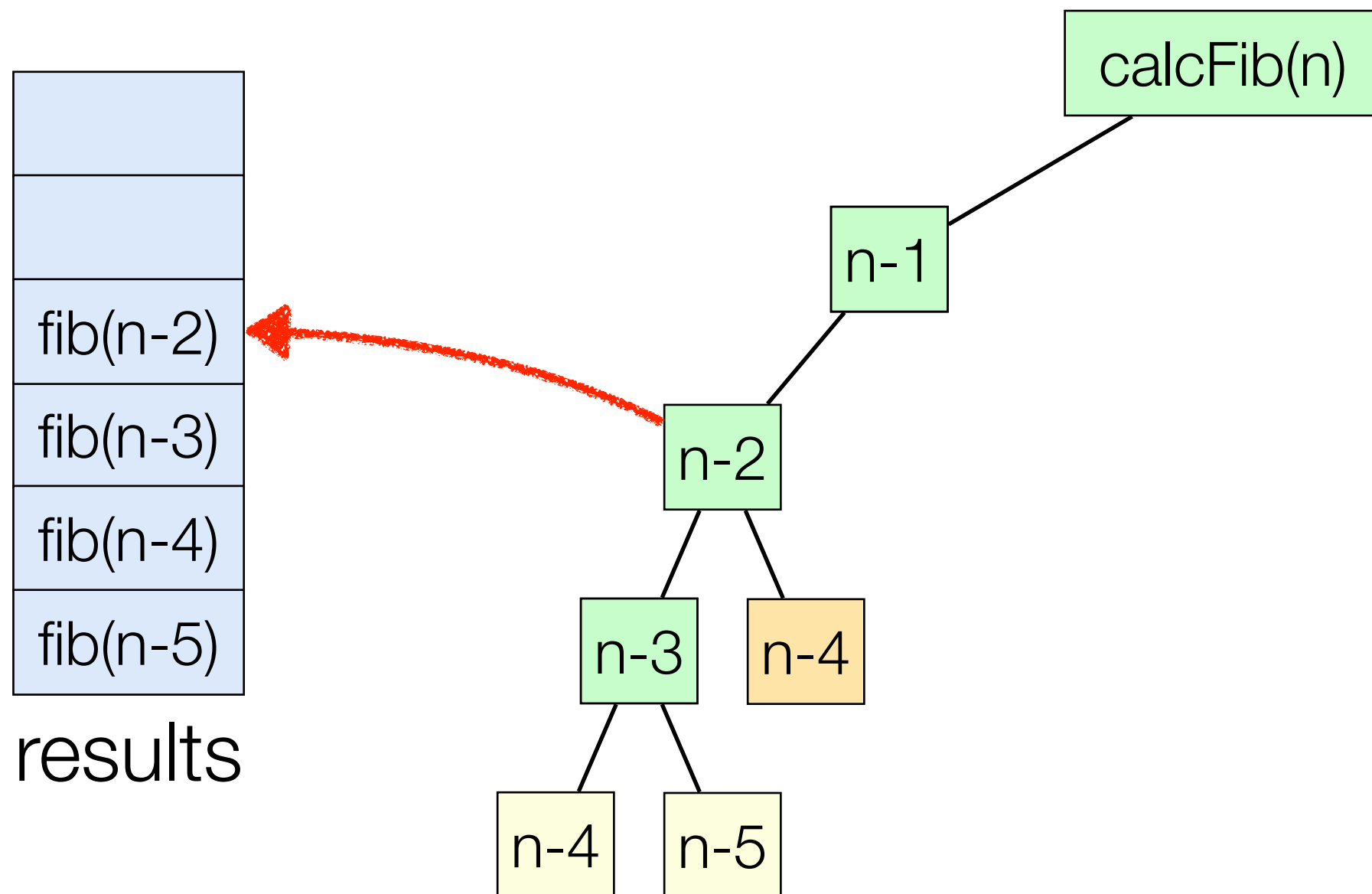
Fibonacci-Folge

Dynamisch



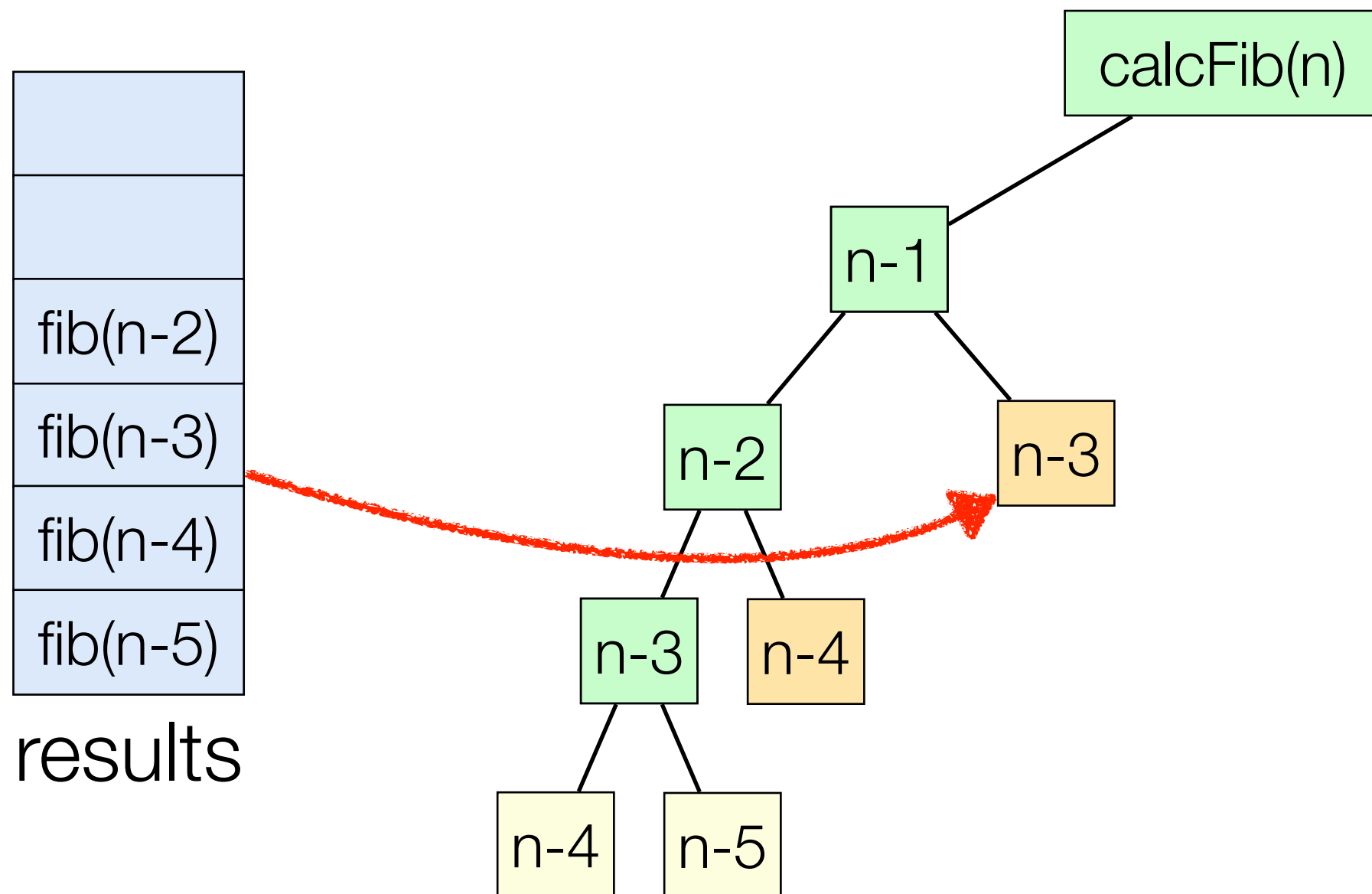
Fibonacci-Folge

Dynamisch



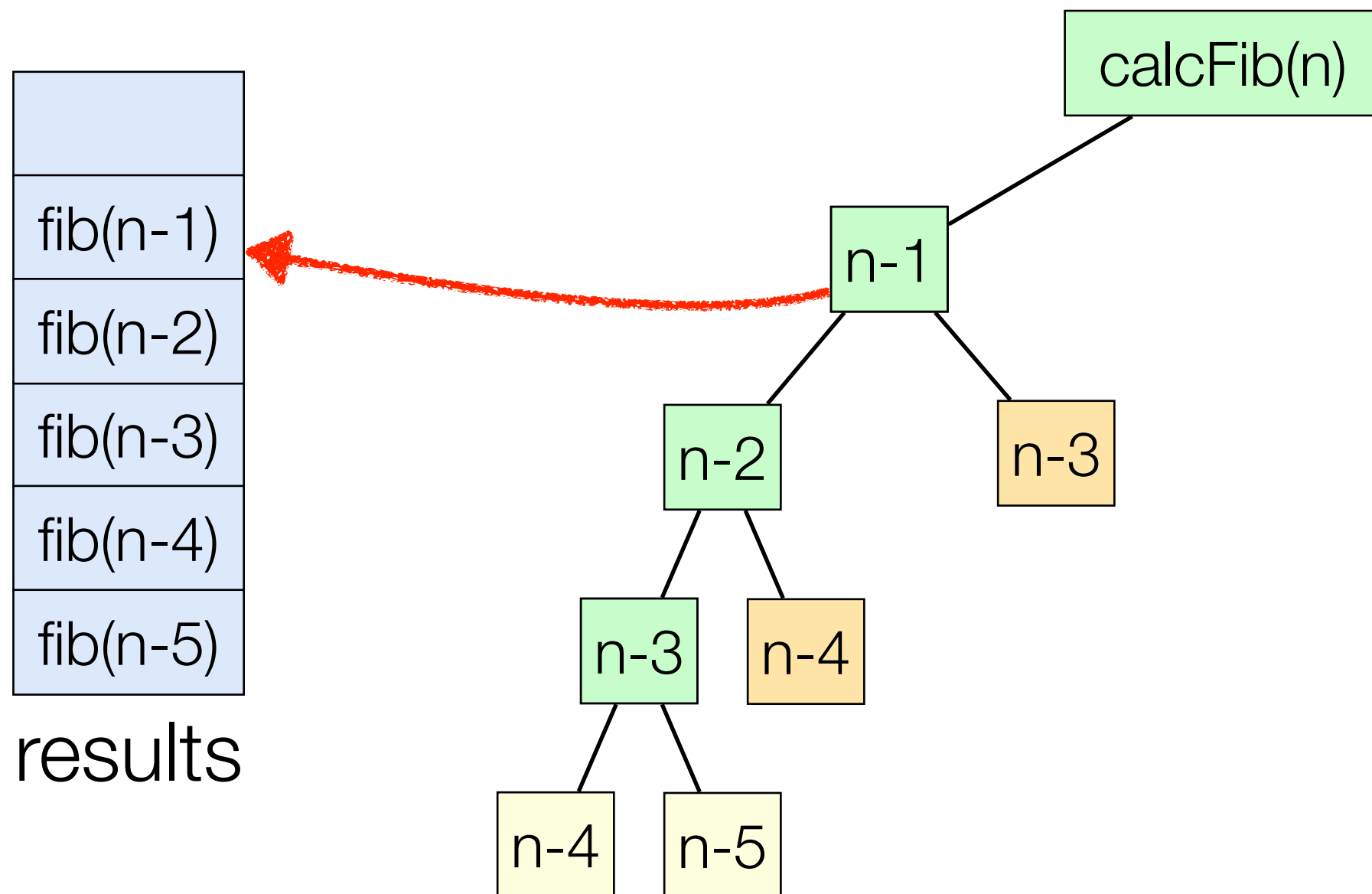
Fibonacci-Folge

Dynamisch



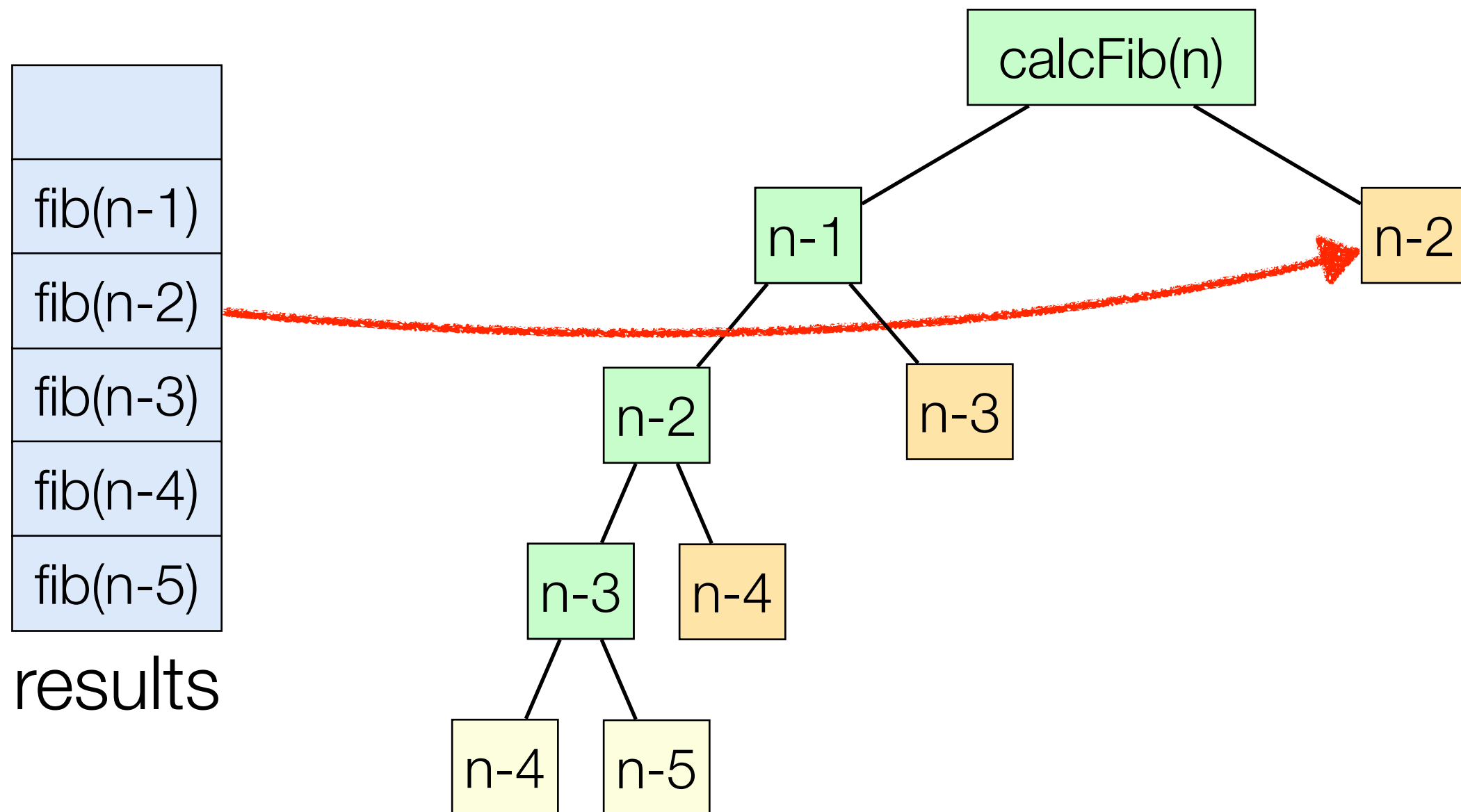
Fibonacci-Folge

Dynamisch



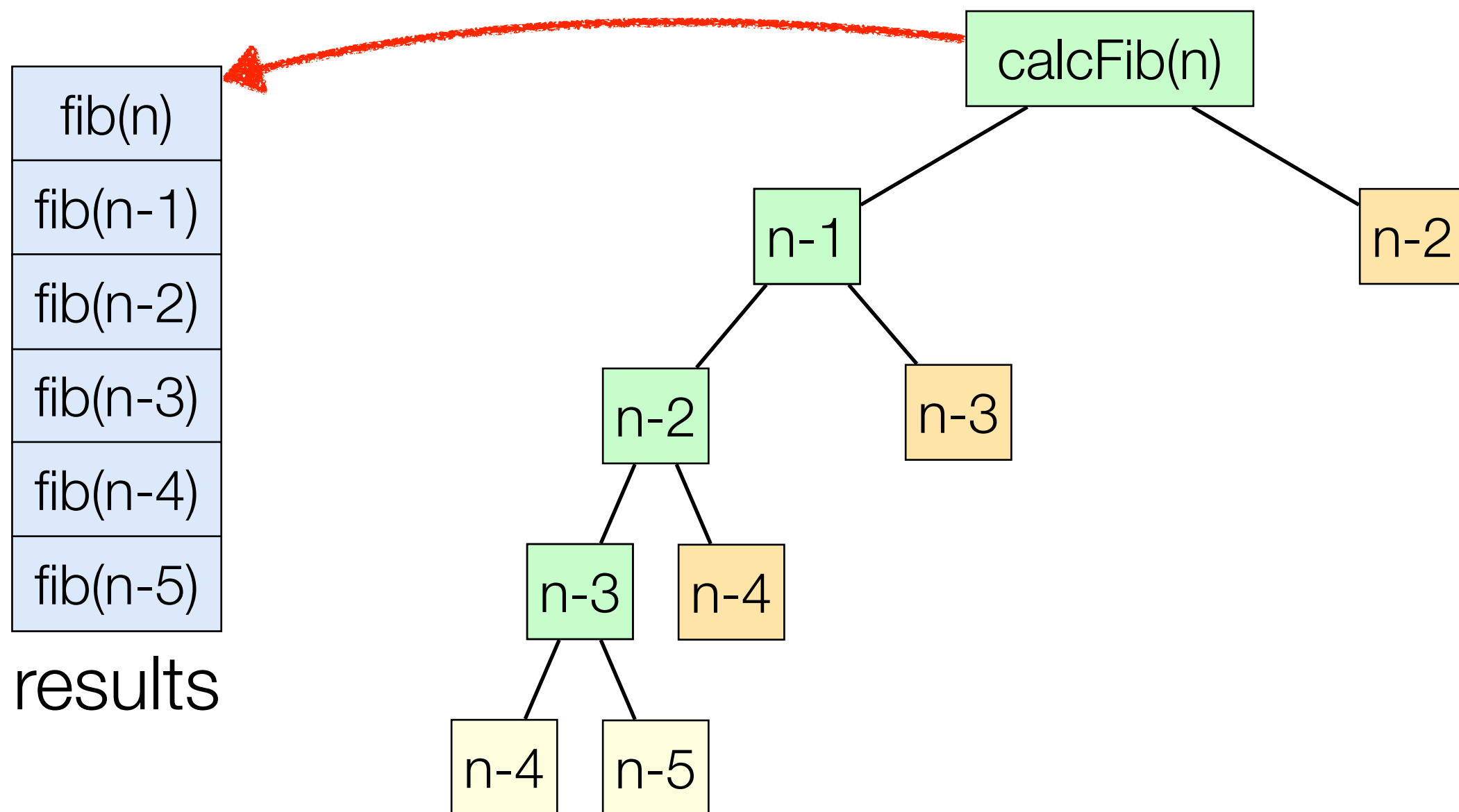
Fibonacci-Folge

Dynamisch



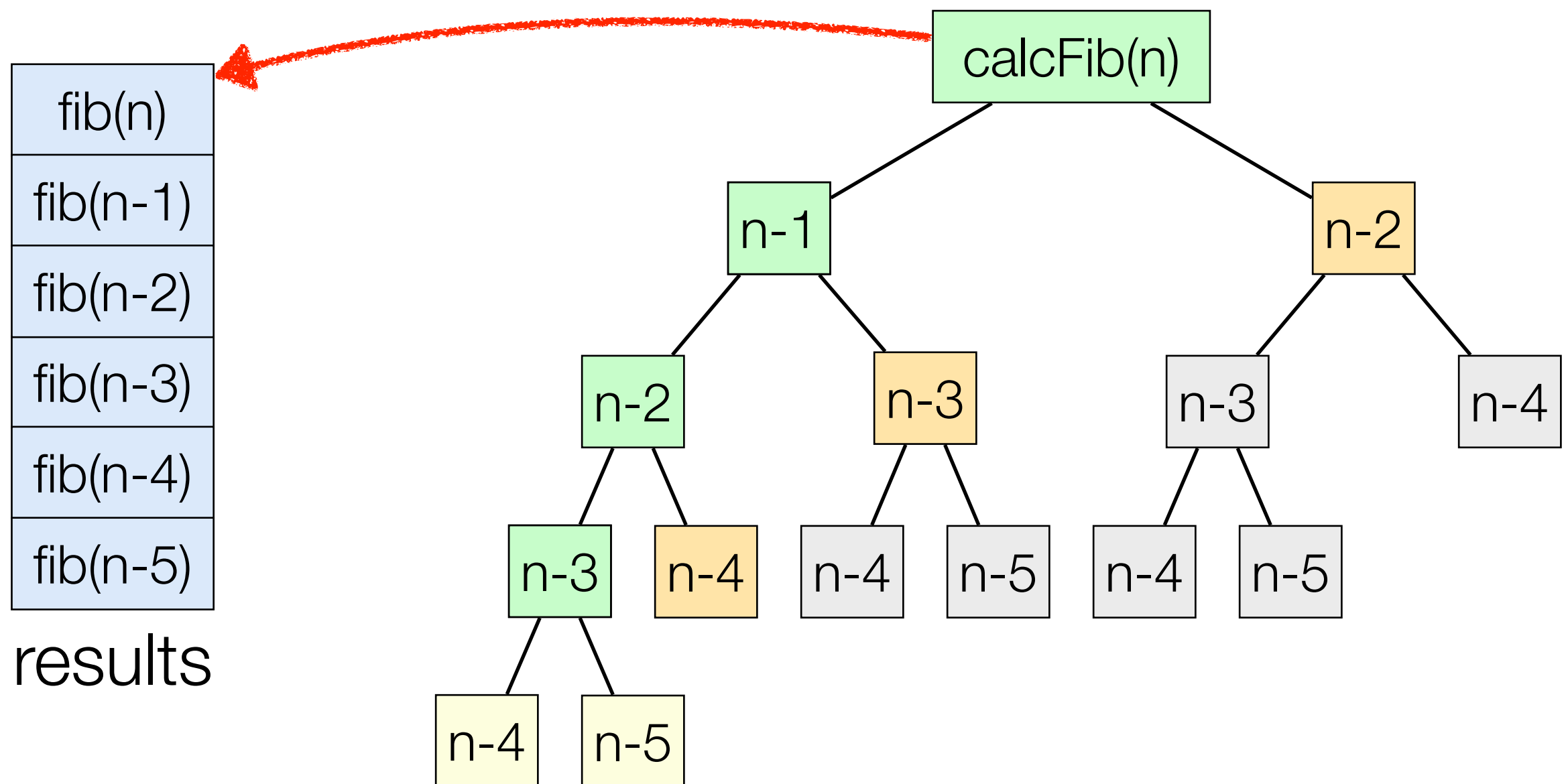
Fibonacci-Folge

Dynamisch



Fibonacci-Folge

Dynamisch



Fibonacci-Folge

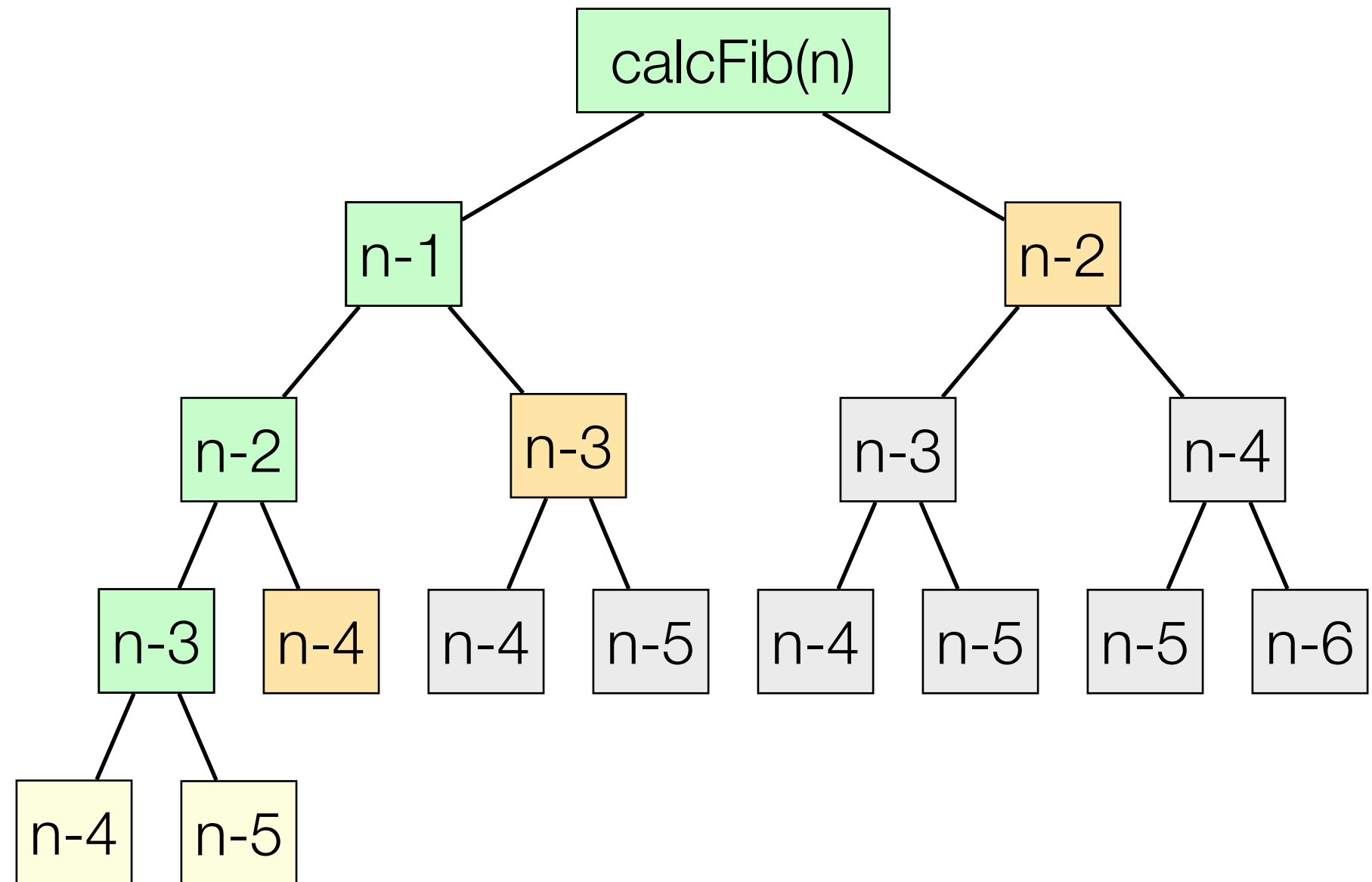
Dynamisch

```
public class Dynamic extends Fibonacci{
    private long[] fibs;
    public long calcFib(int n)
    {
        if(fibs == null){
            fibs = new long[n];
            fibs[0] = 1; //fib(1)
            fibs[1] = 1; //fib(2)
        }
        if(fibs[n-1] != 0)
            return fibs[n-1];
        else
            fibs[n-1] = calcFib(n-1) + calcFib(n-2);
        return fibs[n-1];
    }
}
```

Zeitkomplexität

fib(n)
fib(n-1)
fib(n-2)
fib(n-3)
fib(n-4)
fib(n-5)

results

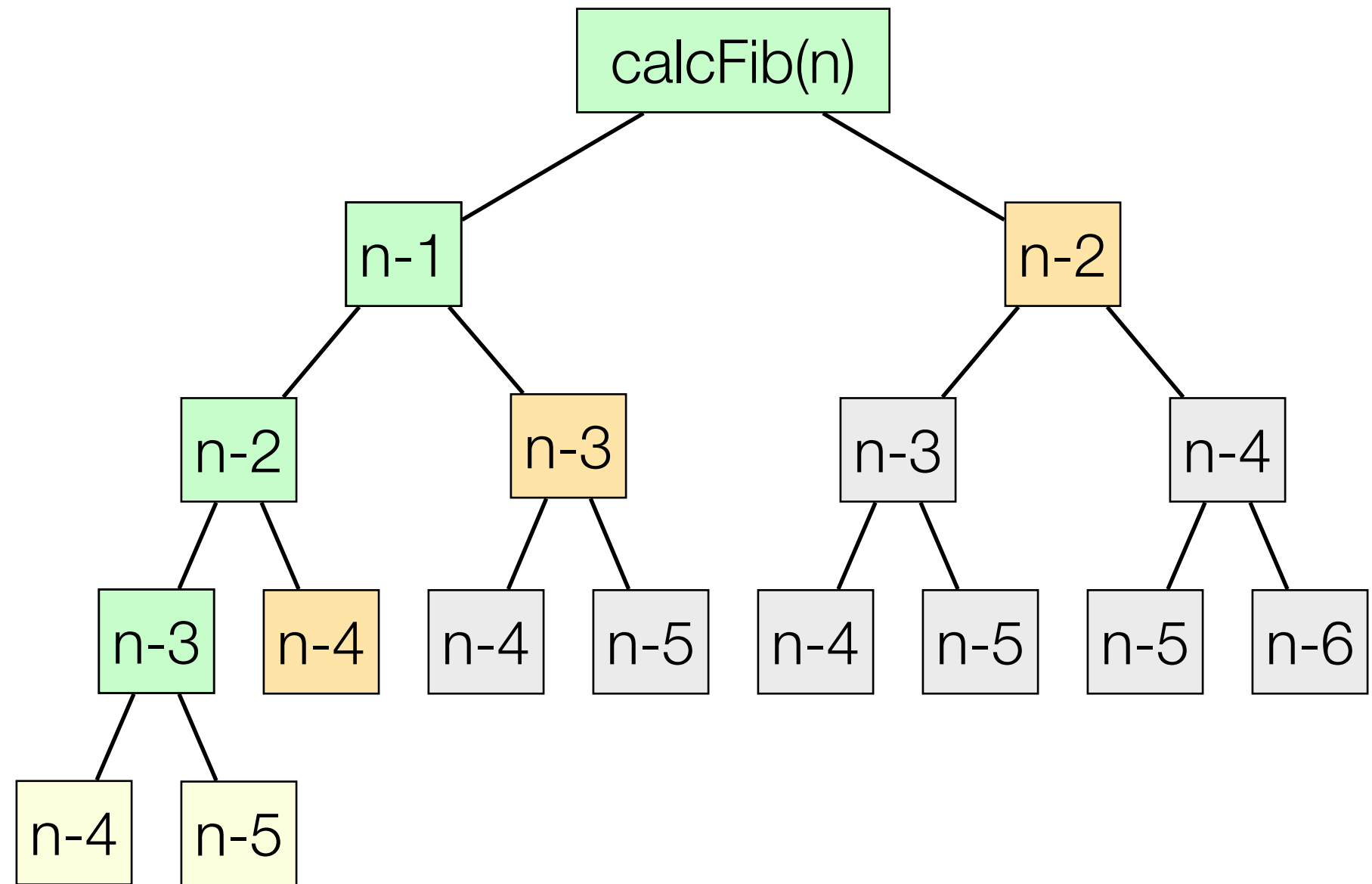


$\Rightarrow O(n)$

Speicherkomplexität

fib(n)
fib(n-1)
fib(n-2)
fib(n-3)
fib(n-4)
fib(n-5)

results



Stack + Array $\Rightarrow O(n)$

$\downarrow$
 $O(n)$

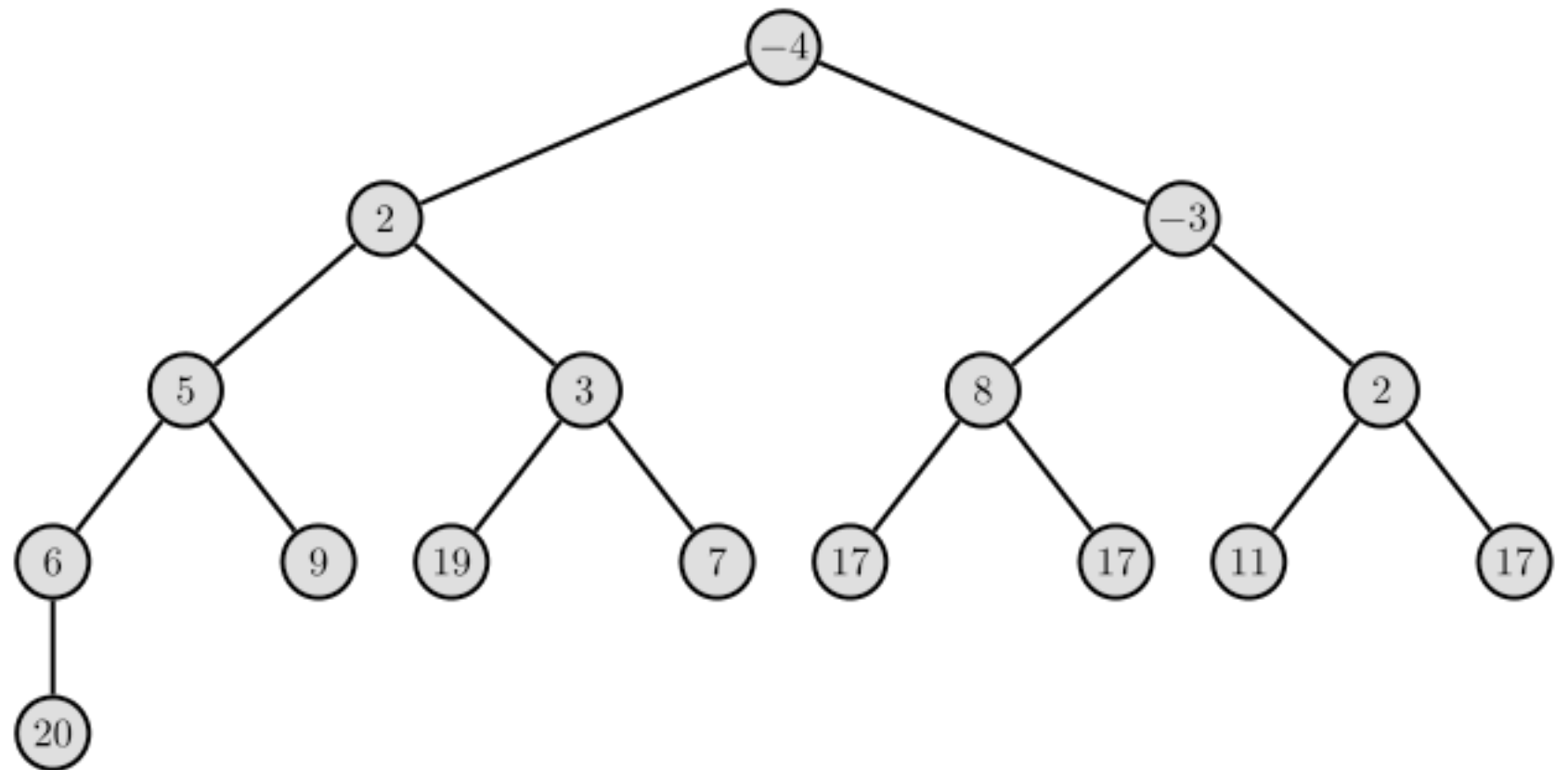
$\downarrow$
 $O(n)$

DEMO

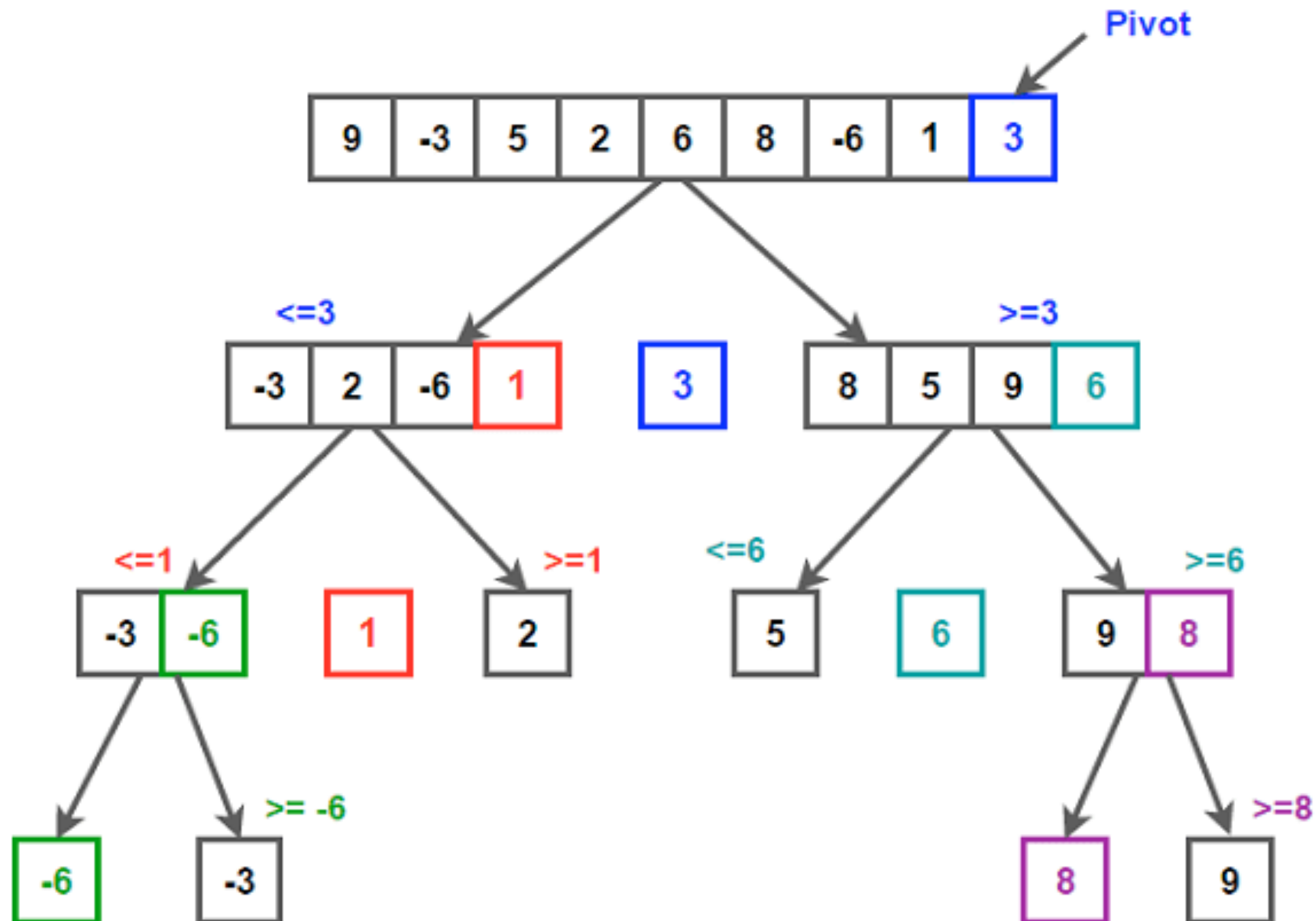
► **FibonacciSpeedtest.java**

Rekursive **Algorithmen** und **Datenstrukturen**

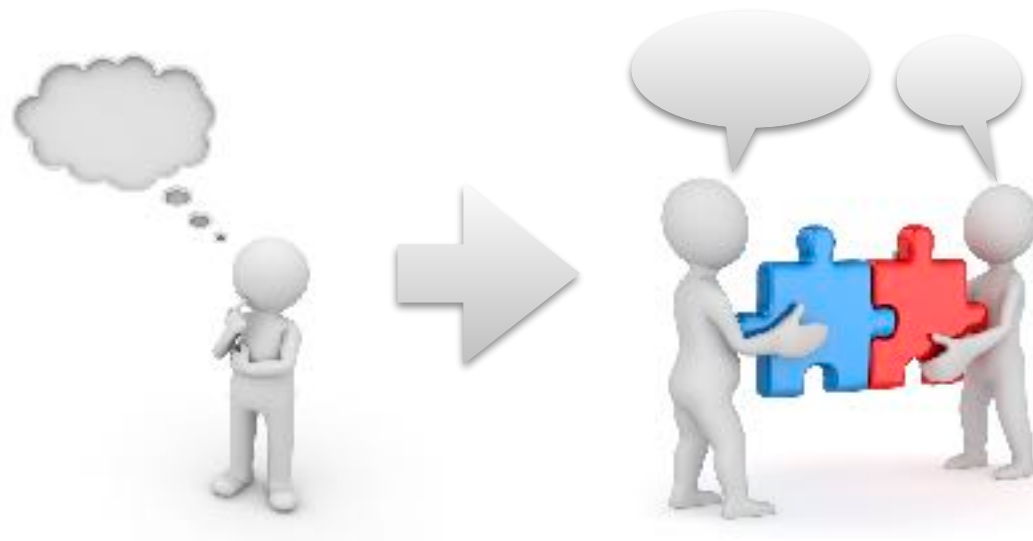
- ▶ Quicksort
- ▶ Mergesort
- ▶ Divide and Conquer Algorithmen
- ▶ Bäume
- ▶ Listen
- ▶ Heaps



Beispiel Quicksort



Beispiel Quicksort



1. Bilden Sie mit ihrem Nachbarn eine 2er-Gruppe.
 - I. Einer liest die rekursive Quicksort-Implementierung, der andere die iterative Implementierung. (2 Minuten)
 - II. Erklären sie sich gegenseitig die Implementierung . (2x1 Minute)
2. Welcher Code war besser verständlich?

Programmierung 2 (PIB-PRG2/DFIW-PRG2) / Code / Code-Beispiele Vorlesung

Code-Beispiele Vorlesung

[Verzeichnis](#) [Einstellungen](#) [Mehr ▾](#)

Wichtiger Hinweis: Die hier bereitgestellten Implementierungen sind nicht immer vollständig. Die Implementierungen fokussieren auf den jeweils darzustellenden Aspekt! Daher fehlen häufig Dinge wie Prüfung der Übergabeparameter, Ausnahmebehandlung etc. Bitte verstehen Sie die Implementierungen nicht als vollumfängliche Musterprogramme sondern im Sinne von kleinen Codebeispielen, die einen bestimmten Aspekt darstellen sollen.

[Bearbeiten](#) [Verzeichnis herunterladen](#)

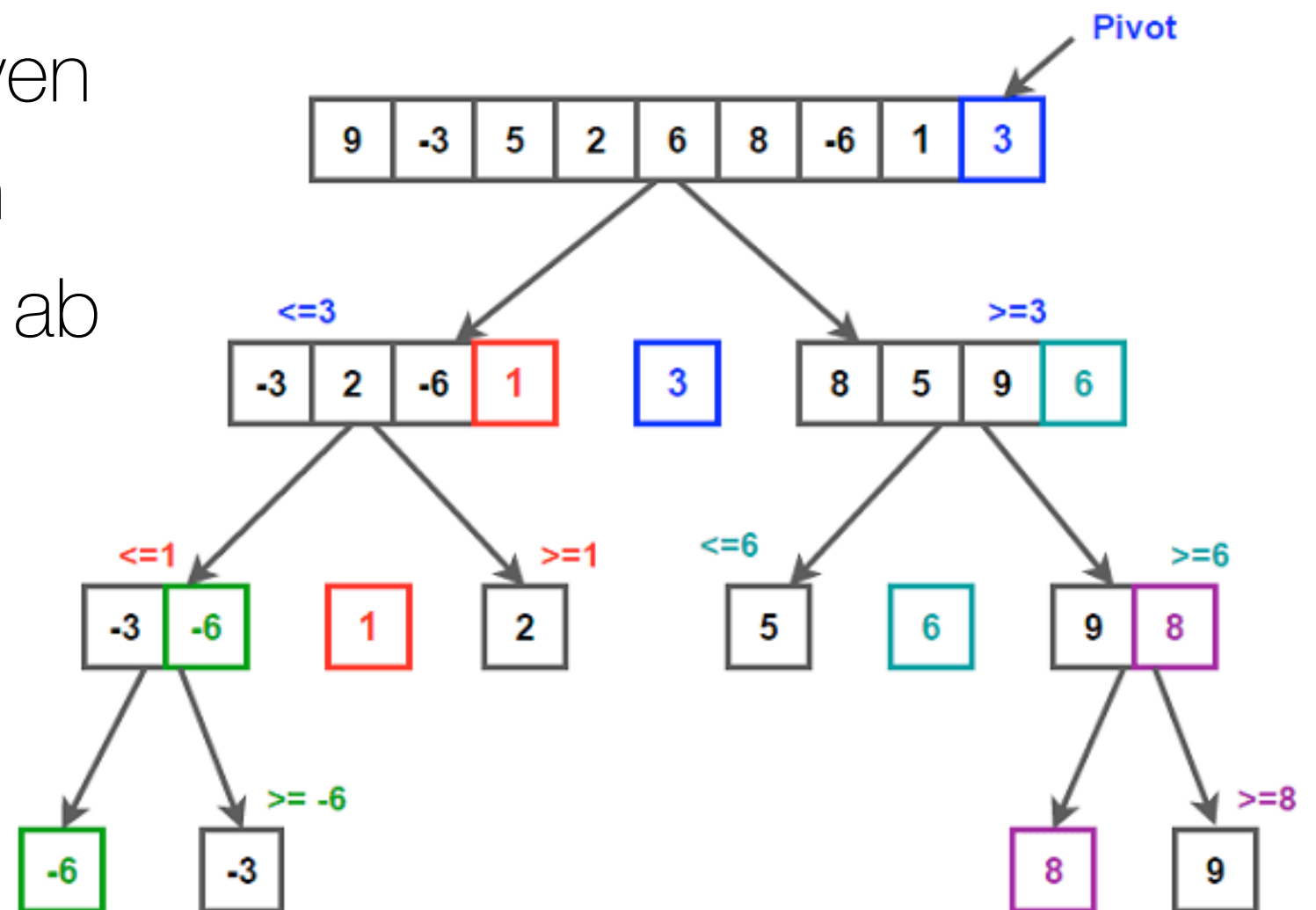
Recursion

DEMO

► **QuicksortSpeedtest.java**

Beispiel Quicksort

- ▶ Quicksort-Implementierung benötigt eine Stack-Datenstruktur
 - ▶ Call Stack ist ein effizienter Stack
 - ▶ Performance der iterativen Lösung hängt stark von Stack-Implementierung ab



PROS

CONS

Rekursion

- ▶ kürzere Formulierung
- ▶ leichter verständliche Lösung
- ▶ teilweise sehr effiziente Problemlösungen (z.B. Quicksort)
- ▶ Bei rekursiven Datenstrukturen (zum Beispiel Bäume, Graphen) besonders empfehlenswert

- ▶ weniger effizientes Laufzeitverhalten (Overhead bei Funktionsaufruf)
- ▶ Verständnisprobleme bei Programmieranfängern
- ▶ höhere Speicherkomplexität

Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was ist Rekursion und wo liegen Vor- und Nachteile?
- Wie unterscheiden sich rekursive- und iterative Implementierungen?
- Wie unterscheiden sich Zeit- und Speicherkomplexität iterativer- und rekursiver Lösungen?
- Was ist eine dynamische Implementierung?



45 Sekunden

Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was ist Rekursion und wo liegen Vor- und Nachteile?
- Wie unterscheiden sich rekursive- und iterative Implementierungen?
- Wie unterscheiden sich Zeit- und Speicherkomplexität iterativer- und rekursiver Lösungen?
- Was ist eine dynamische Implementierung?



45 Sekunden

Kahoot!

go to
kahoot.it

PIN
eingeben



Nickname
eingeben



mitspielen



1

2

3

4

